

Data Science 101: Breaking Up with Excel

September 21-23, 2025

MSACL Montreal

Stephen R Master, MD PhD (masters@email.chop.edu)
Department of Pathology and Laboratory Medicine, Children's Hospital of Philadelphia
Nick Spies, MD (nick.spies@aruplab.com) true
Daniel T. Holmes, MD (dtholmes@mail.ubc.ca)
Department of Pathology and Laboratory Medicine, University of British Columbia

Contents

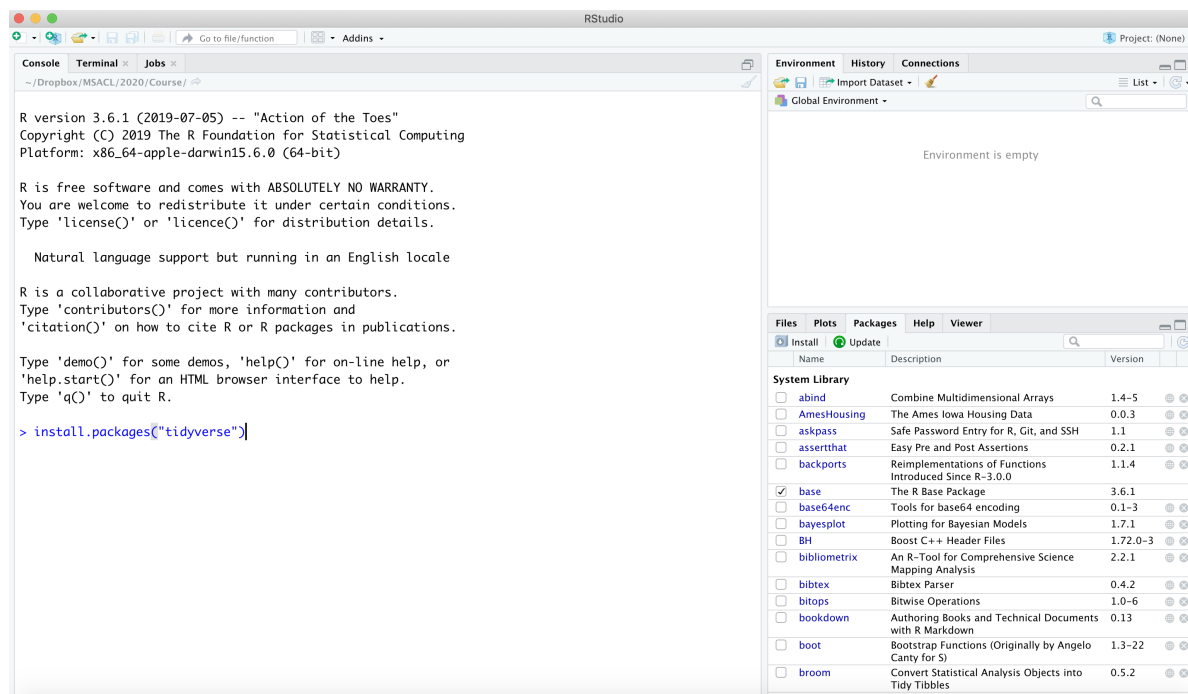
0.1	Preparation for the Course	3
1	Basics and Data Types	3
1.1	What is R?	3
1.2	Algebra	4
1.3	Comments	5
1.4	Variables	5
1.5	Data Types	5
1.6	Vectors	6
2	Matrices, Dataframes and Lists	8
2.1	Matrices	8
2.2	Dataframes	10
2.3	Lists	17
3	Reading in Data and Basic Sanity Checking	18
3.1	The Working Directory	18
3.2	Reading a csv File	19
3.3	Surveying your data	20
3.4	Coping with Non-numeric Data	21
3.5	Simple replacements with <code>str_replace()</code>	21
3.6	Sanity Checking your Data with Simple Visualization	22
4	Regression	25
4.1	Ordinary Least Squares	25
4.2	Weighted Least Squares	28
4.3	Outlier Effects in OLS	30
4.4	Passing Bablok Regression	31
4.5	Deming Regression	32
4.6	Preparing a Difference Plot	33
4.7	Non-Linear Regression	35
5	Things with Strings and Tools for Data Cleansing	40
5.1	Working with Strings	40

5.2	Oh <code>grep()</code> , how I love thee. Let me count the “wheys”	43
6	Meet the ‘tidyverse’ - Pivot, Join, Filter, and Clean	45
6.1	Packages	45
6.2	<code>pivot_longer()</code> and <code>pivot_wider()</code>	46
6.3	<code>arrange()</code> , <code>filter()</code> , and <code>select()</code>	49
6.4	<code>join()</code> ME!	73
6.5	A tidy way to handle strings	76
7	Piping, Mutating, and Summarizing	77
7.1	Pipes	77
7.2	<code>mutate()</code> and <code>transmute()</code>	77
7.3	<code>separate()</code> and <code>unite()</code>	79
7.4	<code>group_by()</code> %>% <code>summarize()</code>	80
7.5	Putting all the pipes together	82
8	Lubridate	83
8.1	A sign of the times: <code>lubridate()</code>	83
9	Functions, Conditional Responses and Loops	87
9.1	Making your own functions	87
9.2	Conditional Responses	88
9.3	Loops-When you need to say the same thing over and over and over	90
10	ggplot	92
10.1	What is ggplot?	92
10.2	Scatterplots	93
10.3	Line Plots	96
10.4	Bar Plots	99
10.5	And more!	101
10.6	Multipanel Plots	102
10.7	Saving a plot	106
11	File Operations	107
11.1	Writing a data frame to a csv file	108
11.2	Writing xlsx files	109
12	Projects	109
12.1	Option 1: Lab Utilization Investigation	109
12.2	Option 2: Clean Up Some Research Data – Bad Medicine	110
12.3	Option 3: Flat File Formatting	110
13	Appendix: Old Skool Base R Plotting	111
13.1	Connecting the dots	121
13.2	A legend from your own mind	126
13.3	Opening a new plot window	126
13.4	Putting more than one plot in a single figure.	126
13.5	Adding text to a graph	128

0.1 Preparation for the Course

This Short Course requires you to bring your own laptop. Plugs will be provided on the tables so that you do not have to rely on your battery life. You must install the necessary software before you arrive. It can take up to 30 mins and you do not want to be stuck doing this in the session because you get behind.

You need to install both the R programming Language and the R-Studio interface to R. While one can use R without R-Studio, we will all use it to make things uniform. The R programming language is available for download from many different places as you will see on the download page <https://cran.r-project.org/mirrors.html>. Rstudio can be downloaded and installed free for personal use from: <https://posit.co/download/rstudio-desktop/>. You want the Open Source Edition.



Throughout this handout, we'll be showing you R code in a shaded box like this

```
R code goes here
```

and output will come be in an unshaded area after two “#” signs

```
## [1] "Here is some R output"
```

1 Basics and Data Types

1.1 What is R?

The purpose of this course is to show you that many of the data management and data analysis task you *wish* you could do in Excel are, in fact, very easy if you move to the statistical programming language R.

R is what is known as a “scripting language”. This means that R does not build portable, stand-alone programs like commercial software we usually purchase. In contrast, R requires the R interpreter to be pre-installed on the computer before we can use any of its utilities. The same is true of other popular scripting languages like Python, PHP, Ruby and JavaScript. In principle this means that if you are using R on a company laptop and you don't have administrative privileges, you are going to need your IT folks to help you install it.

R has been built for Windows, Linux and Mac, which means that R code you write can be run by other people on other systems *without any alterations to the code*, provided that they have installed the R interpreter. There are occasional exceptions to this rule but any code alterations are—most often—trivial. With the command `Sys.info()`, R can tell what kind of system you are using and you can tell R to behave differently based on the underlying operating system.

The programs you write in R are saved as text or “ASCII” files that you save like any old document. You can use any text editor to write your program. This might include programs like Notepad or Notepad++ in the Windows environment or Gedit, eMacs, vi, Nano or Sublime Text in the Linux/Mac environments. In this course we will use a text editor in an environment specifically built for R called RStudio. This program has the benefit that it can both allow you to edit your text and run it without leaving the RStudio program. RStudio has many other convenient features that you will discover should you choose to continue this journey. We could spend a lot of time on the background, but we do not have a great deal of time together so we should just jump right in.

1.2 Algebra

R can act as a calculator. It follows these basic rules or algebra.

Operation	Expression	R Code
Addition	$x + y$	<code>x + y</code>
Subtraction	$x - y$	<code>x - y</code>
Multiplication	$x \times y$	<code>x * y</code>
Division	$x \div y$	<code>x / y</code>
Exponents	x^y	<code>x^y</code> or <code>x**y</code>
Logarithm	$\log(x)$	<code>log(x)</code>
Exponential	e^x	<code>exp(x)</code>
Trig Functions	$\sin(x)$	<code>sin(x)</code>

There are some other useful built-in functions in R. These are:

- `abs(x)`
- `sqrt(x)`
- `floor(x)`
- `ceiling(x)`
- `trunc(x)`
- `round(x, digits = n)`
- `signif(x, digits = n)`

1.2.1 Exercise

1. Experimentation is a great way to find out what a function does. Determine the square root of 64 with the `sqrt()` function and using the fact that the square root of 64 is really $64^{1/2}$
2. Try the following:
 - `ceiling(1.2)`
 - `ceiling(1.49)`
 - `ceiling(1)` What does ceiling do?
3. Try the following:
 - `round(1.4503,1)`
 - `round(exp(1),4)`
 - `round(pi,4)`
 - Now try typing `round(12314,-1)` and `round(12314,-2)`. What’s happening here?
4. What does `trunc(5.99)` give you? What does `trunc(-3.43)` give you? How is `trunc()` different than `floor()`?

1.3 Comments

Note that anything we type on an R line that comes after the `#` sign is ignored by R. This is very useful for including comments in your code and helps to remind you (and others) what you were thinking. Because anything after the `#` is ignored, we can either put a comment on its own line:

```
# here is a comment
```

or after some code that is going to be executed

```
2 + 2 # we had better get the answer "4"
```

```
## [1] 4
```

1.4 Variables

Variables allow you to store your data so that it can be easily retrieved at a later time. For example, suppose you calculated the standard deviation of a data set and you had to use this result over and over again (and you did not want to type it out each time!). The best way to reuse this value is to store it in a variable.

```
my.sd <- 0.352  
my.sd
```

```
## [1] 0.352
```

```
1.96 * my.sd
```

```
## [1] 0.68992
```

Notice that assigning the variable is performed with an “arrow” `<-`. The arrow can actually go the other way too but we don’t do that all too much.

```
a <- 5 -> b  
a
```

```
## [1] 5
```

```
b
```

```
## [1] 5
```

Getting rid of a variable is sometimes convenient, and the way to do this is with the `rm()` function.

```
rm(a)  
a
```

```
## Error in eval(expr, envir, enclos): object 'a' not found
```

To list all active variables type `ls()`. To remove all active variables type `rm(list = ls())`. This can also be achieved by clicking the broom icon in the **Environment** tab of the top right pane of Rstudio.

1.5 Data Types

We will encounter a variety of different data types during this course, and each has specific applications.

```
var_1 <- TRUE # a logical variable  
class(var_1)
```

```
## [1] "logical"
```

```
var_2 <- 32.5 # a numeric variable  
class(var_2)
```

```
## [1] "numeric"
```

```
var_3 <- "Michael" # a character variable  
class(var_3)
```

```
## [1] "character"
```

Also, we can make integer variables and factor variables. R guesses which you want, and if it guesses wrong you may need to force it to assume the correct data type. You will see this later on. Don't assume that R has correctly read your mind on the data type you wanted.

```
var_4 <- 5  
class(var_4)
```

```
## [1] "numeric"
```

```
var_4 <- as.integer(var_4) # coerces the value to the class of integer  
class(var_4)
```

```
## [1] "integer"
```

```
var_5 <- "4"  
class(var_5)
```

```
## [1] "character"
```

```
7 * var_5 # what happened?
```

```
## Error in 7 * var_5: non-numeric argument to binary operator
```

```
var_5 <- as.numeric(var_5) # coerces the character into a numeric  
class(var_5)
```

```
## [1] "numeric"
```

```
7 * var_5 # ahhh no error now.
```

```
## [1] 28
```

1.6 Vectors

Vectors are a way to store multiple data points in a single variable. They are like a column from an Excel file and we will use them a lot. To define a vector you have to use the combine `c()` function to group the data.

```
x <- c(5,3,6,4,7,2,6) # defines the variable x  
class(x) # what class will this be?
```

```
## [1] "numeric"
```

Importantly, every member of the vector must be of the same type and if they are not, R will choose a data type that will be compatible with all the elements.

```
y <- c(5,3,6,4,7,2,"Hi There")  
class(y) # What happened?
```

```
## [1] "character"
```

```
y
```

```
## [1] "5" "3" "6" "4" "7" "2" "Hi There"
```

Let's explore some algebra:

```
x + 2    # What does it do?
```

```
## [1] 7 5 8 6 9 4 8
```

```
x + x    # How is this different from x+2
```

```
## [1] 10 6 12 8 14 4 12
```

```
x / 2    # What does this do?
```

```
## [1] 2.5 1.5 3.0 2.0 3.5 1.0 3.0
```

```
x * x    # What does this do?
```

```
## [1] 25 9 36 16 49 4 36
```

```
x / x    # What does this tell you about dividing vectors?
```

```
## [1] 1 1 1 1 1 1 1
```

```
sum(x)    # What does this calculate?
```

```
## [1] 33
```

```
mean(x)    # And this?
```

```
## [1] 4.714286
```

```
sd(x)      # And this?
```

```
## [1] 1.799471
```

```
length(x)  # And this? This is really useful.
```

```
## [1] 7
```

```
c(x, x)
```

```
## [1] 5 3 6 4 7 2 6 5 3 6 4 7 2 6
```

```
rep(x,3)
```

```
## [1] 5 3 6 4 7 2 6 5 3 6 4 7 2 6 5 3 6 4 7 2 6
```

What if we wanted to find out an individual value from x?

```
x[2]    # Note the SQUARE brackets.
```

```
## [1] 3
```

```
# OK, makes sense
```

What if we wanted to know which value of x was 6, if any?

```
which(x == 6)
```

```
## [1] 3 7
```

CAREFUL “==” is used to compare two values to see if they are equal; don’t confuse this with and “=” or “<=”, which are for assignment.

```
x[which(x == 6)]    # should give us back a number of 6's of course.
```

```
## [1] 6 6
```

```

x == 6      #this gives us a logical vector answering the question "Is the value 6?"

## [1] FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE
y <- x == 6
y

## [1] FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE
x[y]      #this should just give us back the 6's

## [1] 6 6
x[x == 6]

## [1] 6 6

```

1.6.1 Exercise

1. Define a variable, `days`, which contains all the days of the week.
2. Now imagine that you move to a planet where there are an 8th and 9th day of the week, called “Chillday” and “Sleepday”. Can you use `c()` to add these days to your `days` variable so that you do not have to retype everything?

1.6.2 Exercise

1. Type `1:10` and see what happens.
2. Now type `x <- 1:10`. What did this do? Find out by asking R what `x` is.
3. Now type `x <- 5:10`. What did this do?
4. Type `?seq` in the console to find out what the `seq()` function does. Can you replicate the results of `x <- 5:10` with `seq()`? Why is `seq()` more flexible?

1.6.3 Exercise

1. Write an expression in R that will always return the last value in a vector named `z`.
2. Test your expression’s success. We’ll start by generating a sequence of letters of the alphabet that terminates randomly.

```

z <- letters[1:round(runif(1,0,26),0)] # don't worry about why this works right now.
z

## [1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r"
z[length(z)] # check what the last value is

## [1] "r"

```

Now apply your expression to identify the last letter in the sequence you generated.

2 Matrices, Dataframes and Lists

2.1 Matrices

A matrix is a 2D analogue of the vector. Matrices may not seem all that important but there are certain R statistical functions requiring their use and so you are certainly going to encounter them.

```

days <- 1:28 # Generates a sequence of 28 integers
days # confirm that you have done what you thought

```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28
```

```
# Note: another way to do this is with days <- seq(from = 1, to = 28, by = 1)
```

Now, let's convert this to something that looks more like the calendar layout of February.

```
matrix(days, nrow = 4, ncol = 7)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]    1    5    9   13   17   21   25
## [2,]    2    6   10   14   18   22   26
## [3,]    3    7   11   15   19   23   27
## [4,]    4    8   12   16   20   24   28
```

```
# Huh? What happened?
```

```
#
```

```
# Let's try again
```

```
matrix(days, nrow = 4, ncol = 7, byrow = TRUE)
```

```
##      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## [1,]    1    2    3    4    5    6    7
## [2,]    8    9   10   11   12   13   14
## [3,]   15   16   17   18   19   20   21
## [4,]   22   23   24   25   26   27   28
```

```
# Ahhh
```

2.1.1 Exercise

- February 2016 started on a Monday but was a leap-year. Make a matrix for February 2016 and fill days that are not in February with NA. We'll talk later about different places that NA pops up, but for now just know that it's R's way of denoting data that is "Not Available". Hint for this exercise: start by using the `c()` command to prepend the appropriate number of NA's to the beginning and the end of variable days. Assign the name `feb_2016` to your matrix.

To get a specific value of your matrix, you simply refer to the row and column.

```
feb_2016[3, 4] #Gives entry from row 3 column 4. Should return 17 as the result.
```

```
## [1] 17
```

```
# To get all the values in the column, you simply omit the row number.
```

```
feb_2016[, 6] # Gives us what we need: all the Saturdays.
```

```
## [1] 5 12 19 26 NA
```

```
# ...and the same trick works for columns
```

```
feb_2016[3, ] # Gives us week 3.
```

```
## [1] 14 15 16 17 18 19 20
```

2.1.2 Exercise

The raw speed data from your latest bike ride can be exported from your Garmin and brought into R. The ride happens to be exactly 38 mins duration and data is sampled every second. This means that there will be 2280 measurements in total. Execute this code to import the data from a file.

```
library(tidyverse)
speed_vec <- scan("Data_Files/speed.txt")
```

- Now, convert this data into a matrix called `speed.mat` of 38 columns where each column is the speed data of one minute.

2.2 Dataframes

A dataframe (also called a “tibble” in the tidyverse) is the closest thing you are going to get to Excel-like storage of your data. When you read your data into R from a file (if you have saved it from Excel in a standard format), it will become a dataframe.

Let’s convert our February 2016 matrix to a dataframe, and then we will look at how to import Excel-like data to a dataframe. We will spend some time working with dataframes because they are going to be our bread and butter.

First, we can convert a matrix to a dataframe:

```
feb_2016 <- matrix(c(NA,1:29,NA,NA,NA,NA,NA), nrow = 5, ncol = 7, byrow = TRUE)
feb_2016 <- as.data.frame(feb_2016)
feb_2016
```

```
##   V1 V2 V3 V4 V5 V6 V7
## 1 NA  1  2  3  4  5  6
## 2  7  8  9 10 11 12 13
## 3 14 15 16 17 18 19 20
## 4 21 22 23 24 25 26 27
## 5 28 29 NA NA NA NA NA
```

```
# this has uninformative column names, but we can name the columns as we can in Excel
names(feb_2016) <- c("Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat")
feb_2016
```

```
##   Sun Mon Tue Wed Thu Fri Sat
## 1  NA  1  2  3  4  5  6
## 2   7  8  9 10 11 12 13
## 3  14 15 16 17 18 19 20
## 4  21 22 23 24 25 26 27
## 5  28 29 NA NA NA NA NA
```

```
# much better
```

Second, we can build a dataframe from vectors as follows.

```
odd_nums <- c(1,3,5,7)
even_nums <- c(2,4,6,8)
numbers <- data.frame(odds = odd_nums, evens = even_nums)
numbers
```

```
##   odds evens
## 1    1     2
## 2    3     4
## 3    5     6
## 4    7     8
```

```
# alternatively, numbers <- tibble(rbind(odd_nums, even_nums))
```

So, the columns of a dataframe could be the results of different data fields in your study, name, health number, sex, age, date of last visit, blood pressure, medications, creatinine, hemoglobin etc. Whatever you could store in an Excel sheet could be in a dataframe. We often want to pull out specific columns from a dataframe – usually to perform statistical tests.

Pulling out a column is easy. We can do it by the column name or we can do it by the column number.

```

# by column name
feb_2016$Tue # gives all the Tuesdays in Feb 2016
# or
feb_2016[["Tue"]]

# by column number
feb_2016[[3]] #result as vector
#or
feb_2016[,3] #result as tibble

# You can pull out data from an individual cell.
feb_2016[2,3]

# Or you can do the same with the $ approach because feb_2016$Tue is a vector
feb_2016$Tue[2]
# or
feb_2016[["Tue"]][2]
# or
feb_2016[[3]][2]

```

If you need to pull more than one column out, you can do so by numbers or the names:

```

# by number
feb_2016[,3:4]

```

```

##    Tue Wed
## 1     2   3
## 2     9  10
## 3    16  17
## 4    23  24
## 5    NA  NA

```

```

#by name - this is particularly convenient when dealing with large dataframes
feb_2016[,c("Tue","Wed")]

```

```

##    Tue Wed
## 1     2   3
## 2     9  10
## 3    16  17
## 4    23  24
## 5    NA  NA

```

```

#or
select(feb_2016,c("Tue","Wed"))

```

```

##    Tue Wed
## 1     2   3
## 2     9  10
## 3    16  17
## 4    23  24
## 5    NA  NA

```

Let's go back to that bike speed data we have above. We will start by turning the data, stored previously as matrix called `speed.mat`, into a dataframe.

```

speed_vec <- scan("Data_Files/speed.txt")
speed_mat <- matrix(speed_vec, nrow = 60, ncol = 38, byrow = FALSE)

```

```
speed_df <- as.data.frame(speed_mat)
names(speed_df) <- paste0("min_",1:38)
#head(speed_df)
```

2.2.1 Exercise

- Using the `speed_df` dataframe, find the average speed of the 20th minute of your ride.

Note that if you try to find the average by row instead of a column, you will run into a problem that illustrates something about dataframes, namely that if you extract a row, you can't just simply do algebra on it because there is generally no guarantee that different columns of a data frame will all be numbers.

```
min_start_avg <- mean(speed_df[1,]) #does not work
min_start_avg <- mean(speed_mat[1,]) #does work
```

Here are some things you can do with data in a data frame. You can ask R to tell you things about your data points. For example, you could ask R for some descriptive statistics of the last minute of your ride:

```
mean(speed_df$min_38)
```

```
## [1] 25.15944
```

```
median(speed_df$min_38)
```

```
## [1] 23.103
```

```
sd(speed_df$min_38)
```

```
## [1] 4.482852
```

```
summary(speed_df$min_38) # a statistical summary
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      21.50  22.52   23.10   25.16   26.81   38.38
```

```
quantile(speed_df$min_38, probs = c(0.25,0.50, 0.75)) # specific quantiles
```

```
##      25%      50%      75%
## 22.5216 23.1030 26.8137
```

But this does not work when you try to take the grand mean:

```
mean(speed_df)
```

```
## [1] NA
```

But this approach does work for matrices

```
mean(speed_mat)
```

```
## [1] 30.9961
```

Remember, we'll say more later about "NA", but for now notice that R gives us a nasty warning message. The reason that the `mean` function does not operate on data frames in this case because the data in a data frame is often not numeric.

You can also ask R to tell you which values have certain properties.

```
# did you dip below 25 kph in the last minute of your ride?
speed_df$min_38 < 25
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [13] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
## [25] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [37] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [49] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

But the code authors acknowledge that it is a pain to have to write out those \$ signs all the time, so try this:

```
attach(speed_df)
min_1 # now min_1 is a local variable

## [1] 14.1984 17.2188 18.8964 20.0448 21.3732 22.4172 22.9752 23.2992 23.3424
## [10] 23.1228 22.6908 22.2876 21.9456 21.6648 21.3768 21.1140 20.9412 20.7648
## [19] 20.6244 20.5056 20.1960 19.9080 19.3608 18.9612 18.8460 18.8424 18.9324
## [28] 19.1232 19.2600 19.5696 20.3868 20.0412 19.2276 18.0828 16.9488 16.2684
## [37] 15.5304 15.0048 14.7204 14.3928 14.3172 14.5800 15.1416 15.1884 12.4380
## [46] 12.3840 12.8736 13.3128 13.6764 14.0724 14.4720 14.8104 15.1596 15.4800
## [55] 15.6384 15.8400 16.0596 16.2900 16.4952 16.6860

min_2 # ...and so is min_2!
```

```
## [1] 17.0064 17.4456 17.8488 18.1260 18.3996 18.6264 18.9576 19.4436 18.9756
## [10] 19.3752 19.8648 20.4372 21.0564 21.6972 22.6152 23.4756 24.1200 24.5772
## [19] 24.8796 25.1820 25.5852 25.9020 26.1864 26.4996 26.8560 27.2232 27.6732
## [28] 28.2204 28.7856 29.2536 29.6244 29.9376 30.2544 30.5064 30.6576 30.8412
## [37] 31.0320 31.2156 31.3128 31.3488 31.3164 31.3200 31.3452 31.4280 31.4712
## [46] 31.3632 31.3020 31.2048 31.0608 30.9024 30.7404 30.5892 30.3804 30.0780
## [55] 29.5488 28.7388 28.0620 28.0980 28.4004 28.5876
```

Note that if you alter the attached variables, they *do not* alter the original data frame. To remove these variables we type:

```
detach(speed_df)
```

There are lots of reasons not to use `attach`, but it is good for quick and dirty things.

2.2.2 Exercise

- We are going to read some data from a publicly available data set. This time the data will contain information about liver function tests from an Astra Zeneca clinical trial for a drug administered at 4 doses: A, B, C, and D pre- and post-administration of the drug. Pre-dose results are annotated as .B, for baseline and .M for post-dose.

```
liver_data <- read.csv("Data_Files/liver.csv")
head(liver_data)
```

```
## rownames ALP.B ALT.B AST.B TBL.B ALP.M ALT.M AST.M TBL.M dose
## 1 1 80 13 14 12.654 87 22 22 23.085 A
## 2 2 37 15 16 6.498 37 25 23 8.037 A
## 3 3 52 10 13 4.788 55 10 13 6.498 A
## 4 4 36 13 13 6.840 35 11 12 8.037 A
## 5 5 39 18 12 14.364 37 21 15 16.758 A
## 6 6 48 8 13 6.156 50 8 13 5.985 A
```

Isolate from your data those people who received dose B

What you have just done is called filtering or subsetting your dataframe, and it is a very frequent task that we are going to be doing more of in the next hour. Because it is such a common task (e.g. pulling

out all the subjects less than 40 years, pulling out all the male subjects, excluding an outlier), there are specific commands created for this purpose.

```
# you can do it with base R
dose_B_data <- subset(liver_data, dose == "B")

# or you can use filter from dplyr
library(dplyr)
dose_B_data <- filter(liver_data, dose == "B")
```

You can build other logical constraints to isolate data of interest in vectors, matrices and data frames by using & (AND), | (OR), and ! (NOT). For example:

```
x <- c(3, 4, 2, 7, 5, 8, 9, 5, -2, 34, 15, 7)
# values of x greater than 2 and less than 7. Use the & for AND.
x[x > 2 & x < 7]
```

```
## [1] 3 4 5 5
```

```
# values of x not less than 15
x[!(x < 15)]
```

```
## [1] 34 15
```

```
# which is the same as
x[x >= 15]
```

```
## [1] 34 15
```

```
# values of x less than 3 or greater than 5. Use the | for OR.
x[x < 3 | x > 5]
```

```
## [1] 2 7 8 9 -2 34 15 7
```

```
# all values except the first three: use a minus sign
x[-(1:3)]
```

```
## [1] 7 5 8 9 5 -2 34 15 7
```

and with the dataframe, we might ask for people on dose D for which ALT.M was greater than ALT.M.

(Note that – just like algebra – we can use parenthesis to make absolutely clear what order we are using to evaluate the expressions)

```
filter(liver_data, dose == "D" & (ALT.M > ALT.B))
```

```
##      rownames ALP.B ALT.B AST.B TBL.B ALP.M ALT.M AST.M TBL.M dose
## 1         449   35   13   14  8.721   41   17   12 14.364   D
## 2         450   49   26   18 10.944   69   38   24 17.613   D
## 3         451   36   12   15 12.654   43   16   18 16.587   D
## 4         452   33    9   11  8.550   53   11   14 11.115   D
## 5         453   42   19   16 13.509   57   24   23 16.074   D
## 6         454   59   12   16 10.089   84   17   17 12.996   D
## 7         457   35   11   12 20.862   65   13   15 16.074   D
## 8         458   38   15   15 10.431   51   44   30 14.535   D
## 9         460   58   18   15 12.141   69   20   18 14.022   D
## 10        461   59   22   22  9.405   71   31   29  9.234   D
## 11        462   34   15   20  8.379   35   20   19  7.524   D
## 12        463   50    8   14  7.182   49   14   13  9.234   D
## 13        464   54   17   20 10.431   59   33   27 10.260   D
## 14        465   35   27   16 20.349   32   29   21 17.442   D
```

## 15	466	42	11	15	11.628	47	15	19	8.892	D
## 16	467	50	13	17	9.063	65	38	21	11.115	D
## 17	471	77	19	20	9.063	89	31	25	13.167	D
## 18	472	53	19	15	8.550	65	25	17	11.457	D
## 19	473	54	18	20	24.111	62	25	25	21.546	D
## 20	474	38	21	20	9.918	39	22	17	10.089	D
## 21	475	61	10	12	8.037	73	16	15	7.866	D
## 22	476	48	16	14	7.182	56	20	19	10.602	D
## 23	477	62	17	17	11.115	78	29	19	17.613	D
## 24	478	60	19	17	19.152	74	27	19	19.323	D
## 25	479	54	21	16	17.955	54	39	25	23.427	D
## 26	480	56	16	25	7.353	66	21	25	9.405	D
## 27	481	78	13	19	7.182	115	36	31	9.234	D
## 28	482	71	9	15	5.985	105	12	20	9.747	D
## 29	483	67	23	16	6.498	146	324	250	6.840	D
## 30	484	117	19	16	10.089	136	29	16	11.115	D
## 31	485	47	13	16	9.405	51	19	19	9.063	D
## 32	486	61	11	12	16.587	67	19	15	12.996	D
## 33	487	54	11	20	5.985	53	16	19	6.669	D
## 34	488	58	11	18	7.524	60	16	22	14.193	D
## 35	489	45	19	18	11.970	49	42	29	11.799	D
## 36	490	55	21	20	22.743	56	31	25	35.568	D
## 37	491	66	15	17	13.338	71	28	35	12.654	D
## 38	492	67	13	12	8.892	66	16	11	11.286	D
## 39	493	50	21	18	6.669	70	27	19	12.312	D
## 40	494	35	15	18	6.840	56	19	21	10.260	D
## 41	496	48	9	13	10.773	52	25	22	15.048	D
## 42	498	70	32	18	10.431	110	101	43	10.431	D
## 43	500	43	25	19	17.442	52	28	18	16.587	D
## 44	501	32	18	18	7.011	44	25	27	9.918	D
## 45	502	73	15	19	4.617	76	16	19	9.405	D
## 46	503	78	15	15	13.338	82	28	22	16.929	D
## 47	504	45	8	13	6.669	51	11	13	9.063	D
## 48	505	42	15	16	10.431	53	16	16	6.498	D
## 49	507	57	6	27	6.840	69	11	22	8.379	D
## 50	508	32	31	19	13.167	41	38	26	13.509	D
## 51	509	44	17	9	6.498	49	18	13	6.156	D
## 52	510	51	13	16	22.914	245	277	154	29.754	D
## 53	511	43	18	16	11.286	43	24	21	19.323	D
## 54	514	66	11	16	7.182	125	90	70	13.338	D
## 55	516	99	9	16	3.249	96	12	18	4.275	D
## 56	517	52	15	15	12.654	59	47	30	14.877	D
## 57	518	47	13	17	9.234	47	22	21	11.286	D
## 58	519	39	6	15	13.338	47	7	18	11.115	D
## 59	520	45	10	10	9.063	62	28	18	8.208	D
## 60	521	49	12	14	7.011	50	22	21	11.115	D
## 61	525	47	12	15	8.379	53	16	16	14.535	D
## 62	526	90	10	15	16.587	159	45	38	19.323	D
## 63	528	72	14	17	11.799	77	22	21	14.364	D
## 64	529	46	22	17	9.405	53	39	24	9.918	D
## 65	531	37	15	17	7.353	43	22	22	13.167	D
## 66	532	41	9	10	6.327	57	21	19	7.353	D
## 67	534	42	20	16	12.825	47	29	20	14.535	D
## 68	536	52	13	14	6.669	59	21	19	7.011	D

## 69	537	42	9	12	6.840	45	10	15	7.524	D
## 70	538	47	12	15	18.810	55	25	19	15.048	D
## 71	539	50	15	16	7.695	51	22	19	9.063	D
## 72	541	41	19	16	8.892	50	21	15	11.457	D
## 73	543	44	12	11	10.089	47	16	14	11.286	D
## 74	544	56	20	15	6.669	73	35	24	6.327	D
## 75	545	49	14	19	6.327	53	22	23	7.695	D
## 76	546	38	18	17	10.431	50	44	23	6.498	D
## 77	547	59	9	17	6.840	66	11	15	6.840	D
## 78	548	51	10	10	12.996	59	18	16	27.360	D
## 79	549	80	13	17	13.167	107	41	35	15.048	D
## 80	550	55	9	13	8.379	67	11	12	10.260	D
## 81	551	50	13	17	16.245	56	19	19	12.483	D
## 82	552	76	11	17	10.089	90	22	22	19.494	D
## 83	553	69	18	15	12.483	71	19	15	12.483	D
## 84	554	49	15	19	9.576	54	28	26	9.747	D
## 85	555	47	9	12	6.156	54	13	19	7.182	D
## 86	556	83	12	18	7.695	81	14	18	9.234	D
## 87	557	55	14	20	6.498	68	23	26	8.550	D
## 88	559	41	24	18	8.892	44	30	20	9.234	D
## 89	561	123	22	21	7.353	341	196	113	9.063	D
## 90	562	42	12	17	8.550	45	15	19	7.695	D
## 91	563	69	9	11	5.130	90	27	23	5.472	D
## 92	564	68	18	14	14.535	71	21	15	16.929	D
## 93	565	79	11	13	7.695	97	19	20	17.784	D
## 94	567	76	13	14	7.353	163	27	23	8.037	D
## 95	569	49	25	17	9.576	54	38	20	13.851	D
## 96	571	46	32	22	14.877	49	48	26	13.680	D
## 97	573	77	9	12	17.100	108	30	23	20.007	D
## 98	574	47	8	12	4.788	63	10	16	8.037	D
## 99	575	40	17	19	9.918	38	24	21	17.100	D
## 100	576	49	7	11	6.669	58	11	13	10.431	D
## 101	577	31	15	10	6.840	38	16	10	8.550	D
## 102	579	46	8	12	7.695	53	14	14	10.602	D
## 103	581	41	12	16	10.944	53	20	27	11.115	D
## 104	582	40	27	22	4.617	46	37	24	5.472	D
## 105	584	90	10	13	6.669	77	15	16	12.312	D
## 106	585	38	14	13	12.141	38	36	20	18.810	D
## 107	586	43	9	16	13.338	101	36	27	16.929	D
## 108	588	34	22	22	6.669	39	31	29	7.524	D
## 109	590	37	35	17	12.141	34	36	21	21.033	D
## 110	591	43	12	13	4.959	53	14	14	5.814	D
## 111	592	44	10	12	6.498	61	18	19	11.628	D
## 112	593	75	19	20	3.933	77	36	35	3.933	D
## 113	594	129	19	17	5.985	85	22	16	7.695	D
## 114	596	56	13	14	7.011	62	21	18	11.799	D
## 115	597	40	15	16	8.550	43	20	17	8.721	D
## 116	598	71	11	12	6.669	64	13	13	6.669	D
## 117	599	38	17	18	9.747	45	37	27	7.524	D
## 118	601	46	10	17	13.167	53	11	15	16.245	D
## 119	602	52	22	21	7.353	59	43	27	10.089	D
## 120	603	57	11	15	5.472	91	21	23	7.695	D
## 121	604	55	11	14	7.866	67	23	18	7.695	D
## 122	605	50	15	18	11.286	66	28	25	15.048	D

What do you infer about dose D's affect on ALT given that 122 of 158 subjects have higher ALT.M than ALT.B?

2.3 Lists

Lists are another way of storing data in a conveniently accessible way. Usually they are used for data types that are different in structure (or store different types of data) but are related because they are part of the same analysis. For example, R frequently provides output of statistical analysis in the form of a list. Suppose you had a vector, a data frame and a matrix:

```
a <- c("Doe", "a", "deer")
b <- data.frame(lyrics1 = c("a", "female", "deer"), lyrics2 = c("Ray", "a", "drop"))
d <- matrix(seq(from = 0, to = 100, length.out = 25), nrow = 5, ncol = 5)
```

But they were all related to some specific problem and you wanted to bundle them together in another variable. This is where you would use a list.

```
my_list <- list(a,b,d)
my_list
```

```
## [[1]]
## [1] "Doe"  "a"    "deer"
##
## [[2]]
##   lyrics1 lyrics2
## 1      a      Ray
## 2 female      a
## 3   deer    drop
##
## [[3]]
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.000000 20.83333 41.66667 62.50000 83.33333
## [2,] 4.166667 25.00000 45.83333 66.66667 87.50000
## [3,] 8.333333 29.16667 50.00000 70.83333 91.66667
## [4,] 12.50000 33.33333 54.16667 75.00000 95.83333
## [5,] 16.66667 37.50000 58.33333 79.16667 100.00000
```

Components of the list are addressed using a double bracket notation. For example:

```
my_list[[2]] # gives the dataframe.
```

```
##   lyrics1 lyrics2
## 1      a      Ray
## 2 female      a
## 3   deer    drop
```

If you happen to give the components of the list names, you can address with the \$ notation:

```
my_list <- list(one = a, two = b, three = d)
my_list
```

```
## $one
## [1] "Doe"  "a"    "deer"
##
## $two
##   lyrics1 lyrics2
## 1      a      Ray
## 2 female      a
```

```
## 3    deer    drop
##
## $three
##      [,1]      [,2]      [,3]      [,4]      [,5]
## [1,] 0.000000 20.83333 41.66667 62.50000 83.33333
## [2,] 4.166667 25.00000 45.83333 66.66667 87.50000
## [3,] 8.333333 29.16667 50.00000 70.83333 91.66667
## [4,] 12.500000 33.33333 54.16667 75.00000 95.83333
## [5,] 16.666667 37.50000 58.33333 79.16667 100.00000
```

```
my_list$two # gives the same dataframe data
```

```
## lyrics1 lyrics2
## 1      a      Ray
## 2 female      a
## 3    deer    drop
```

R very often uses lists for statistical reports. For example, the `lm()` function is used to generate a regression report by R. You can assign the output of `lm()` to a variable and this variable contains a very useful list.

```
x <- 1:10
y <- 2 * x + rnorm(10,0,1) # generates some fake data
reg <- lm(y ~ x)
str(reg) # shows what variables that reg stores.
```

```
# ...but they generate a lot of output, so try it on your own
```

```
reg$residuals
reg$fitted.values
```

```
# OK, fine--we'll show one...
reg$coefficients
```

```
## (Intercept)          x
##  0.3398373  1.8700627
```

As a final trivia point, dataframes are really just lists comprised columns, each of which must be an equal-length vector.

3 Reading in Data and Basic Sanity Checking

3.1 The Working Directory

First we need to learn how to read in Excel data. While `.xls` and `.xlsx` files can be read in in their native formats (we will show this later), usually we use the simpler approach of dealing with a `.csv` file. You can save your Excel spreadsheet as a `.csv` file by clicking “Save As” and selecting “Comma Separated Values”.

Before we do this, however, we need to start by telling R where it should consider its current working directory. You can check the current working directory by typing `getwd()`. The directory will display. You are going to need to get R to the place where you want to go. You can do this with RStudio by choosing **Session -> Set Working Directory -> Choose Directory** and then navigate where you want to go. However, if this navigation process has to be part of the code in the R Script (which it frequently does), you can get code to do the same thing.

To do the same thing in code, you need to know the *path* that you want to send R to.

- In Windows, you will right-click and then examine Properties of the folder or file.
- In Mac OS X you will use CTRL-click on the folder or file and select “Get Info” and look at the “Where” row of the General tab. You can also drag a folder into the terminal window and the full path of the directory will appear on the command line.
- In Linux you will know what to do because Linux people are never confused by navigating directories from the command line.

```
getwd() # where am I now
setwd("C:/Users/drdan/BUWE")
getwd() # see that things have changed
```

**** Caution to Windows users: **** Windows uses backslash (\) not forwardslash (/) between subdirectories. The \ is a special character in R (and many other languages) called an “escape” and it is used to denote special characters like tabs and carriage returns. To overcome this you can either turn all your \ to \\ (an escaped backslash) or you can turn all your \ to /, to be like the rest of the world.

3.2 Reading a csv File

The command `list.files()` or this tidyverse alternative, `fs::dir_ls()`, will show you all files in the directory you are working in. To read in the data we first need to make sure that R knows what directory to read from. As mentioned earlier, we can set the working directory using the R Studio Gui under Session > Set Working Directory > Choose Directory. Alternatively we can use the `set.wd()` command as previously discussed.

from the csv file we type:

```
read.csv("Data_Files/intersalt.csv")
```

The `read.csv()` function has lots of options. Type `?read.csv` to display them.

The `read.delim()` and `read.table()` functions are more generalized version of the same thing for files that are not comma-separated. For example:

- `sep = "|"` tells R that a | is what separates different columns.
- `sep = "\t"` is another common option for tab delimited files
- `header = FALSE` tells R that the first row is not the column names
- `skip = 5` tells R that the first 5 rows are not to be read in
- `na.strings = c("NA", "N/A", "")` tells R that missing values are encoded as NA, N/A and empty space.
- `strip.white = TRUE` is an option to prevent something like "M" and "M " and " M " from being considered three different things.

If you have a file from a collaborator in Quebec or French-Speaking Europe, you may use a comma as decimal place in your work. This means that you are not going to be using vanilla `read.csv()`. First, you will need to be sure that you do not use a comma as your column separator when you save a csv file. You will need to use something else. Most often this is a semicolon. R has a built-in function for those who use commas as decimals. This is `read.csv2()`, which is the same as `read.csv()` except that the default parameter `dec = "."` is changed to `dec = ","` and `sep = ","` is changed to `sep = ";"`.

Returning to the intersalt data set, we should store this data in a variable so that we can do some analysis of it. Since it is data from the intersalt study we can name the variable `intersalt`. The file contains blood pressure (bp), sodium excretion (na) and nationality data. We have deliberately inserted a bad data point for instructional purposes.

```
intersalt <- read.csv("Data_Files/intersalt.csv")
```

The `head()` function is a quick way to look at what the data looks like in a snippet.

```
head(intersalt) # gives us the first 6 lines
```

```
##      b   bp   na  country
## 1 0.512   72 149.3 Argentina
## 2 0.226  78.2 133.0   Belgium
## 3 0.316  73.9 142.6   Belgium
## 4 0.042  61.7   5.8    Brazil
## 5 0.086  61.4   0.2    Brazil
## 6 0.265  73.4 148.9   Canada
```

```
head(intersalt, 10) #gives us the first 10 lines
```

```
##      b   bp   na  country
## 1 0.512   72 149.3  Argentina
## 2 0.226  78.2 133.0   Belgium
## 3 0.316  73.9 142.6   Belgium
## 4 0.042  61.7   5.8    Brazil
## 5 0.086  61.4   0.2    Brazil
## 6 0.265  73.4 148.9   Canada
## 7 0.384  79.2 184.3   Canada
## 8 0.501  66.6 194.1  Colombia
## 9 0.352  82.1 135.6   Denmark
## 10 0.443   75 138.7 East Germany
```

Notice that the dataframe displays the data type of each column underneath the column name.

3.3 Surveying your data

The `str()` or structure function is where we should go next. We want to find out what assumptions R has made about the nature of the data and change them if need be.

The command `str(intersalt)` gives the following output:

```
## 'data.frame':   52 obs. of  4 variables:
## $ b      : num  0.512 0.226 0.316 0.042 0.086 0.265 0.384 0.501 0.352 0.443 ...
## $ bp     : chr  "72" "78.2" "73.9" "61.7" ...
## $ na     : num  149.3 133 142.6 5.8 0.2 ...
## $ country: chr  "Argentina" "Belgium" "Belgium" "Brazil" ...
```

The `glimpse()` function is similar to `str()`, except that it displays all columns down the page and tries to show you as much data as possible:

```
## Rows: 52
## Columns: 4
## $ b      <dbl> 0.512, 0.226, 0.316, 0.042, 0.086, 0.265, 0.384, 0.501, 0.352, ~
## $ bp     <chr> "72", "78.2", "73.9", "61.7", "61.4", "73.4", "79.2", "66.6", ~
## $ na     <dbl> 149.3, 133.0, 142.6, 5.8, 0.2, 148.9, 184.3, 194.1, 135.6, 138~
## $ country <chr> "Argentina", "Belgium", "Belgium", "Brazil", "Brazil", "Canada~
```

`summary(intersalt)` conveys some summative information:

```
##      b              bp              na              country
## Min.   :0.0420   Length:52   Min.    : 0.2   Length:52
## 1st Qu.:0.2930   Class :character  1st Qu.:135.0   Class :character
## Median :0.3460   Mode  :character  Median :152.5   Mode  :character
## Mean   :0.3502                                Mean   :148.3
## 3rd Qu.:0.4422                                3rd Qu.:172.7
## Max.   :0.6790                                Max.    :242.1
```

This means that something is wrong with the blood pressure data. The phenomenon observed here occurs when there is non-numeric data in the data set. This is extremely common when there are measurements that are below or above a calibration range. For example, glucose < 1 mmol/L or troponin < 5 ng/L.

3.4 Coping with Non-numeric Data

So what can we do? See what happens when we type:

```
as.numeric(intersalt$bp)
```

```
## [1] 72.0 78.2 73.9 61.7 61.4 73.4 79.2 66.6 82.1 75.0 77.5 77.4 79.2 71.7 70.7
## [16] 73.9 79.6 69.9 76.0 72.9 68.4 67.2 73.5 67.9 77.2 72.6 79.7 62.9 66.1 67.4
## [31] 70.2 75.7 77.9 78.2 71.4 72.1 72.7 68.0 75.2 75.5 73.1 71.2 72.4 NA 78.5
## [46] 72.4 73.2 81.4 76.2 74.7 73.1 75.6
```

The `as.numeric()` function converts the `bp` data from a character to a numeric. But, why is there now a NA in the data?

This is what all non-numerics become, even if one is < 40 and another is > 200 (“NA” stands for “not available”). So, be careful. If we want to preserve something of what was actually in the data file, we will need another approach. If we are happy to have all non-numerics display NAs, then what we have here is fine.

But if we now want to look at the statistics of the `bp` column, the NA results cannot contribute. It would be good to figure out what is going on.

3.4.1 Exercise

1. Define vector `x <- c("The", "story", "of", "Hansel", "and", NA, "is", "frightening")`
2. Apply the function `is.na()` to `x`. What do you get?
3. Use this logical vector to replace NA with “Gretel”.
4. Look at the raw `intersalt` data and figure out what is wrong with the `bp` on the entry that is becoming NA.

3.5 Simple replacements with `str_replace()`

There is another supremely useful functions called `str_replace()` and `str_replace_all` which are sort of equivalent to Excel’s find and replace but ultimately much more powerful. In their simplest form it is invoked on vector `x` with this syntax: `str_replace(string, pattern, replacement)`. The `str_replace()` function replaces the first instance of a match and `str_replace_all()` replaces all instances of a match. These functions are vectorized, that is, it operates on every element of a character vector.

For example, we already have a vector `x <- c("The", "story", "of", "Hansel", "and", "Gretel", "is", "frightening")`. But if we wanted to invent a new story we could write:

```
x <- c("The", "story", "of", "Hansel", "and", "Gretel", "is", "frightening")
str_replace(x, "Gretel", "Olga")
```

```
## [1] "The"          "story"        "of"           "Hansel"       "and"
## [6] "Olga"         "is"           "frightening"
```

```
#and to change x itself
```

```
x <- str_replace(x, "Gretel", "Olga")
x
```

```
## [1] "The"          "story"        "of"           "Hanzel"       "and"
## [6] "Olga"         "is"           "frightening"
```

The equivalent base-R functionality is available in the `sub()` and `gsub()` functions. The `sub()` function replaces the first instance of a match and `gsub()` replaces all instances of a match.

```
my.sentences <- c("I visited the island of Samos, an island that is close to Turkey", "Greece has a lot of isles, you know. You could even say Greece was isleandy.")

sub("island", "isle", my.sentences)
```

```
## [1] "I visited the isle of Samos, an island that is close to Turkey"
## [2] "Greece has a lot of isles, you know. You could even say Greece was isleandy."
```

```
#compare
gsub("island", "isle", my.sentences)
```

```
## [1] "I visited the isle of Samos, an isle that is close to Turkey"
## [2] "Greece has a lot of isles, you know. You could even say Greece was isley."
```

3.5.1 Exercise

1. Read back in your original intersalt data from the csv file using the `read.csv()` command we used earlier.
 - Replace all the letter O's in the bp column with 0 (zero) using `str_replace()`.
 - Convert the bp column to numerics as it should be
 - Now get a summary of the data and convince yourself it makes sense.

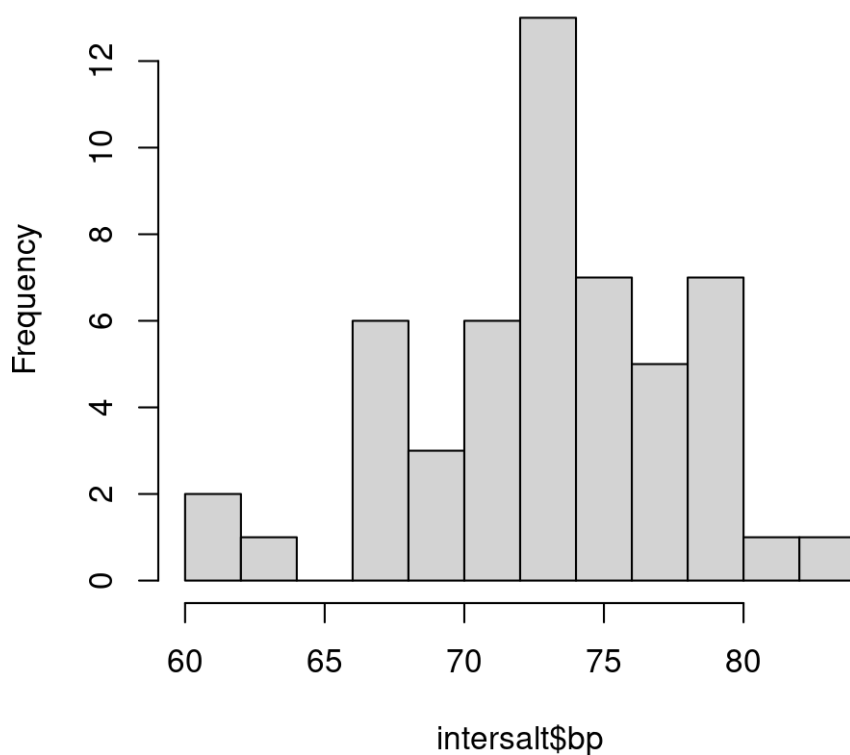
3.6 Sanity Checking your Data with Simple Visualization

3.6.1 Histograms

To produce a histogram, we use the command `hist(x)`. To increase the number of bins, we can write `hist(x, breaks = n)` where n is any number you like.

```
hist(intersalt$bp, breaks = 10)
```

Histogram of intersalt\$bp

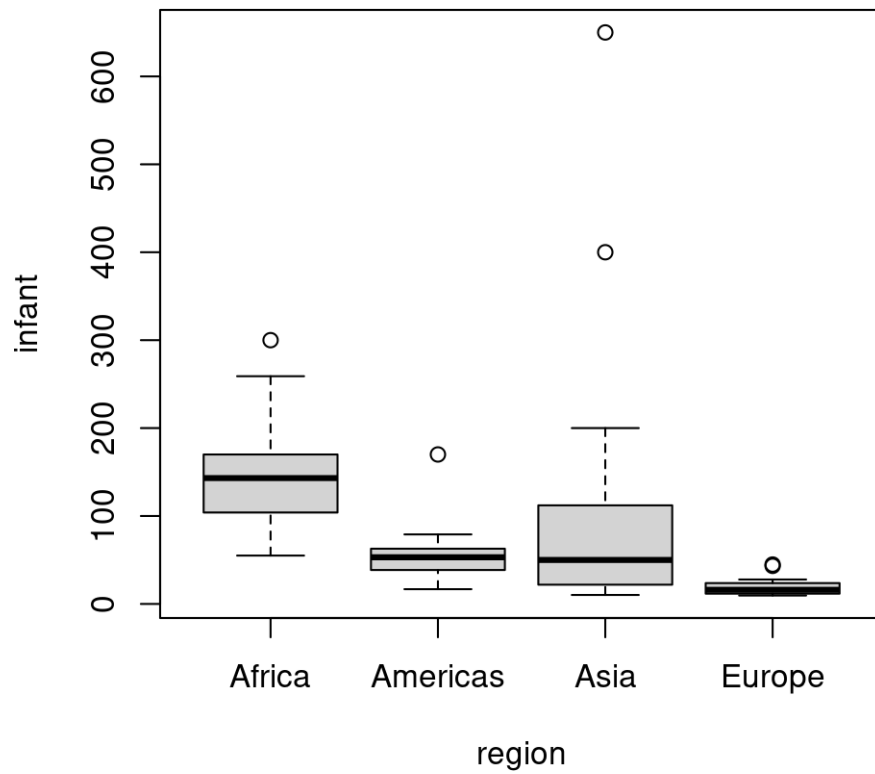


We will go into detail on beautifying our plots later, but there are many things you can do. Customization is infinite.

3.6.2 Boxplots

We can read in a toy data set in order to show how boxplots are made. This data set is comprised of infant mortality data as a function of country.

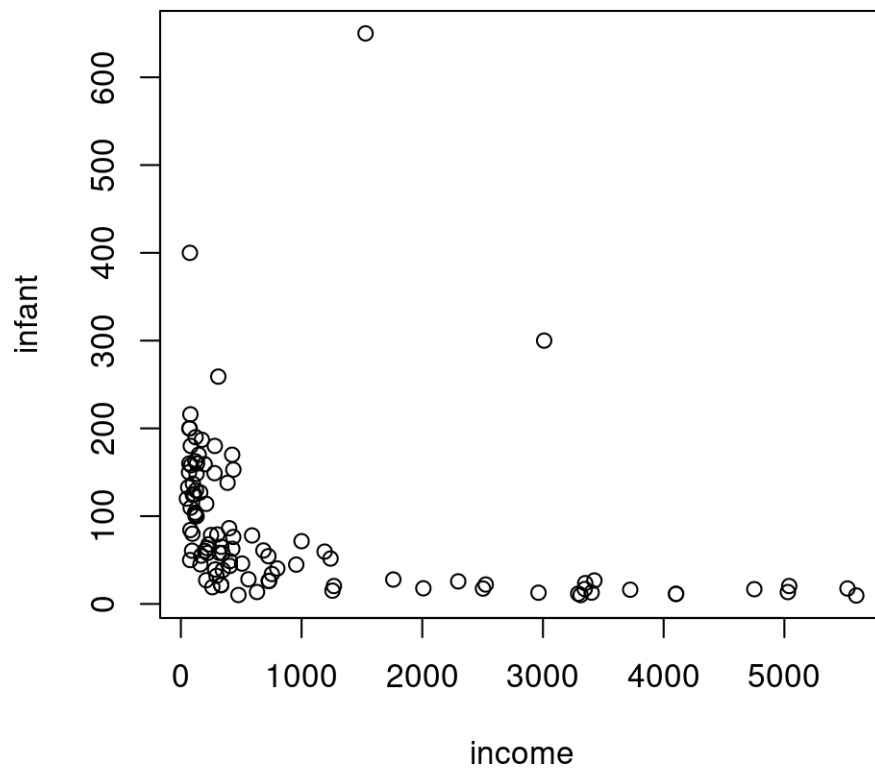
```
mortality <- read.csv("Data_Files/Leinhardt.csv")
boxplot(infant ~ region, data = mortality)
```



3.6.3 Basic Scatterplot

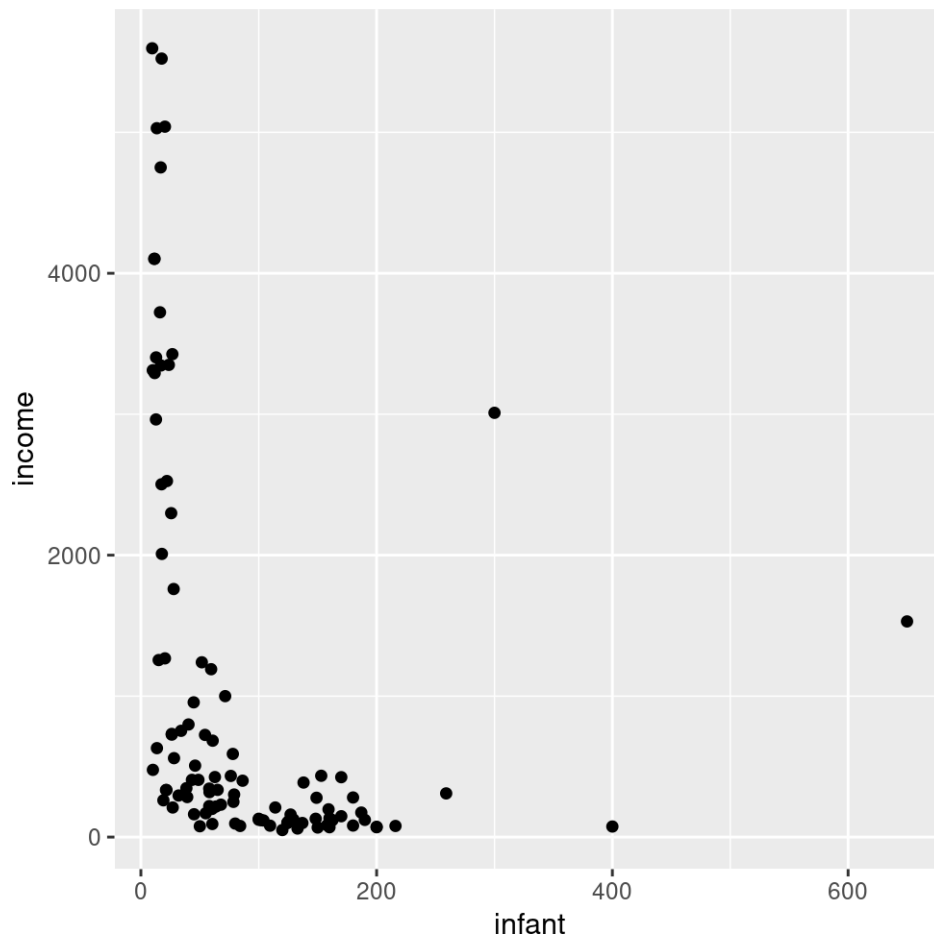
Old school style.

```
plot(infant ~ income, data = mortality)
```



The style preferred by the cool people is:

```
library(ggplot2)
qplot(infant, income, data = mortality)
```



4 Regression

4.1 Ordinary Least Squares

In this hour we are going to look at one of our very common tasks, which is regression. We will cover ordinary least squares (OLS), Deming and Passing Bablok. These latter two forms of regression have found a neurotic devotion in the Clinical Chemistry literature, and so you have to use them when you publish. They are not available in Excel. The nice thing about Passing Bablok is that it is very resistant to the effect of an outlier, though it is certainly not the only form of robust regression.

Let's start with OLS and let's use the tube.type dataset we used in the last hour. Let's plot the PTH results of EDTA and SST against one another, since we know that there are statistical differences in the mean and median. The function we will use is called `lm()`

```
tube_data <- read.csv(file = "Data_Files/tube_data.csv")
OLS_reg <- lm(EDTA ~ SST, data = tube_data)
summary(OLS_reg)
```

```
##
## Call:
```

```
## lm(formula = EDTA ~ SST, data = tube_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.64067 -0.90371 -0.04158  0.68455  2.44498
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.22948    0.62417   -1.97  0.0644 .
## SST          1.33713    0.08776   15.24 9.93e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.176 on 18 degrees of freedom
## Multiple R-squared:  0.928, Adjusted R-squared:  0.924
## F-statistic: 232.1 on 1 and 18 DF, p-value: 9.925e-12

#alternative syntax
lm(tube_data$EDTA ~ tube_data$SST)
```

```
##
## Call:
## lm(formula = tube_data$EDTA ~ tube_data$SST)
##
## Coefficients:
## (Intercept) tube_data$SST
##      -1.229      1.337
```

Now let's look at all of the things that R calculates:

```
str(OLS_reg)

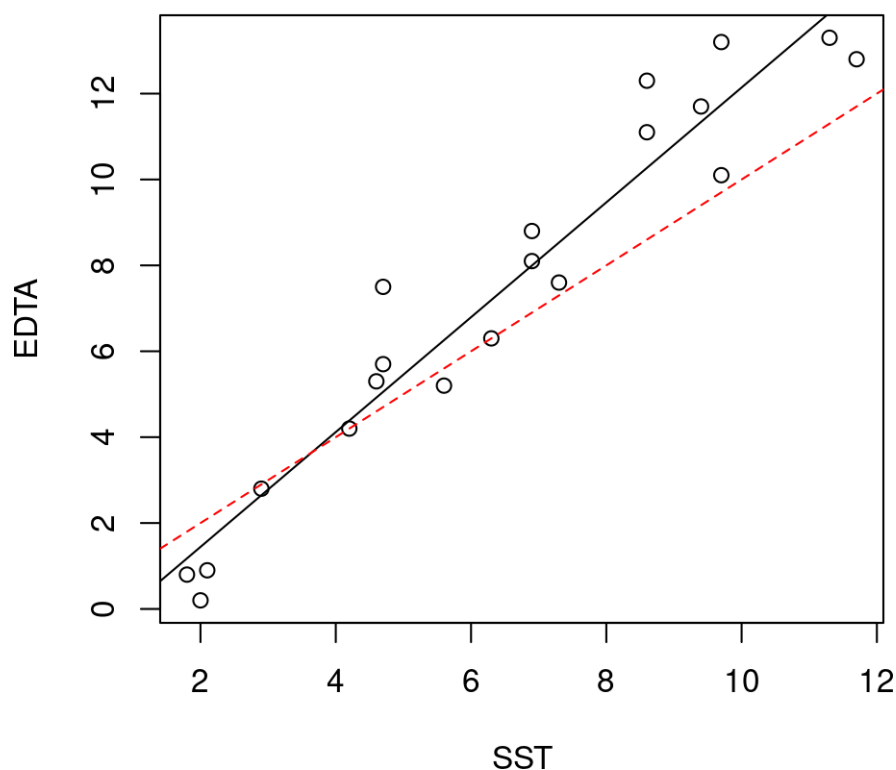
## List of 12
## $ coefficients : Named num [1:2] -1.23 1.34
## .. attr(*, "names")= chr [1:2] "(Intercept)" "SST"
## $ residuals    : Named num [1:20] 1.459 0.379 0.803 0.36 0.83 ...
## .. attr(*, "names")= chr [1:20] "1" "2" "3" "4" ...
## $ effects      : Named num [1:20] -33.071 17.919 0.536 0.11 0.574 ...
## .. attr(*, "names")= chr [1:20] "(Intercept)" "SST" "" "" ...
## $ rank         : int 2
## $ fitted.values: Named num [1:20] 11.74 4.92 8 11.34 10.27 ...
## .. attr(*, "names")= chr [1:20] "1" "2" "3" "4" ...
## $ assign       : int [1:2] 0 1
## $ qr           :List of 5
## ..$ qr        : num [1:20, 1:2] -4.472 0.224 0.224 0.224 0.224 ...
## .. ..- attr(*, "dimnames")=List of 2
## .. ..$ : chr [1:20] "1" "2" "3" "4" ...
## .. ..$ : chr [1:2] "(Intercept)" "SST"
## .. ..- attr(*, "assign")= int [1:2] 0 1
## ..$ qraux: num [1:2] 1.22 1.18
## ..$ pivot: int [1:2] 1 2
## ..$ tol  : num 1e-07
## ..$ rank : int 2
## ..- attr(*, "class")= chr "qr"
## $ df.residual : int 18
## $ xlevels     : Named list()
```

```
## $ call      : language lm(formula = EDTA ~ SST, data = tube_data)
## $ terms     :Classes 'terms', 'formula' language EDTA ~ SST
## ..- attr(*, "variables")= language list(EDTA, SST)
## ..- attr(*, "factors")= int [1:2, 1] 0 1
## ...- attr(*, "dimnames")=List of 2
## ..$ : chr [1:2] "EDTA" "SST"
## ..$ : chr "SST"
## ..- attr(*, "term.labels")= chr "SST"
## ..- attr(*, "order")= int 1
## ..- attr(*, "intercept")= int 1
## ..- attr(*, "response")= int 1
## ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
## ..- attr(*, "predvars")= language list(EDTA, SST)
## ..- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## ...- attr(*, "names")= chr [1:2] "EDTA" "SST"
## $ model      :'data.frame': 20 obs. of 2 variables:
## ..$ EDTA: num [1:20] 13.2 5.3 8.8 11.7 11.1 12.3 5.2 8.1 0.8 6.3 ...
## ..$ SST : num [1:20] 9.7 4.6 6.9 9.4 8.6 8.6 5.6 6.9 1.8 6.3 ...
## ..- attr(*, "terms")=Classes 'terms', 'formula' language EDTA ~ SST
## ...- attr(*, "variables")= language list(EDTA, SST)
## ....- attr(*, "factors")= int [1:2, 1] 0 1
## ....- attr(*, "dimnames")=List of 2
## .....$ : chr [1:2] "EDTA" "SST"
## .....$ : chr "SST"
## ....- attr(*, "term.labels")= chr "SST"
## ....- attr(*, "order")= int 1
## ....- attr(*, "intercept")= int 1
## ....- attr(*, "response")= int 1
## ....- attr(*, ".Environment")=<environment: R_GlobalEnv>
## ....- attr(*, "predvars")= language list(EDTA, SST)
## ....- attr(*, "dataClasses")= Named chr [1:2] "numeric" "numeric"
## ....- attr(*, "names")= chr [1:2] "EDTA" "SST"
## - attr(*, "class")= chr "lm"
```

This is probably more information than you wanted for right now, but it's a fairly complete list of the things that you might like to know at some point. To take a look at your data and add the regression line, you can type:

```
plot(EDTA ~ SST, data = tube_data, main = "OLS Regression")
#to add the regression line
abline(OLS_reg$coefficients)
# abline(OLS_reg$coefficients[1], OLS_reg$coefficients[2]) does same thing
# abline(OLS_reg) also does the same but is less intuitive
# lines(tube_data$SST,OLS_reg$fitted.values) does the same thing
#
# to add the line of identity...
abline(0, 1, col = "red", lty = 2)
```

OLS Regression



If you enter `plot(OLS.reg)`, you can cycle through some diagnostic plots of the regression.

4.2 Weighted Least Squares

Now, sometimes you want to perform weighted regression. This is what we do on the mass spectrometer when we want to improve the recoveries of the low-level calibrators and the expense of the high level calibrators. In other words, when accuracy at the low end is clinically required, we weight the regression. How we weight is fairly arbitrary, but the bigger the weight, the more fitting effect a point has.

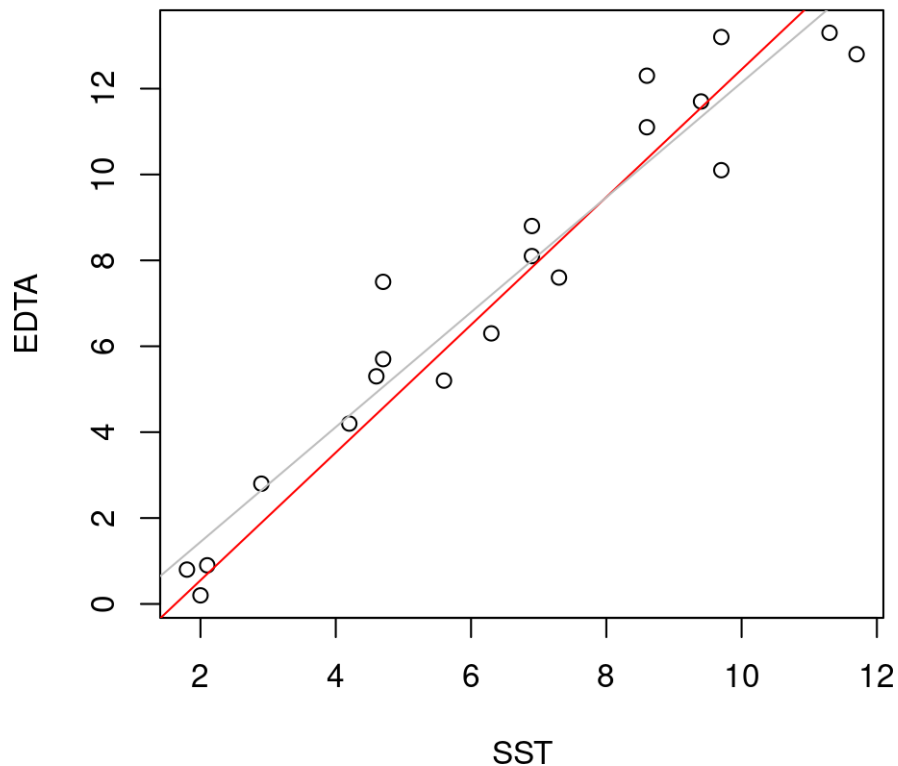
```
w_OLS_reg <- lm(EDTA ~ SST, data = tube_data, weights = 1/EDTA)
summary(w_OLS_reg)
```

```
##
## Call:
## lm(formula = EDTA ~ SST, data = tube_data, weights = 1/EDTA)
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -0.7859 -0.2977  0.1973  0.4062  1.0712
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -2.42248    0.27376  -8.849 5.66e-08 ***
## SST          1.48698    0.07372  20.170 8.32e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 0.5016 on 18 degrees of freedom
## Multiple R-squared:  0.9576, Adjusted R-squared:  0.9553
## F-statistic: 406.8 on 1 and 18 DF,  p-value: 8.319e-14
```

```
plot(EDTA ~ SST, data = tube_data, main = "Weighted OLS Regression")
#to add the regression line
abline(w_OLS_reg, col = "red")
#put the unweighted line in for comparison
abline(OLS_reg, col = "gray")
```

Weighted OLS Regression

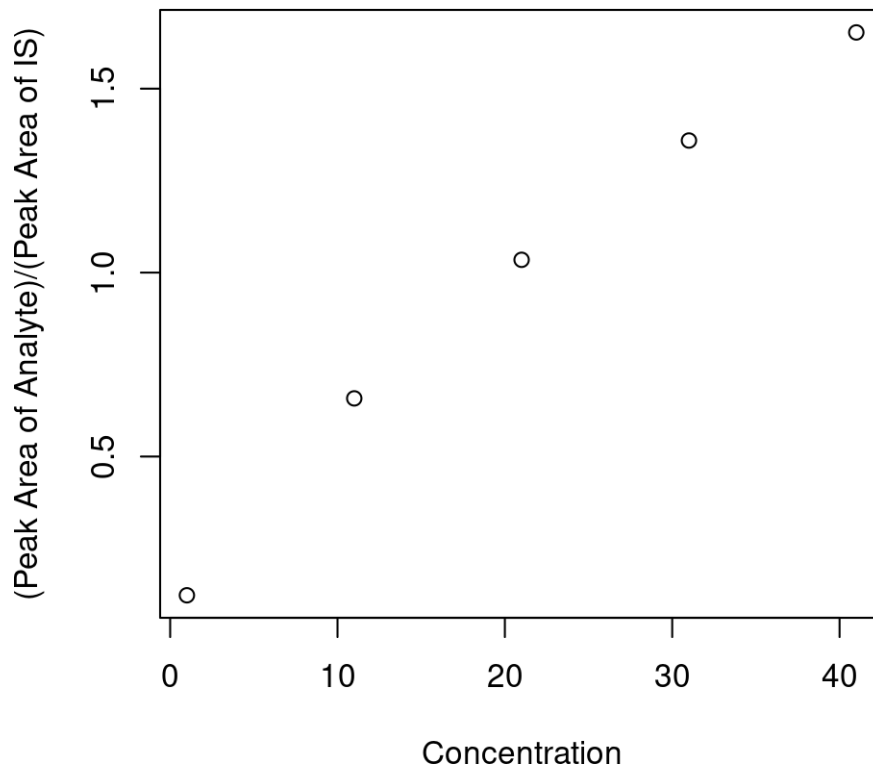


#What do you notice?

4.2.1 Exercise

- Here is some fake cal curve data that shows the characteristic flattening we see when we are trying to extend our linear range too far:

```
conc <- seq(from = 1, to = 50, by = 10)
response <- (conc/20)^(0.7)
plot(conc, response, xlab = "Concentration",
      ylab = "(Peak Area of Analyte)/(Peak Area of IS)")
```



- Find and plot the OLS regression line. Add it to the plot using `abline()` in green.
- Find and plot the $1/x^2$ weighted OLS regression line. Add it to the plot using `abline()` in purple.
- What is the effect of the weighting?

4.3 Outlier Effects in OLS

Outlier effects are significant with OLS regression. To illustrate, we can do the following (you don't need to understand the code, you just need to see the effect an outlier has):

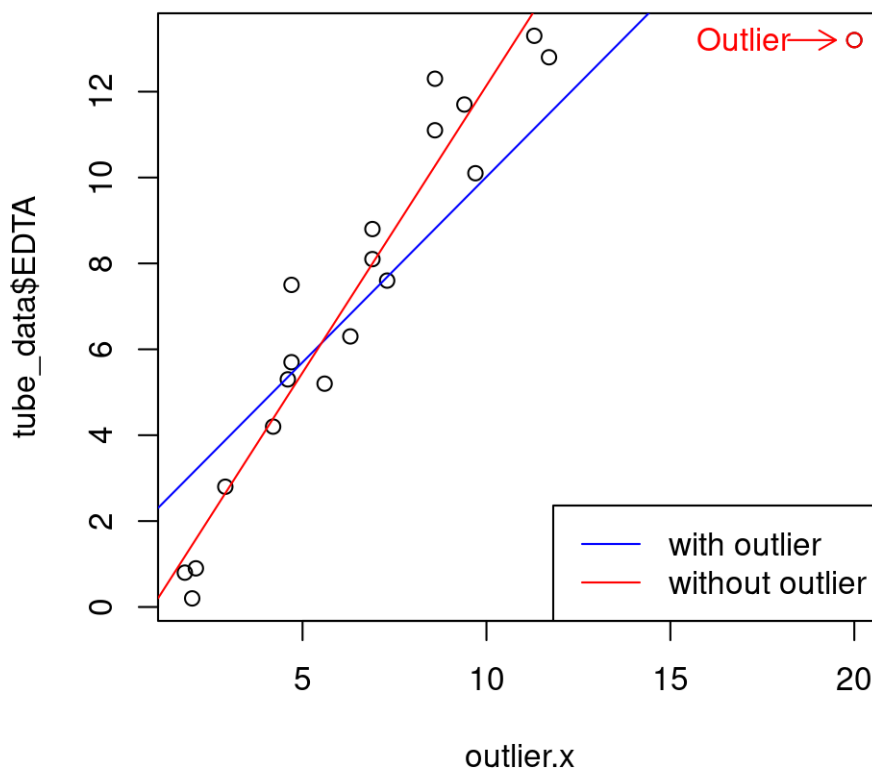
```
tube_data <- read.csv(file = "Data_Files/tube_data.csv")
outlier.x <- tube_data$SST
outlier.x[1] <- 20 #introduce a single outlier point
summary(lm(tube_data$EDTA ~ outlier.x))
```

```
##
## Call:
## lm(formula = tube_data$EDTA ~ outlier.x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.4511 -1.0334  0.1041  1.6111  3.4931
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.3805     0.9574   1.442   0.167
## outlier.x      0.8635     0.1180   7.320 8.47e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.198 on 18 degrees of freedom
```

```
## Multiple R-squared:  0.7486, Adjusted R-squared:  0.7346
## F-statistic: 53.59 on 1 and 18 DF,  p-value: 8.471e-07

plot(outlier.x, tube_data$EDTA, main = "Effect of an outlier")
abline(lm(tube_data$EDTA ~ outlier.x), col = "blue") #regression with outlier
abline(OLS_reg, col = "red") #regression without outlier
legend("bottomright",
      c("with outlier", "without outlier"),
      lty = c(1, 1),
      col = c("blue", "red"))
points(20, 13.2, col = "red")
text(17, 13.2, "Outlier", col = "red")
arrows(x0 = 18.2,
       y0 = 13.2,
       x1 = 19.5,
       y1 = 13.2,
       length = 0.1,
       col = "red")
```

Effect of an outlier

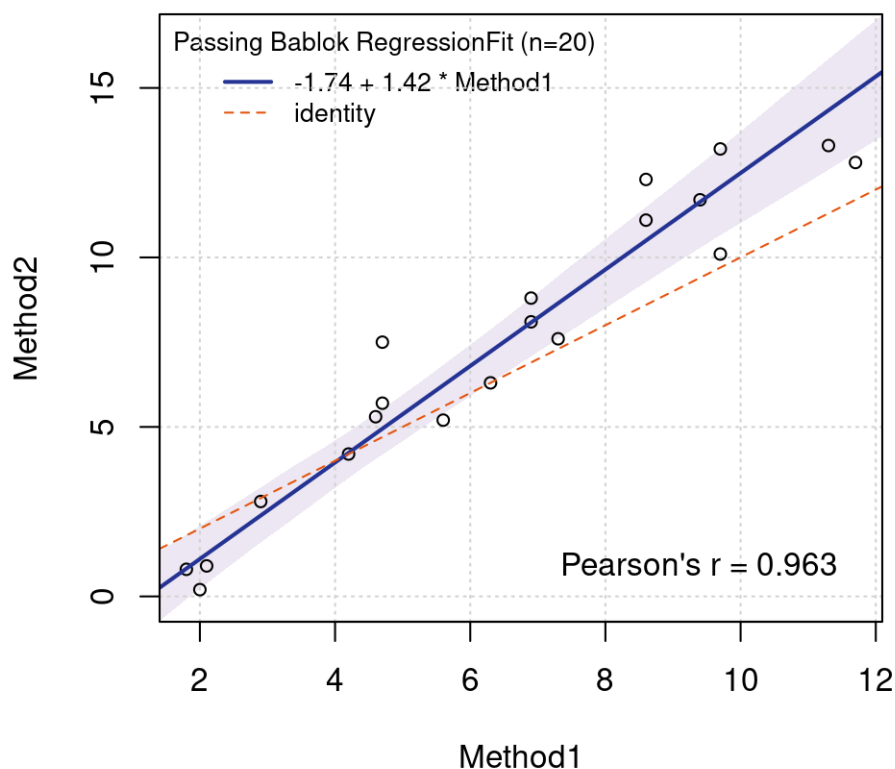


4.4 Passing Bablok Regression

Now that you have seen the effect of an outlier, you can see that it would be great to have a regression method that more resistant to the effect of outliers. Passing Bablok regression is such a method. However, this function is not built-in to R. The good thing is that there are packages that include it and we can install them. Statisticians at Roche have contributed a package called “mcr” that contains PB regression (remember to run `install.packages("mcr")` before this next step).

```
library("mcr")
PB_reg <- mcreg(tube_data$SST, tube_data$EDTA, method.reg = "PaBa")
plot(PB_reg) # plots a nice generic plot automatically
```

Passing Bablok Regression Fit



0.95-confidence bounds are calculated with the bootstrap(quantile)

4.4.1 Exercise

- Use the `mcreg()` function to show that PB regression is more resistant to an outlier than OLS regression. For your x data use `outlier.x` and for your y data use `tube_data$EDTA`. Plot the regression line. Use `abline()` to add the regression line from the `PB.reg` model shown immediately above.

Note that PB regression cannot be weighted by virtue of how the method works. There is no minimization of residuals, so there is nothing to weight. PB regression is very computationally intensive for larger data sets. For this reason, the code authors of the “mcr” package have developed a method called **PaBaLarge** for large data sets. It’s not exact, but it’s very close. It would be called like this:

```
PB.reg <- mcreg(tube_data$SST, tube_data$EDTA, method.reg = "PaBaLarge")
```

4.5 Deming Regression

OLS regression assumes that there is no error in the x-axis data. This is only true if the x-axis is mass spectrometry and the y-axis is an immunoassay (ba-dum-*ching*). OK—that was facetious. Deming regression also assumes that the ratios of the variances (i.e. CVs) is known for the two methods. This can only be meaningfully known if both x and y results are run in duplicate. Generally we don’t do this because of the expense. For the most part, if the precision behavior of the two methods is approximately

the same, then this value, called δ , is taken to be its default value of 1. The “mcr” package has both a Deming and a weighted Deming regression.

```
Deming_reg <- mcreg(tube_data$SST,
                  tube_data$EDTA,
                  method.reg = "Deming")
WDeming_reg <- mcreg(tube_data$SST,
                  tube_data$EDTA,
                  method.reg = "WDeming")
```

4.6 Preparing a Difference Plot

It is frequently necessary to prepare a difference plot of two methods as part of our work. Difference plots usually display the average of two methods on the x-axis and the difference of two methods on the y-axis—the difference can be expressed as an absolute or a percent. If one of the two methods happens to be a reference method, then the x-axis would display the values of the reference method rather than the average of the two methods.

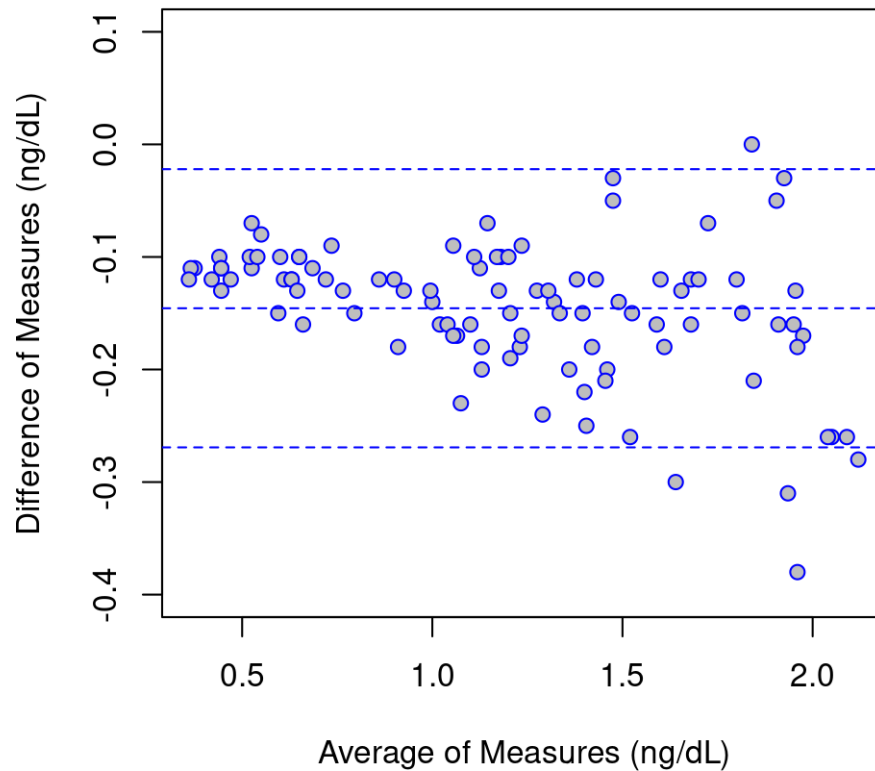
Let’s suppose we are comparing two creatinine methods. Let’s read in the data from the prepared csv file “cre_data.csv”. Remember that you have you use `setwd()` to get yourself to the right directory.

4.6.1 Exercise

1. Create a vector called `avg`, which stores the average of the creatinine values of Method 1 and Method 2. Use `cbind()` to append this vector to your dataframe.
2. Create a vector called `diff`, which stores the difference of the creatinine values of Method 1 and Method 2 (i.e. Method1 - Method2). Use `cbind()` to append the vector to your dataframe.
3. Use `plot()` to create a scatter plot of `diff` vs `avg`
4. What does the “fanning” of the difference as `cre` increases mean? What is the technical term for this phenomenon of inconstant scatter?

Now, we are not done. The plot does not look very nice. It also does not have a horizontal line at the mean difference and it does not have the upper and lower confidence intervals of the mean. We want it to look something like this ultimately:

Difference Plot



We now need the standard deviation of the difference and the average difference. We will add these to our graph with `abline()`.

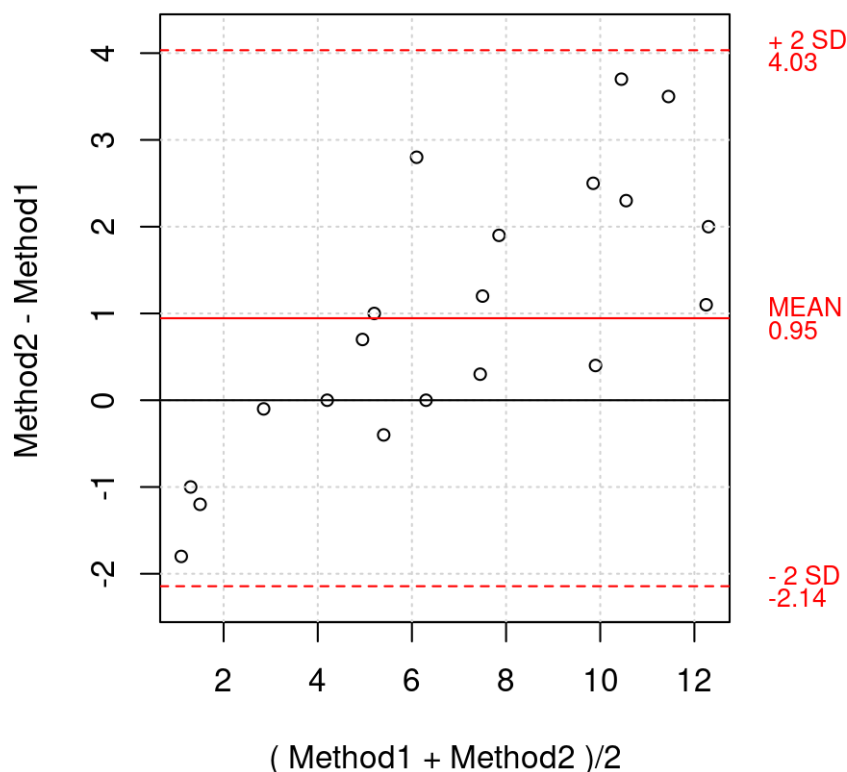
To add the mean difference we can type:

```
abline(h = mean(cre$diff))
```

As a matter of simplicity, the `mcr` package has a function for producing difference plots:

```
library(mcr)
tube_data <- read.csv(file = "Data_Files/tube_data.csv")
PB_reg <- mcreg(tube_data$SST, tube_data$EDTA, method.reg = "PaBa")
MCResult.plotDifference(PB_reg)
```

Difference plot



4.6.2 Exercise

- The `abline()` function can be used to make both horizontal and vertical lines. The syntax is `abline(h = 5)` for a horizontal line at $y = 5$ and `abline(v = 3)` for a vertical line at $x = 3$. Whenever you are drawing a line, you can add the parameter `lty = 1` through `6` to change the appearance of the line. For example, `abline(h = 4, col = "hotpink", lty = 2)` makes a horizontal dashed line in hot pink at $y = 4$. Groovy.

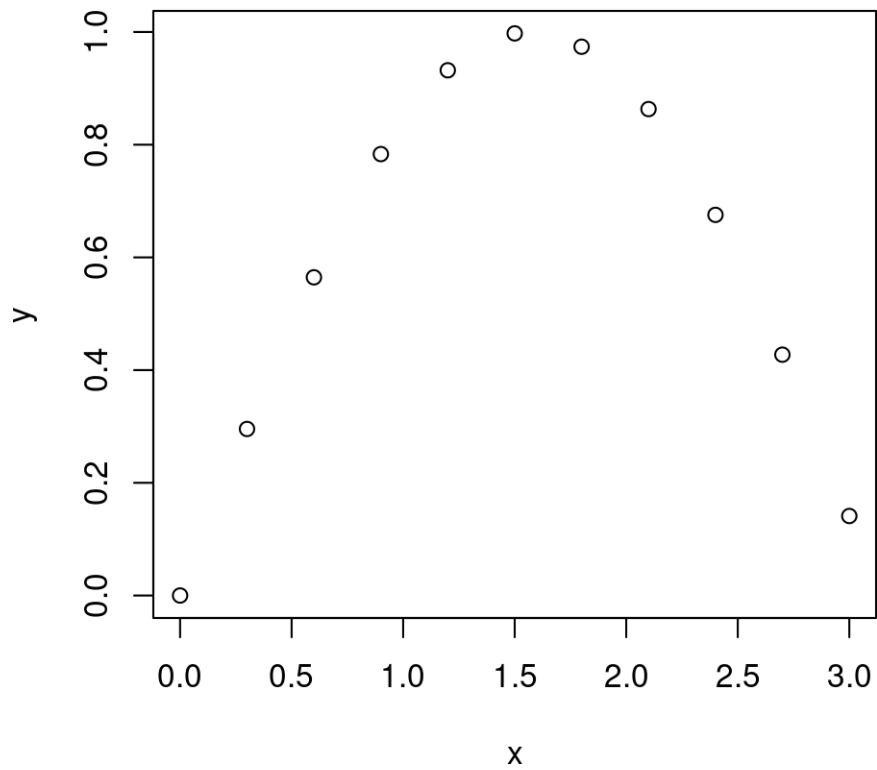
Calculate the SD of the difference and add horizontal lines at the mean + 2SD and mean - 2SD. Use `abline()` to add the horizontal lines to the plot. Make the color blue and add the parameter `lty = 2` to make the horizontal lines dashed.

4.7 Non-Linear Regression

It is more than occasionally necessary to fit a series of points that lie on a curve. Now, you have to have some idea what shape of function you want to fit to your data... but if you do, based on some reasoned physical principle, you can determine the “best fit” using any function you want: polynomial, exponential, sinusoidal, etc.

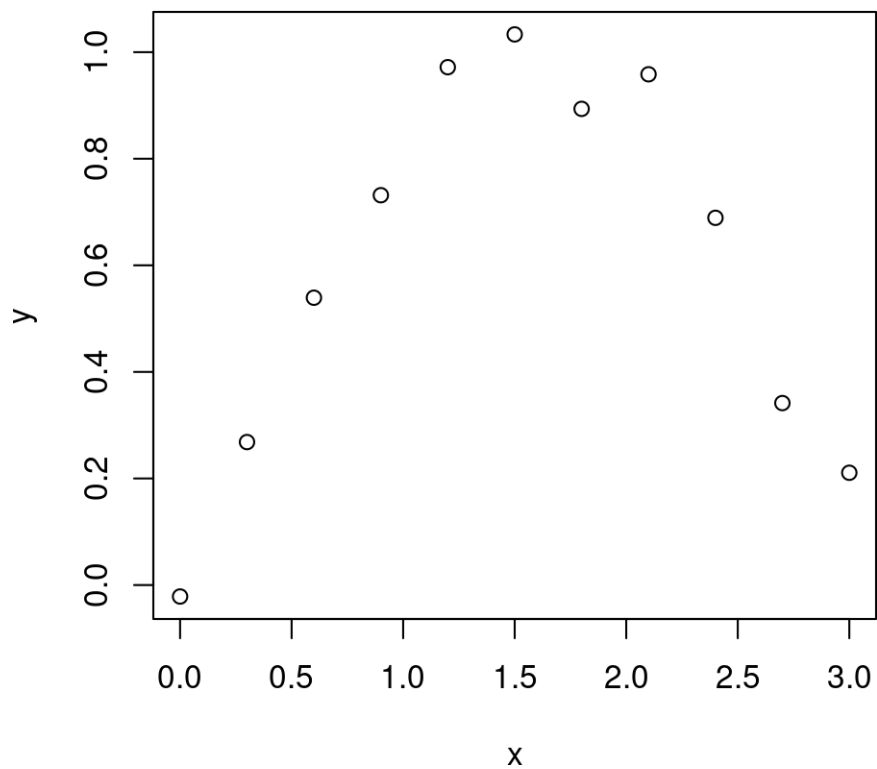
Just to illustrate the principle, let’s take a few points that lie on the curve $y = \sin(x)$. To do this we type:

```
x <- seq(from = 0, to = pi, by = 0.3)
y <- sin(x)
plot(x,y)
```



Now we want to add a little “noise” like we would have in real data sets... for example, if this data had come from the angle of a pendulum with a vertical line as a function of time or something.

```
y <- sin(x) + rnorm(11,0,0.05) #11 points, mean of 0, SD of 0.05
plot(x,y)
```



Now we can invoke the `nls()` function, which performs non-linear least squares. The syntax is `nls(formula, data, start)`. The `formula` is the functional form you want to fit to with unknowns included as variables, the `data` is the data you are fitting (in this case `x` and `y`), and `start` is a list of your best initial guesses for your unknowns.

In our case we are going to make no assumptions about our `sin` function. We will say that $y = A \sin(kx + B) + C$, where A, B, C and k are unknowns. We will just guess that A, B, C and k are 1 for the simplicity. We could do better, but it illustrates the process.

```
sin.data <- data.frame(x,y)
nl.reg <- nls(y ~ A*sin(k*x+B)+C,
             data = sin.data,
             start = list(A = 1,B = 1,C = 1,k = 1),
             trace = TRUE)

## 11.19449      (1.85e+01): par = (1 1 1 1)
## 1.312533      (6.62e+00): par = (0.5003102 0.1069116 0.02479001 1.034135)
## 0.09871427    (1.51e+00): par = (0.4138918 -1.208629 0.5925691 1.727233)
## 0.06646956    (1.09e+00): par = (0.4979158 -0.8422831 0.5135715 1.512746)
## 0.04776411    (7.53e-01): par = (0.6584595 -0.4445437 0.3591367 1.2677)
## 0.03048637    (7.20e-03): par = (0.7065534 -0.4586217 0.3116418 1.273789)
## 0.03048485    (3.90e-04): par = (0.7084927 -0.4534716 0.3095397 1.270808)
## 0.03048484    (2.82e-05): par = (0.7083147 -0.4538873 0.3097338 1.271051)
## 0.03048484    (2.10e-06): par = (0.7083285 -0.4538564 0.3097188 1.271033)

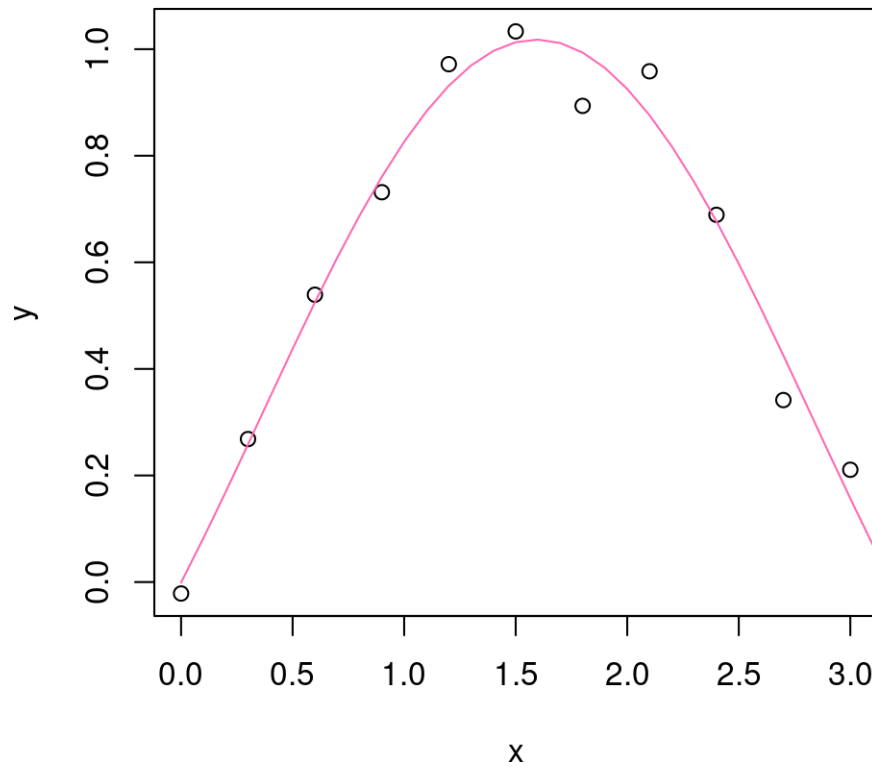
summary(nl.reg)
```

```
##
## Formula: y ~ A * sin(k * x + B) + C
##
## Parameters:
##   Estimate Std. Error t value Pr(>|t|)
## A    0.7083     0.2419   2.928  0.02210 *
## B   -0.4539     0.5126  -0.885  0.40534
## C    0.3097     0.2612   1.186  0.27435
## k    1.2710     0.3167   4.013  0.00511 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06599 on 7 degrees of freedom
##
## Number of iterations to convergence: 8
## Achieved convergence tolerance: 2.105e-06
```

The parameter `trace = TRUE` shows the work that `nls()` is doing as it tries to find a solution from the start values. If the start values really stink, `nls()` will choke and tell you so. You will need to find better guesses for the start values. There are ways to accomplish this which are outside the scope of the present discussion.

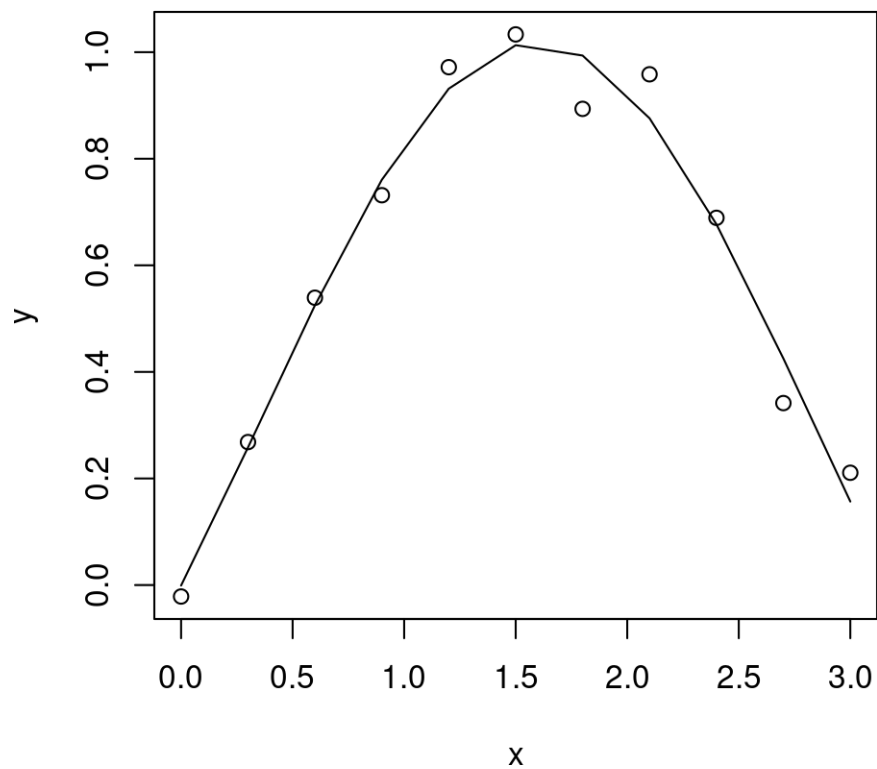
So now we can show our fit:

```
plot(x,y)
x.fit <- seq(from = 0, to = pi, by = 0.1) #make a bunch of tightly spaced points
y.fit <- 0.7083 * sin(1.2710 * x.fit - 0.4539) + 0.3097
lines(x.fit, y.fit, col = "hotpink")
```



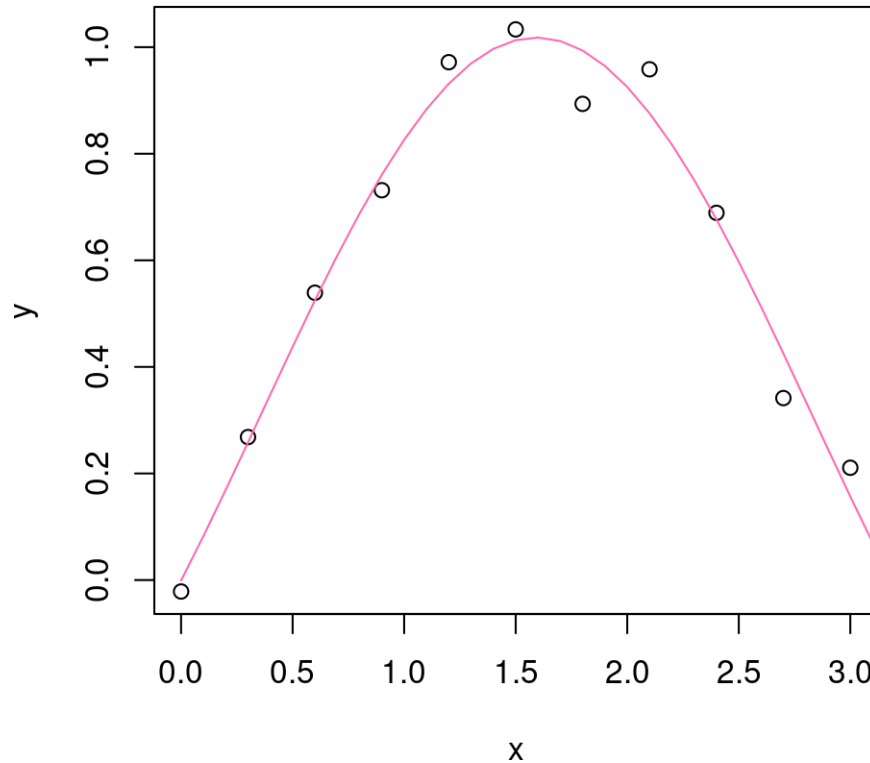
A less aesthetically pleasing but faster way to get this is to plot the fitted values from the `nl.reg()` output.

```
plot(x,y)
lines(x, predict(nl.reg))
```



...or to make a smooth curve:

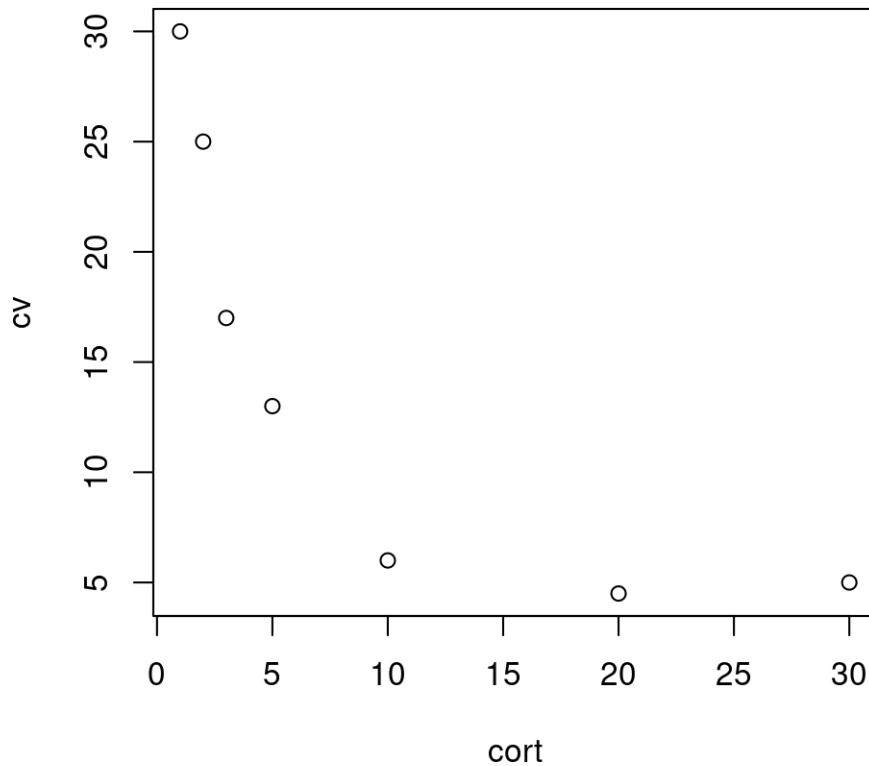
```
plot(x,y)
x.fit <- seq(from = 0, to = pi, by = 0.1)
y.fit <- predict(nl.reg , list(x = x.fit)) #saves effort of transcribing A,B, C and k
lines(x.fit, y.fit, col = "hotpink")
```



4.7.1 Exercise

- Suppose that you have the following data for the precision of your cortisol assay at different concentrations. In Canada we live in a nmol/L world, so we'll use those units just to keep America as confused as possible. Enter the following initial data into R.

```
cort <- c(1,2,3,5,10,20,30)
cv <- c(30, 25, 17,13,6,4.5,5)
cv.data <- data.frame(cort,cv)
plot(cort,cv)
```



...giving this graph.

- Fit this data to a hyperbolic function of the form $y = A/(x - B) + C$. If your start list for A , B and C gives you errors, try fitting to $y = A/x$ without B and C and then use your result for A with B and $C = 0$.
- Calculate the fitted values of y from `x.fit <- seq(from = 0, to = 30, by = 1)`.
- Draw the fitted curve in the color “coral”.
- Do your best to estimate the functional sensitivity – the concentration at which the CV = 20%.

5 Things with Strings and Tools for Data Cleansing

5.1 Working with Strings

In programming, the word ‘string’ refers to anything treated as text. Strings may be one word or several words, and can include other characters as well. For example, the word “computer” is a string, as is the sentence “I have 4 computers.” There are an array of tools in R specifically for working with strings.

For example, suppose you have to generate 100 jpeg figures using a loop (we will be covering this kind of thing shortly) and you need to name them according to the index of the loop. In this circumstance you are going to need to string together some variables to make the names. The base-r function `paste()` does this for us as does `str_c()` from the `stringr` package. For example:

```
number <- seq(from = 99, to = 95, by = -1)
for (i in number) {
  phrase <- paste(i, "bottles of beer on the wall", i,
                  "bottles of beer. \nTake one down, pass it around.",
                  i-1, "bottles of beer on the wall \n", sep = " ")
  cat(phrase)
}
```

```
## 99 bottles of beer on the wall 99 bottles of beer.
```

```
## Take one down, pass it around. 98 bottles of beer on the wall
## 98 bottles of beer on the wall 98 bottles of beer.
## Take one down, pass it around. 97 bottles of beer on the wall
## 97 bottles of beer on the wall 97 bottles of beer.
## Take one down, pass it around. 96 bottles of beer on the wall
## 96 bottles of beer on the wall 96 bottles of beer.
## Take one down, pass it around. 95 bottles of beer on the wall
## 95 bottles of beer on the wall 95 bottles of beer.
## Take one down, pass it around. 94 bottles of beer on the wall
```

In other words, `paste()` lets us create uninterrupted strings from other strings or variables. Here is a more practical example where we need to make a bunch of jpegs all at once. The script creates a directory so that you don't have to delete all the jpegs we are going to make. Paste automatically uses a space between all the things you are pasting together. This is not always what you want so you can specify with `sep = "blablabla"`.

```
paste("When my boss is talking my brain says, '", "'. It's like I am dying inside.",
      sep = "Bla Bla Bla")
```

```
## [1] "When my boss is talking my brain says, 'Bla Bla Bla'. It's like I am dying inside."
```

Here is a slightly more practical example:

```
dir.create("my_figures") # create a junk folder in the current working directory.
#to create a directory elsewhere you would need the full path

for (i in 1:10) {
  random.data <- rnorm(1000,i,0.3) #create Gaussian random data
  #with a mean of i and an SD of 0.3
  file.name <- file.path("my_figures",
                        paste("figure_number", i, ".jpg", sep = " "))
  print(file.name) #for your benefit to see what is going on
  jpeg(file.name) #open a jpeg file to dump the image into
  hist(random.data, col = rainbow(10)[i]) #if you want base-r style graphics
  dev.off() #close the jpeg file
}
```

Now go into the “my_figures” directory and see what has happened. You will notice that R takes care of the fact that Mac/Linux and Windows platforms use different kinds of slashes in the file structures.

There are other cool things that you can do with strings. The `strsplit()` function break strings apart based on separators within the string:

```
line <- "Can I please have a shot of ouzo. This class is killing me"
split.line <- strsplit(line, " ")
split.line
```

```
## [[1]]
## [1] "Can"      "I"        "please"   "have"     "a"        "shot"     "of"
## [8] "ouzo."    "This"     "class"    "is"       "killing"  "me"
```

At times it is necessary to reduce all letters to capitals or to smalls. The functions that can accomplish this are `toupper()` and `tolower()`. This is particularly useful to process data for further analysis. For example, if you had survey data containing “Yes”, “YES”, “yes”, “Y”, and “y” and wanted all of them to be turned into “Y” so that you could do statistics later, you might do something like this:

```
library(stringr)
responses <- c("Yes", "no", "y", "Y", "n", "N", "NO", "Y", "YES", "Y")
```

```

#clean up this mess
responses <- str_to_upper(responses)
responses

## [1] "YES" "NO" "Y" "Y" "N" "N" "NO" "Y" "YES" "Y"

responses <- str_replace(responses, "YES", "Y")
responses <- str_replace(responses, "NO", "N")
# alternatively using pure base R
# responses <- toupper(responses)
# responses <- gsub("YES", "Y", responses)
# responses <- gsub("NO", "N", responses)
responses

## [1] "Y" "N" "Y" "Y" "N" "N" "N" "Y" "Y" "Y"

```

Thought: What if you had a lot more possible responses that all started with “Y” but all meant “Yes”, like “Yo”, “Yepper”, “Yessiree”? Did you know that you can actually look for the pattern “Y” followed by any number of other characters? You can. It’s called a “Regular Expression”, and just to show you the magic:

```

responses <- c("Yes", "no", "y", "y", "n", "N", "NO", "Y", "YES", "Y", "yes sir",
               "yep", "YO", "Nay Laddie", "Nej", "Nein", "Non", "Yayaya",
               "Nee", "No way Jose", "Nah", "Nei", "nãõ")
responses <- str_replace(responses, "^[Yy].*", "Y")
responses <- str_replace(responses, "^[Nn].*", "N")
responses #magic

## [1] "Y" "N" "Y" "Y" "N" "N" "N" "Y" "Y" "Y" "Y" "Y" "Y" "N" "N" "N" "N" "Y" "N"
## [20] "N" "N" "N" "N"

# ^ means "starts with"
# [Yy] means "Y" or "y" and [Nn] means "N" or "n"
# . means "followed by anything"
# * means "the preceding character can occur 0 or more times"

```

This is just meant to show you that stuff like this is possible. You can learn about it another time.

The `substr()` function has syntax of `substr(x, start, stop)`. This can be conveniently used to get part of a string when what you want is stuck in the middle. For example if you wanted the last 5 characters of an 18 character string—because it contained, say, the time—you could do something like this:

```
substr("Feb,02,2015 07:34",14,18)
```

```
## [1] "07:34"
```

If you don’t happen to know how long the string you are breaking up is, you can use the `nchar()` function. The `length()` function won’t work; it tells you how many elements are in a vector. You want `str_length()`.

```
nchar("Here kitty kitty. Pspspsp")
```

```
## [1] 26
```

```
length("Here kitty kitty. Pspspsp")
```

```
## [1] 1
```

5.1.1 Exercise

- Convert a full name (let's start with `name <- "William Osler"`) to an abbreviated name made up of the first three letters of the first name and the first the letters of the last name, all in lower case. For example “Stephen Master” would become “stemas”.

NOTE: the `strsplit()` function produces a list as its output which we find kind of inconvenient generally. To turn it back into a vector just apply the command `unlist` to it. ie:

```
x <- strsplit("Dan Holmes", " ") #creates a list
str(x)
```

```
## List of 1
## $ : chr [1:2] "Dan" "Holmes"
```

```
x[[1]][1]
```

```
## [1] "Dan"
```

```
x[[1]][2] #inconvenient syntax
```

```
## [1] "Holmes"
```

```
x <- unlist(x)
x
```

```
## [1] "Dan" "Holmes"
```

```
x[1]
```

```
## [1] "Dan"
```

```
x[2] #happy syntax
```

```
## [1] "Holmes"
```

5.2 Oh `grep()`, how I love thee. Let me count the “wheys”

The function `grep()` is a very useful function that allows you to look for patterns rather than an exact match. This can be very helpful when the value of interest is buried in other text. For example, you might want to pull out anything that looks like a Personal Health Number or telephone number.

The syntax for `grep()` is

```
grep(pattern-to-find, strings)
```

Note that `grep()` returns the *indices* of the pattern matches in the vector. For example, let's pull the entire web of science publications list and then find all the journals that contain the word “ophthalmology”.

This code will download the Web of Science publication abbreviations database

```
if(!file.exists("Data_Files/wos_abbrev_table.txt")){
  download.file("https://ndownloader.figshare.com/files/5212423",
               "wos_abbrev_table.csv")
}
wos <- read.delim("Data_Files/wos_abbrev_table.txt",
                 sep = ";",
                 header = TRUE)
```

Now, let's find all the journals with the word ophthalmology in them.

```
ophtho_journals <- grep("[Oo]phthalmology", wos$full)
ophtho_journals[1:10] # show the first 10
```

```
## [1] 5533 7277 7512 8419 8420 8421 9804 11081 11139 13309
```

```
length(optho_journals)
```

```
## [1] 58
```

If we want to find the *names* of these journals, it can be accomplished in two ways:

```
# wos[optho.journals,1]  
# or  
# wos[optho.journals,]$full  
wos[optho_journals,]$full[1:10] # names of the first 10
```

```
## [1] "Advances in Ophthalmology"  
## [2] "Ama Archives of Ophthalmology"  
## [3] "American Journal of Ophthalmology"  
## [4] "Annals of Ophthalmology"  
## [5] "Annals of Ophthalmology & Glaucoma"  
## [6] "Annals of Ophthalmology-glaucoma"  
## [7] "Archives of Ophthalmology"  
## [8] "Australian and New Zealand Journal of Ophthalmology"  
## [9] "Australian Journal of Ophthalmology"  
## [10] "Bmc Ophthalmology"
```

The tidyverse equivalent of `grep()` is `str_which()`. Consider also the functions `grepl()` and `str_detect` which return logical vectors. Sometimes you will need a logical vector and not the indices...you just need to believe me. It's going to happen.

5.2.1 Exercise

- Read in `starwars_menu.csv`. The data is the pre-selected menu for a banquet. Have a look at the data. The caterer needs to know how many individuals want “whey” but the “whey” is buried in things like “the whey, the shoots and the rice”.
 - How many people want “whey”?
 - Produce a dataframe of all the rows containing the word whey.
 - Which individuals chose “whey”?
 - Which individuals did not choose “whey”?

Finding exact matches is just scratching the surface of what `grep()` and `str_which()` can do. They can also be used to match sophisticated patterns using the same regular expressions tools that `gsub()` and `str_replace()` use. For example, blood collection specimen numbers in lab start with X,M,T,W,H,F, or S and are then followed by 3–6 digits. So we can make a `grep()` search that finds only this pattern so we can see what samples represent patient results (as opposed to quality control material).

```
x <- c("D123", "F12414", "X1324556", "M432", "H2141", "S1")
```

Some of these will match the accession numbers

```
# ^ means "starts with"  
# | means "OR"  
# [0-9] means any digit  
# {m,n} means previous item matches at least m and at most n times  
# $ means ends with  
pattern <- "[X|M|T|W|H|F|S][0-9]{3,6}$"  
grep(pattern, x)
```

```
## [1] 2 4 5
```

Again, Regular Expressions could be an entire two day course to itself. For now, just know that this type of advanced pattern finding is possible.

6 Meet the ‘tidyverse’ - Pivot, Join, Filter, and Clean

As we have said, there is a popular dialect/paradigm of R programming referred to as the “tidyverse” that is particularly useful for certain types of data summary and data visualization. It allows for rapid-high level commands accomplishing a great deal in few lines of highly readable code. The challenge of using tidyverse packages is that they are under very rapid development and this can mean that updates in the packages can cause your code to stop functioning. Additionally, many statistical packages use traditional “base R” and they may not cooperate with tidyverse output. However, in the context of our work in health care, we are usually doing fairly simple one-off reports for research or answering a specific question so using the tidyverse makes sense.

6.1 Packages

Before we can start using these functions we will have to install and load them. This is because the ‘tidyverse’ does not exist in ‘base’ R (that is, the pre-loaded set of function which are installed when you install the R program). Rather, the tidyverse is a set of packages. Packages are themselves sets of functions which have been written by whomever authored the package. These user-created functions have been ‘packaged’ and made available for anyone to download and use.

Packages are what makes R great. Lots can be done in base R but often to achieve what you want, you end up having to write your own functions. This is fine if you like programming and find coding your own functions enjoyable. Sometimes, though, we just want to ‘get it done’ - and whatever ‘it’ is that you are trying to do, chances are that someone out there has done it before and written a package for you to make it easier.

A final word about packages - they are only as good as the person who wrote them. The tidyverse is a group of packages which have been written by Hadley Wickham (who created RStudio) and his team - which means that we can rely on them being functional, updated and usually bug-free. The same cannot always be said of smaller packages, so if you are ever poking around for a package to fill a niche need, read the documentation and do some google searches to see how others have been able to work with the package before spending too much time trying to code with it.

6.1.1 Exercise

- Hopefully you installed the tidyverse when you first installed R as per the pre-course instructions. Just in case, the following code will install the tidyverse package if it is not already there.

```
if("tidyverse" %in% rownames(installed.packages()) == FALSE) {install.packages("tidyverse")}
```

This command asks R to download the most current version of the tidyverse from CRAN, which is an online package repository. Note that you need to be connected to the internet for this to work.

Since quite a few packages are being downloaded, this may take a few minutes. Perhaps a good time for a coffee break.

Once tidyverse is installed, we need to load it by calling library():

```
library(tidyverse)
```

You only need to install a package once, but you will need to load any packages you need each time you restart R.

6.2 pivot_longer() and pivot_wider()

Now that we are in the tidyverse, the first thing we want to introduce is what is meant by “tidy data”.

When humans prepare spreadsheet data, they typically have each row represent all the observations on single subject. This is usually pretty easy to look at but it happens to have a number of disadvantages from a statistical programming standpoint.

For example, in traditional “untidy” data, you might have each subject on a row and all of their lab tests represented as columns: Sodium, Potassium, Chloride, Bicarbonate, Creatinine, pH, Troponin etc. But in the tidy data paradigm, all of the blood tests should be factors under a single column labelled “Test” and then each row represents a single observation, not a single patient.

For example, this is a traditional view:

Patient	Sodium	Potassium	Chloride	Total CO2
1	143	4.2	110	25
2	129	4.4	100	18
3	142	4.6	105	27
4	148	5.0	111	25

And this is a tidy view of the same data:

Patient	Test	Result
1	Sodium	143.0
1	Potassium	4.2
1	Chloride	110.0
1	Total CO2	25.0
2	Sodium	129.0
2	Potassium	4.4
2	Chloride	100.0
2	Total CO2	18.0
3	Sodium	142.0
3	Potassium	4.6
3	Chloride	105.0
3	Total CO2	27.0
4	Sodium	148.0
4	Potassium	5.0
4	Chloride	111.0
4	Total CO2	25.0

It is frequently necessary to jump back and forth between the traditional view, which we call “data wide” and the tidy view which we call “data long”. This is accomplished with the functions `pivot_longer()` and `pivot_wider()`.

Let’s starting exploring these funtions by reading in some data on “anthropometric” (for want of a better term) measurements on opossum.

```
possum <- read.csv("Data_Files/possum.csv")
# we used read.csv here (a tidyverse function) instead of read.csv (base R)
# the anthropomorphic measurements of the possums are wide
head(possum)
```

```
##      X1 case site Pop sex age hdlngth skullw totlngth taill footlngth earconch
```

```
## 1 C3 1 1 Vic m 8 94.1 60.4 89.0 36.0 74.5 54.5
## 2 C5 2 1 Vic f 6 92.5 57.6 91.5 36.5 72.5 51.2
## 3 C10 3 1 Vic f 6 94.0 60.0 95.5 39.0 75.4 51.9
## 4 C15 4 1 Vic f 6 93.2 57.1 92.0 38.0 76.1 52.2
## 5 C23 5 1 Vic f 2 91.5 56.3 85.5 36.0 71.0 53.2
## 6 C24 6 1 Vic f 1 93.1 54.8 90.5 35.5 73.2 53.6
## eye chest belly
## 1 15.2 28.0 36
## 2 16.0 28.5 33
## 3 15.5 30.0 34
## 4 15.2 28.0 34
## 5 15.1 28.5 33
## 6 14.2 30.0 32
```

We can convert them to data long format with the `gather()` function. In the this function we must determine three things:

1. What columns are to be gathered together as factors of a similar type?—This parameter referred to as the “key” and in our output the column will be named “Possum_Metric”.
2. What do you want to call the column that contains the associated values?—This parameter is referred to as the “value” and in our output the column will be named “Result”.
3. Which columns (by name or number) are to be gathered?—In this case it is columns 7–15. You can also denote which columns you *don't* want gathered – so we could also write this as “-c(1:6)”.

```
library(tidyr)
possum_long <- pivot_longer(possum, cols = 7:15,
                             names_to = "bodypart",
                             values_to = "measurement")
head(possum_long, 10)

## # A tibble: 10 x 8
##   X1      case site Pop  sex   age bodypart measurement
##   <chr> <int> <int> <chr> <chr> <int> <chr>          <dbl>
## 1 C3      1     1 Vic  m     8 hdlngth      94.1
## 2 C3      1     1 Vic  m     8 skullw       60.4
## 3 C3      1     1 Vic  m     8 totlngth     89
## 4 C3      1     1 Vic  m     8 taill        36
## 5 C3      1     1 Vic  m     8 footlngth   74.5
## 6 C3      1     1 Vic  m     8 earconch    54.5
## 7 C3      1     1 Vic  m     8 eye         15.2
## 8 C3      1     1 Vic  m     8 chest        28
## 9 C3      1     1 Vic  m     8 belly        36
## 10 C5     2     1 Vic  f     6 hdlngth     92.5
```

As it turns out (and we will explain more later), this permits very slick calculations:

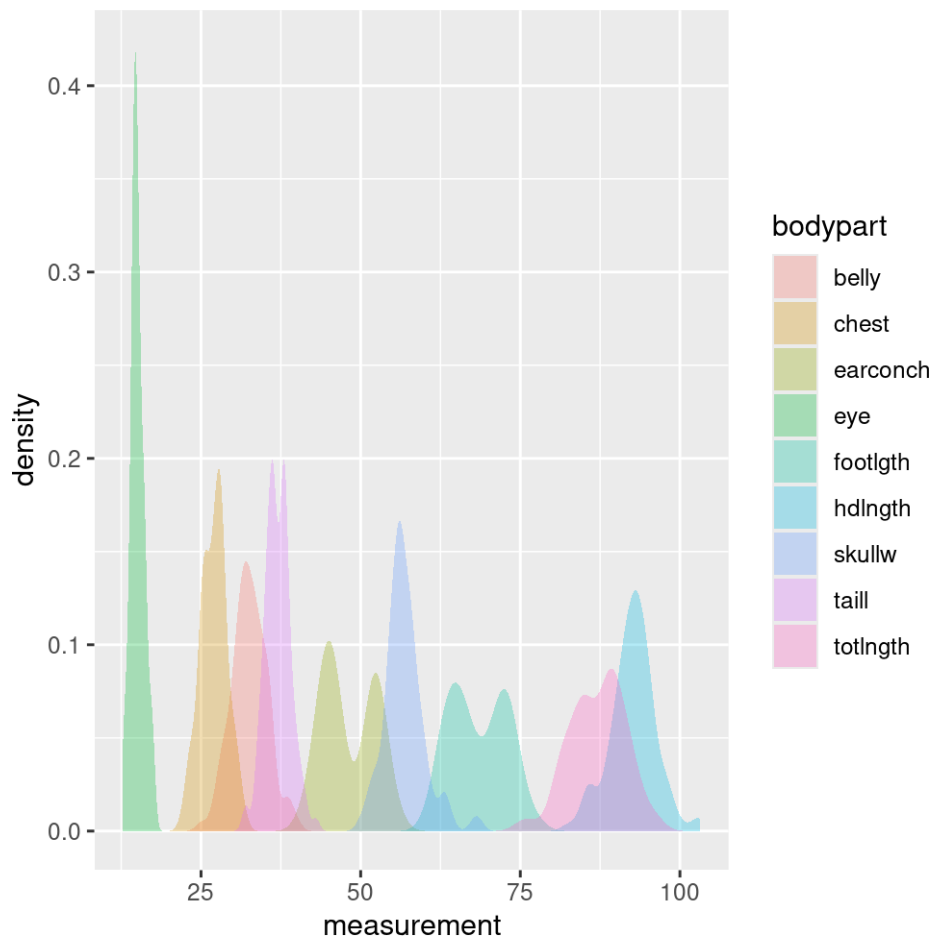
```
#long data permits rapid calculations
group_by(possum_long, bodypart) %>%
  summarize(mean = mean(measurement, na.rm = TRUE),
             median = median(measurement, na.rm = TRUE),
             sd = sd(measurement, na.rm = TRUE))

## # A tibble: 9 x 4
##   bodypart mean median sd
##   <chr>    <dbl> <dbl> <dbl>
## 1 belly    32.6   32.5  2.76
## 2 chest    27     27    2.05
```

```
## 3 earconch 48.1 46.8 4.11
## 4 eye 15.0 14.9 1.05
## 5 footlgth 68.5 68 4.40
## 6 hdlngth 92.6 92.8 3.57
## 7 skullw 56.9 56.4 3.11
## 8 taill 37.0 37 1.96
## 9 totlngth 87.1 88 4.31
```

and plots:

```
#long data permits rapid visualizations which we will cover later
library(ggplot2)
p <- ggplot(possum_long, aes(x = measurement, fill = bodypart)) +
  geom_density(alpha = 0.3, color = NA)
p
```



If we want to convert back to ‘untidy’ data, this is accomplished by `pivot_wider()`. For this function again we need to determine our “names_from” and “values_from” columns:

1. The “names_from” column will be pivoted wider across the top of the table (this column becomes the column names).
2. The “value_from” column contains the values we want to populate these new columns.

```
possum_wide <- pivot_wider(possum_long,
  names_from = bodypart,
  values_from = measurement)
```

```
head(possum_wide, 10)
```

```
## # A tibble: 10 x 15
##   X1      case site Pop  sex    age hdlngth skullw totlngth taill footlngth
##   <chr> <int> <int> <chr> <chr> <int>   <dbl>   <dbl>    <dbl> <dbl>    <dbl>
## 1 C3      1     1 Vic  m     8    94.1    60.4     89    36     74.5
## 2 C5      2     1 Vic  f     6    92.5    57.6    91.5   36.5    72.5
## 3 C10     3     1 Vic  f     6    94      60      95.5   39      75.4
## 4 C15     4     1 Vic  f     6    93.2    57.1     92    38     76.1
## 5 C23     5     1 Vic  f     2    91.5    56.3    85.5   36      71
## 6 C24     6     1 Vic  f     1    93.1    54.8    90.5   35.5    73.2
## 7 C26     7     1 Vic  m     2    95.3    58.2    89.5   36     71.5
## 8 C27     8     1 Vic  f     6    94.8    57.6     91    37     72.7
## 9 C28     9     1 Vic  f     9    93.4    56.3    91.5   37     72.4
## 10 C31    10     1 Vic  f     6    91.8    58      89.5   37.5    70.9
## # i 4 more variables: earconch <dbl>, eye <dbl>, chest <dbl>, belly <dbl>
```

6.3 arrange(), filter(), and select()

There are some more very useful and simple functions in the tidyverse which we will need before we get too much further.

The first is `arrange()` and it does pretty much what you think it would. Let's try it out on the possum data:

```
arrange(possum, age)
```

```
##      X1 case site Pop sex age hdlngth skullw totlngth taill footlngth
## 1   C24   6    1  Vic  f   1   93.1   54.8    90.5   35.5    73.2
## 2   C45  17    1  Vic  f   1   94.7   67.7    89.5   36.5    73.2
## 3  BB31  39    2  Vic  f   1   84.7   51.5    75.0   34.0    68.7
## 4  CD12  72    5 other m   1   85.9   52.4    80.5   35.0    62.0
## 5  CD13  73    5 other m   1   82.5   52.3    82.0   36.5    65.7
## 6  BSF5  78    6 other m   1   86.5   51.0    81.0   36.5    63.0
## 7  BSF6  79    6 other m   1   85.8   50.0    81.0   36.5    62.8
## 8  BSF7  80    6 other m   1   86.7   52.6    84.0   38.0    62.3
## 9  BTP16 100    7 other m   1   89.5   56.0    81.5   36.5    66.0
## 10 BTP17 101    7 other m   1   88.6   54.7    82.5   39.0    64.4
## 11  C23   5    1  Vic  f   2   91.5   56.3    85.5   36.0    71.0
## 12  C26   7    1  Vic  m   2   95.3   58.2    89.5   36.0    71.5
## 13  C47  18    1  Vic  m   2   93.5   55.7    90.0   36.0    73.7
## 14  C59  24    1  Vic  m   2   94.4   54.9    84.0   34.0    75.0
## 15  C63  27    1  Vic  f   2   90.5   54.5    85.0   35.0    70.3
## 16   A2  30    1  Vic  f   2   92.1   54.4    84.0   33.5    70.6
## 17  BB4  34    2  Vic  m   2   90.6   55.7    85.5   36.5    73.1
## 18 BB17  37    2  Vic  f   2   89.3   54.8    82.5   35.0    71.2
## 19 BB40  43    2  Vic  f   2   90.0   55.5    81.0   32.0    72.0
## 20  WW1  47    3 other m   2   90.1   54.8    89.0   37.5    66.0
## 21  BR2  55    4 other m   2  103.1   63.2    92.5   38.0    72.5
## 22  BR6  59    4 other m   2  102.5   62.8    96.0   40.0    73.2
## 23  BR7  60    4 other f   2   91.3   57.7    88.0   39.0    63.1
## 24  BTP1  87    7 other m   2   98.5   60.7    93.0   41.5    71.7
## 25  BTP3  88    7 other f   2   89.6   58.0    87.5   38.0    66.7
## 26  BTP7  92    7 other m   2   90.7   56.3    85.0   37.0    67.6
## 27  C37  14    1  Vic  m   3   95.4   57.6    91.5   36.0    74.3
```

## 28	C54	21	1	Vic	f	3	95.9	58.1	96.5	39.5	77.9
## 29	C55	22	1	Vic	m	3	96.3	58.5	91.0	39.5	73.5
## 30	C60	25	1	Vic	m	3	95.8	58.5	91.5	35.5	72.3
## 31	A1	29	1	Vic	f	3	92.8	56.0	88.0	35.0	74.9
## 32	A3	31	1	Vic	m	3	92.8	54.1	93.0	37.0	68.0
## 33	AD1	33	1	Vic	m	3	91.4	54.6	89.0	37.0	70.8
## 34	BB33	40	2	Vic	f	3	91.0	55.0	84.5	36.0	72.8
## 35	BB38	42	2	Vic	m	3	85.3	54.1	77.0	32.0	62.7
## 36	BB44	45	2	Vic	m	3	90.7	55.9	81.0	34.0	71.5
## 37	WW7	53	3	other	f	3	93.1	58.1	91.0	38.0	67.4
## 38	BR3	56	4	other	m	3	99.9	61.5	93.7	38.0	68.7
## 39	BR5	58	4	other	m	3	94.5	64.2	91.0	39.0	66.5
## 40	CD2	62	5	other	f	3	91.3	58.0	90.5	39.0	65.5
## 41	CD4	64	5	other	f	3	96.9	56.5	89.5	38.5	63.0
## 42	CD6	66	5	other	f	3	90.4	55.8	86.0	36.5	63.2
## 43	BSF8	81	6	other	m	3	90.6	56.0	85.5	38.0	65.6
## 44	BSF10	83	6	other	f	3	90.0	53.8	81.5	36.0	62.0
## 45	BSF11	84	6	other	m	3	88.4	54.6	80.5	36.0	62.6
## 46	BSF12	85	6	other	m	3	89.5	56.2	92.0	40.5	65.6
## 47	BSF13	86	6	other	f	3	88.2	53.2	86.5	38.5	60.3
## 48	BTP5	90	7	other	m	3	92.6	54.6	85.0	38.5	69.8
## 49	BTP6	91	7	other	m	3	97.8	59.6	89.0	38.0	65.5
## 50	BTP8	93	7	other	m	3	89.2	54.0	82.0	38.0	63.8
## 51	BTP13	97	7	other	m	3	91.0	53.1	86.0	38.0	63.8
## 52	BTP15	99	7	other	f	3	93.3	56.2	86.5	38.5	64.8
## 53	BTP21	104	7	other	f	3	93.6	59.9	89.0	40.0	67.6
## 54	C40	16	1	Vic	m	4	91.6	56.0	86.0	34.5	73.0
## 55	C50	20	1	Vic	f	4	94.8	56.3	89.0	38.0	73.8
## 56	C58	23	1	Vic	f	4	92.5	56.1	89.0	36.0	72.8
## 57	C64	28	1	Vic	m	4	93.8	56.8	87.0	34.5	73.2
## 58	A4	32	1	Vic	f	4	94.3	56.7	94.0	39.0	74.8
## 59	BB13	35	2	Vic	m	4	94.4	57.9	85.0	35.5	71.2
## 60	WW3	49	3	other	m	4	95.4	59.2	85.0	37.0	69.0
## 61	BR4	57	4	other	f	4	95.1	59.4	93.0	41.0	67.2
## 62	CD7	67	5	other	m	4	93.3	57.6	85.0	36.5	64.7
## 63	BSF1	74	6	other	f	4	88.7	52.0	83.0	38.0	61.5
## 64	BSF9	82	6	other	f	4	86.0	54.0	82.0	36.5	60.7
## 65	BTP10	95	7	other	m	4	91.6	56.6	88.5	37.5	64.5
## 66	BTP12	96	7	other	m	4	94.8	55.7	83.0	38.0	66.5
## 67	BTP20	103	7	other	m	4	91.5	55.2	82.5	36.5	62.9
## 68	C34	12	1	Vic	f	5	94.9	55.6	92.0	35.5	71.7
## 69	C36	13	1	Vic	m	5	95.1	59.9	89.5	36.0	71.0
## 70	C39	15	1	Vic	m	5	92.9	57.6	85.5	34.0	69.7
## 71	C48	19	1	Vic	f	5	94.4	55.4	90.5	35.0	73.4
## 72	BB36	41	2	Vic	f	5	88.4	57.0	83.0	36.5	NA
## 73	WW2	48	3	other	m	5	98.6	63.2	85.0	34.0	66.9
## 74	WW4	50	3	other	f	5	91.6	56.4	88.0	38.0	65.0
## 75	WW5	51	3	other	f	5	95.6	59.6	85.0	36.0	64.0
## 76	CD5	65	5	other	f	5	93.5	57.4	88.5	38.0	68.2
## 77	CD8	68	5	other	m	5	94.1	56.0	88.5	38.0	65.9
## 78	CD9	69	5	other	m	5	98.0	55.6	88.0	37.5	65.0
## 79	BSF3	76	6	other	m	5	92.4	56.8	89.0	41.0	64.5
## 80	BTP14	98	7	other	m	5	93.2	68.6	84.0	35.0	65.6
## 81	C5	2	1	Vic	f	6	92.5	57.6	91.5	36.5	72.5

## 82	C10	3	1	Vic	f	6	94.0	60.0	95.5	39.0	75.4
## 83	C15	4	1	Vic	f	6	93.2	57.1	92.0	38.0	76.1
## 84	C27	8	1	Vic	f	6	94.8	57.6	91.0	37.0	72.7
## 85	C31	10	1	Vic	f	6	91.8	58.0	89.5	37.5	70.9
## 86	WW6	52	3	other	m	6	97.6	61.0	93.5	40.0	67.9
## 87	CD3	63	5	other	f	6	92.0	56.4	88.5	38.0	64.1
## 88	CD11	71	5	other	m	6	92.8	57.6	90.0	40.0	65.7
## 89	BSF2	75	6	other	m	6	93.8	58.1	89.0	38.0	66.2
## 90	BSF4	77	6	other	m	6	93.6	56.2	84.0	36.0	62.8
## 91	BTP4	89	7	other	m	6	97.7	58.4	84.5	35.0	64.4
## 92	BTP19	102	7	other	f	6	92.4	55.0	89.0	38.0	63.5
## 93	C61	26	1	Vic	m	7	96.0	59.0	90.0	36.0	73.6
## 94	BB15	36	2	Vic	m	7	93.3	59.3	88.0	35.0	74.3
## 95	BB25	38	2	Vic	m	7	92.4	56.0	80.5	35.5	68.4
## 96	BR1	54	4	other	m	7	96.9	63.0	91.5	43.0	71.3
## 97	CD1	61	5	other	m	7	95.7	59.0	86.0	38.0	63.1
## 98	CD10	70	5	other	f	7	91.9	56.4	87.0	38.0	65.4
## 99	BTP9	94	7	other	m	7	91.8	57.6	84.0	35.5	64.2
## 100	C3	1	1	Vic	m	8	94.1	60.4	89.0	36.0	74.5
## 101	C28	9	1	Vic	f	9	93.4	56.3	91.5	37.0	72.4
## 102	C32	11	1	Vic	f	9	93.3	57.2	89.5	39.0	77.2
## 103	BB41	44	2	Vic	m	NA	85.1	51.5	76.0	35.5	70.3
## 104	BB45	46	2	Vic	m	NA	91.4	54.4	84.0	35.0	72.8
##	earconch	eye	chest	belly							
## 1		53.6	14.2	30.0	32.0						
## 2		53.2	14.7	29.0	31.0						
## 3		53.4	13.0	25.0	25.0						
## 4		42.4	14.1	25.5	30.0						
## 5		44.7	16.0	23.5	28.0						
## 6		44.3	13.2	23.0	28.0						
## 7		43.0	14.8	22.0	28.5						
## 8		44.8	15.0	23.5	30.5						
## 9		46.8	14.8	23.0	27.0						
## 10		48.0	14.0	25.0	33.0						
## 11		53.2	15.1	28.5	33.0						
## 12		52.0	14.2	30.0	34.5						
## 13		55.4	15.3	28.0	32.0						
## 14		53.5	16.2	27.0	32.0						
## 15		50.8	14.2	23.0	28.0						
## 16		50.8	14.5	24.5	33.0						
## 17		53.1	14.4	26.0	28.5						
## 18		52.0	13.6	28.0	31.5						
## 19		49.4	13.4	29.0	31.0						
## 20		45.5	15.0	25.0	33.0						
## 21		44.9	16.4	30.5	36.0						
## 22		44.5	14.7	32.0	36.0						
## 23		47.0	14.4	26.0	30.0						
## 24		46.8	15.0	26.0	36.0						
## 25		43.5	16.0	25.5	31.5						
## 26		46.8	14.5	25.5	31.0						
## 27		53.7	15.1	28.0	31.5						
## 28		52.9	14.2	30.0	40.0						
## 29		52.1	16.2	28.0	36.0						
## 30		51.6	14.9	31.0	35.0						

## 31	51.8	14.0	24.0	32.0
## 32	52.5	14.5	27.0	31.0
## 33	51.8	14.8	24.0	30.0
## 34	51.4	13.6	27.0	30.0
## 35	51.2	13.8	25.5	33.0
## 36	54.0	14.6	27.0	31.5
## 37	46.0	16.5	26.0	33.5
## 38	46.8	16.4	27.5	31.5
## 39	46.4	14.4	30.5	33.0
## 40	41.3	16.0	27.0	32.0
## 41	45.1	17.1	25.5	33.0
## 42	44.2	15.7	26.5	34.0
## 43	41.7	17.0	27.5	35.0
## 44	43.3	14.0	25.0	29.0
## 45	43.6	16.3	25.0	28.5
## 46	43.5	14.5	27.0	31.5
## 47	43.7	13.6	26.0	31.0
## 48	44.8	14.5	25.5	32.5
## 49	48.0	15.0	26.0	32.0
## 50	44.9	12.8	24.0	31.0
## 51	46.0	14.5	25.0	31.5
## 52	43.8	14.0	28.0	35.0
## 53	46.0	14.8	28.5	33.5
## 54	51.4	14.4	28.0	32.0
## 55	52.4	15.5	27.0	36.0
## 56	53.3	15.4	28.0	35.0
## 57	53.0	15.3	27.0	30.0
## 58	52.0	14.9	28.0	34.0
## 59	55.5	16.4	28.0	35.5
## 60	45.0	15.9	29.5	35.5
## 61	45.3	14.5	31.0	39.0
## 62	44.1	16.5	27.5	29.5
## 63	45.9	14.7	26.0	34.0
## 64	42.9	15.4	26.0	32.0
## 65	45.4	14.9	27.0	31.0
## 66	47.7	14.0	25.0	33.0
## 67	45.9	15.4	25.0	29.0
## 68	51.0	15.3	28.0	33.0
## 69	49.8	15.8	27.0	32.0
## 70	51.8	15.7	28.0	35.0
## 71	53.9	15.2	28.0	32.0
## 72	40.3	15.9	27.0	30.5
## 73	44.9	17.0	28.0	35.0
## 74	47.2	14.9	28.0	36.0
## 75	43.9	17.4	28.0	38.5
## 76	41.7	14.0	29.0	38.5
## 77	43.1	17.4	27.0	30.0
## 78	45.6	15.0	28.5	34.0
## 79	46.4	17.8	26.0	33.0
## 80	44.3	14.5	28.5	32.0
## 81	51.2	16.0	28.5	33.0
## 82	51.9	15.5	30.0	34.0
## 83	52.2	15.2	28.0	34.0
## 84	53.9	14.5	29.0	34.0

```
## 85      53.4 14.4 27.5 32.0
## 86      44.3 15.8 28.5 32.5
## 87      46.3 15.2 25.5 28.5
## 88      42.8 15.0 27.5 34.0
## 89      45.6 16.9 26.0 33.5
## 90      42.9 16.2 25.0 35.0
## 91      46.2 14.4 29.0 30.5
## 92      45.4 13.0 25.0 30.0
## 93      56.2 15.0 29.0 38.0
## 94      52.0 14.9 25.5 36.0
## 95      49.5 15.9 27.0 30.0
## 96      46.0 17.5 30.0 36.5
## 97      44.9 15.0 26.5 31.0
## 98      44.1 13.0 27.0 34.0
## 99      45.1 14.4 29.0 35.0
## 100     54.5 15.2 28.0 36.0
## 101     52.9 15.5 28.0 33.0
## 102     51.3 14.9 31.0 34.0
## 103     52.6 14.4 23.0 27.0
## 104     51.2 14.4 24.5 35.0
```

If we want to arrange from largest to smallest, we would ask for the ages in descending `desc()` order:

```
arrange(possum, desc(age))
```

```
##      X1 case site  Pop sex age hdlngth skullw totlngth taill footlngth
## 1    C28   9     1  Vic  f   9   93.4   56.3    91.5   37.0    72.4
## 2    C32  11     1  Vic  f   9   93.3   57.2    89.5   39.0    77.2
## 3     C3   1     1  Vic  m   8   94.1   60.4    89.0   36.0    74.5
## 4    C61  26     1  Vic  m   7   96.0   59.0    90.0   36.0    73.6
## 5   BB15  36     2  Vic  m   7   93.3   59.3    88.0   35.0    74.3
## 6   BB25  38     2  Vic  m   7   92.4   56.0    80.5   35.5    68.4
## 7    BR1  54     4 other m   7   96.9   63.0    91.5   43.0    71.3
## 8    CD1  61     5 other m   7   95.7   59.0    86.0   38.0    63.1
## 9   CD10  70     5 other f   7   91.9   56.4    87.0   38.0    65.4
## 10   BTP9  94     7 other m   7   91.8   57.6    84.0   35.5    64.2
## 11    C5   2     1  Vic  f   6   92.5   57.6    91.5   36.5    72.5
## 12   C10   3     1  Vic  f   6   94.0   60.0    95.5   39.0    75.4
## 13   C15   4     1  Vic  f   6   93.2   57.1    92.0   38.0    76.1
## 14   C27   8     1  Vic  f   6   94.8   57.6    91.0   37.0    72.7
## 15   C31  10     1  Vic  f   6   91.8   58.0    89.5   37.5    70.9
## 16   WW6  52     3 other m   6   97.6   61.0    93.5   40.0    67.9
## 17   CD3  63     5 other f   6   92.0   56.4    88.5   38.0    64.1
## 18   CD11  71     5 other m   6   92.8   57.6    90.0   40.0    65.7
## 19   BSF2  75     6 other m   6   93.8   58.1    89.0   38.0    66.2
## 20   BSF4  77     6 other m   6   93.6   56.2    84.0   36.0    62.8
## 21   BTP4  89     7 other m   6   97.7   58.4    84.5   35.0    64.4
## 22  BTP19 102     7 other f   6   92.4   55.0    89.0   38.0    63.5
## 23   C34  12     1  Vic  f   5   94.9   55.6    92.0   35.5    71.7
## 24   C36  13     1  Vic  m   5   95.1   59.9    89.5   36.0    71.0
## 25   C39  15     1  Vic  m   5   92.9   57.6    85.5   34.0    69.7
## 26   C48  19     1  Vic  f   5   94.4   55.4    90.5   35.0    73.4
## 27  BB36  41     2  Vic  f   5   88.4   57.0    83.0   36.5      NA
## 28   WW2  48     3 other m   5   98.6   63.2    85.0   34.0    66.9
## 29   WW4  50     3 other f   5   91.6   56.4    88.0   38.0    65.0
```

## 30	WW5	51	3	other	f	5	95.6	59.6	85.0	36.0	64.0
## 31	CD5	65	5	other	f	5	93.5	57.4	88.5	38.0	68.2
## 32	CD8	68	5	other	m	5	94.1	56.0	88.5	38.0	65.9
## 33	CD9	69	5	other	m	5	98.0	55.6	88.0	37.5	65.0
## 34	BSF3	76	6	other	m	5	92.4	56.8	89.0	41.0	64.5
## 35	BTP14	98	7	other	m	5	93.2	68.6	84.0	35.0	65.6
## 36	C40	16	1	Vic	m	4	91.6	56.0	86.0	34.5	73.0
## 37	C50	20	1	Vic	f	4	94.8	56.3	89.0	38.0	73.8
## 38	C58	23	1	Vic	f	4	92.5	56.1	89.0	36.0	72.8
## 39	C64	28	1	Vic	m	4	93.8	56.8	87.0	34.5	73.2
## 40	A4	32	1	Vic	f	4	94.3	56.7	94.0	39.0	74.8
## 41	BB13	35	2	Vic	m	4	94.4	57.9	85.0	35.5	71.2
## 42	WW3	49	3	other	m	4	95.4	59.2	85.0	37.0	69.0
## 43	BR4	57	4	other	f	4	95.1	59.4	93.0	41.0	67.2
## 44	CD7	67	5	other	m	4	93.3	57.6	85.0	36.5	64.7
## 45	BSF1	74	6	other	f	4	88.7	52.0	83.0	38.0	61.5
## 46	BSF9	82	6	other	f	4	86.0	54.0	82.0	36.5	60.7
## 47	BTP10	95	7	other	m	4	91.6	56.6	88.5	37.5	64.5
## 48	BTP12	96	7	other	m	4	94.8	55.7	83.0	38.0	66.5
## 49	BTP20	103	7	other	m	4	91.5	55.2	82.5	36.5	62.9
## 50	C37	14	1	Vic	m	3	95.4	57.6	91.5	36.0	74.3
## 51	C54	21	1	Vic	f	3	95.9	58.1	96.5	39.5	77.9
## 52	C55	22	1	Vic	m	3	96.3	58.5	91.0	39.5	73.5
## 53	C60	25	1	Vic	m	3	95.8	58.5	91.5	35.5	72.3
## 54	A1	29	1	Vic	f	3	92.8	56.0	88.0	35.0	74.9
## 55	A3	31	1	Vic	m	3	92.8	54.1	93.0	37.0	68.0
## 56	AD1	33	1	Vic	m	3	91.4	54.6	89.0	37.0	70.8
## 57	BB33	40	2	Vic	f	3	91.0	55.0	84.5	36.0	72.8
## 58	BB38	42	2	Vic	m	3	85.3	54.1	77.0	32.0	62.7
## 59	BB44	45	2	Vic	m	3	90.7	55.9	81.0	34.0	71.5
## 60	WW7	53	3	other	f	3	93.1	58.1	91.0	38.0	67.4
## 61	BR3	56	4	other	m	3	99.9	61.5	93.7	38.0	68.7
## 62	BR5	58	4	other	m	3	94.5	64.2	91.0	39.0	66.5
## 63	CD2	62	5	other	f	3	91.3	58.0	90.5	39.0	65.5
## 64	CD4	64	5	other	f	3	96.9	56.5	89.5	38.5	63.0
## 65	CD6	66	5	other	f	3	90.4	55.8	86.0	36.5	63.2
## 66	BSF8	81	6	other	m	3	90.6	56.0	85.5	38.0	65.6
## 67	BSF10	83	6	other	f	3	90.0	53.8	81.5	36.0	62.0
## 68	BSF11	84	6	other	m	3	88.4	54.6	80.5	36.0	62.6
## 69	BSF12	85	6	other	m	3	89.5	56.2	92.0	40.5	65.6
## 70	BSF13	86	6	other	f	3	88.2	53.2	86.5	38.5	60.3
## 71	BTP5	90	7	other	m	3	92.6	54.6	85.0	38.5	69.8
## 72	BTP6	91	7	other	m	3	97.8	59.6	89.0	38.0	65.5
## 73	BTP8	93	7	other	m	3	89.2	54.0	82.0	38.0	63.8
## 74	BTP13	97	7	other	m	3	91.0	53.1	86.0	38.0	63.8
## 75	BTP15	99	7	other	f	3	93.3	56.2	86.5	38.5	64.8
## 76	BTP21	104	7	other	f	3	93.6	59.9	89.0	40.0	67.6
## 77	C23	5	1	Vic	f	2	91.5	56.3	85.5	36.0	71.0
## 78	C26	7	1	Vic	m	2	95.3	58.2	89.5	36.0	71.5
## 79	C47	18	1	Vic	m	2	93.5	55.7	90.0	36.0	73.7
## 80	C59	24	1	Vic	m	2	94.4	54.9	84.0	34.0	75.0
## 81	C63	27	1	Vic	f	2	90.5	54.5	85.0	35.0	70.3
## 82	A2	30	1	Vic	f	2	92.1	54.4	84.0	33.5	70.6
## 83	BB4	34	2	Vic	m	2	90.6	55.7	85.5	36.5	73.1

## 84	BB17	37	2	Vic	f	2	89.3	54.8	82.5	35.0	71.2
## 85	BB40	43	2	Vic	f	2	90.0	55.5	81.0	32.0	72.0
## 86	WW1	47	3	other	m	2	90.1	54.8	89.0	37.5	66.0
## 87	BR2	55	4	other	m	2	103.1	63.2	92.5	38.0	72.5
## 88	BR6	59	4	other	m	2	102.5	62.8	96.0	40.0	73.2
## 89	BR7	60	4	other	f	2	91.3	57.7	88.0	39.0	63.1
## 90	BTP1	87	7	other	m	2	98.5	60.7	93.0	41.5	71.7
## 91	BTP3	88	7	other	f	2	89.6	58.0	87.5	38.0	66.7
## 92	BTP7	92	7	other	m	2	90.7	56.3	85.0	37.0	67.6
## 93	C24	6	1	Vic	f	1	93.1	54.8	90.5	35.5	73.2
## 94	C45	17	1	Vic	f	1	94.7	67.7	89.5	36.5	73.2
## 95	BB31	39	2	Vic	f	1	84.7	51.5	75.0	34.0	68.7
## 96	CD12	72	5	other	m	1	85.9	52.4	80.5	35.0	62.0
## 97	CD13	73	5	other	m	1	82.5	52.3	82.0	36.5	65.7
## 98	BSF5	78	6	other	m	1	86.5	51.0	81.0	36.5	63.0
## 99	BSF6	79	6	other	m	1	85.8	50.0	81.0	36.5	62.8
## 100	BSF7	80	6	other	m	1	86.7	52.6	84.0	38.0	62.3
## 101	BTP16	100	7	other	m	1	89.5	56.0	81.5	36.5	66.0
## 102	BTP17	101	7	other	m	1	88.6	54.7	82.5	39.0	64.4
## 103	BB41	44	2	Vic	m	NA	85.1	51.5	76.0	35.5	70.3
## 104	BB45	46	2	Vic	m	NA	91.4	54.4	84.0	35.0	72.8
##	earconch	eye	chest	belly							
## 1		52.9	15.5	28.0	33.0						
## 2		51.3	14.9	31.0	34.0						
## 3		54.5	15.2	28.0	36.0						
## 4		56.2	15.0	29.0	38.0						
## 5		52.0	14.9	25.5	36.0						
## 6		49.5	15.9	27.0	30.0						
## 7		46.0	17.5	30.0	36.5						
## 8		44.9	15.0	26.5	31.0						
## 9		44.1	13.0	27.0	34.0						
## 10		45.1	14.4	29.0	35.0						
## 11		51.2	16.0	28.5	33.0						
## 12		51.9	15.5	30.0	34.0						
## 13		52.2	15.2	28.0	34.0						
## 14		53.9	14.5	29.0	34.0						
## 15		53.4	14.4	27.5	32.0						
## 16		44.3	15.8	28.5	32.5						
## 17		46.3	15.2	25.5	28.5						
## 18		42.8	15.0	27.5	34.0						
## 19		45.6	16.9	26.0	33.5						
## 20		42.9	16.2	25.0	35.0						
## 21		46.2	14.4	29.0	30.5						
## 22		45.4	13.0	25.0	30.0						
## 23		51.0	15.3	28.0	33.0						
## 24		49.8	15.8	27.0	32.0						
## 25		51.8	15.7	28.0	35.0						
## 26		53.9	15.2	28.0	32.0						
## 27		40.3	15.9	27.0	30.5						
## 28		44.9	17.0	28.0	35.0						
## 29		47.2	14.9	28.0	36.0						
## 30		43.9	17.4	28.0	38.5						
## 31		41.7	14.0	29.0	38.5						
## 32		43.1	17.4	27.0	30.0						

## 33	45.6	15.0	28.5	34.0
## 34	46.4	17.8	26.0	33.0
## 35	44.3	14.5	28.5	32.0
## 36	51.4	14.4	28.0	32.0
## 37	52.4	15.5	27.0	36.0
## 38	53.3	15.4	28.0	35.0
## 39	53.0	15.3	27.0	30.0
## 40	52.0	14.9	28.0	34.0
## 41	55.5	16.4	28.0	35.5
## 42	45.0	15.9	29.5	35.5
## 43	45.3	14.5	31.0	39.0
## 44	44.1	16.5	27.5	29.5
## 45	45.9	14.7	26.0	34.0
## 46	42.9	15.4	26.0	32.0
## 47	45.4	14.9	27.0	31.0
## 48	47.7	14.0	25.0	33.0
## 49	45.9	15.4	25.0	29.0
## 50	53.7	15.1	28.0	31.5
## 51	52.9	14.2	30.0	40.0
## 52	52.1	16.2	28.0	36.0
## 53	51.6	14.9	31.0	35.0
## 54	51.8	14.0	24.0	32.0
## 55	52.5	14.5	27.0	31.0
## 56	51.8	14.8	24.0	30.0
## 57	51.4	13.6	27.0	30.0
## 58	51.2	13.8	25.5	33.0
## 59	54.0	14.6	27.0	31.5
## 60	46.0	16.5	26.0	33.5
## 61	46.8	16.4	27.5	31.5
## 62	46.4	14.4	30.5	33.0
## 63	41.3	16.0	27.0	32.0
## 64	45.1	17.1	25.5	33.0
## 65	44.2	15.7	26.5	34.0
## 66	41.7	17.0	27.5	35.0
## 67	43.3	14.0	25.0	29.0
## 68	43.6	16.3	25.0	28.5
## 69	43.5	14.5	27.0	31.5
## 70	43.7	13.6	26.0	31.0
## 71	44.8	14.5	25.5	32.5
## 72	48.0	15.0	26.0	32.0
## 73	44.9	12.8	24.0	31.0
## 74	46.0	14.5	25.0	31.5
## 75	43.8	14.0	28.0	35.0
## 76	46.0	14.8	28.5	33.5
## 77	53.2	15.1	28.5	33.0
## 78	52.0	14.2	30.0	34.5
## 79	55.4	15.3	28.0	32.0
## 80	53.5	16.2	27.0	32.0
## 81	50.8	14.2	23.0	28.0
## 82	50.8	14.5	24.5	33.0
## 83	53.1	14.4	26.0	28.5
## 84	52.0	13.6	28.0	31.5
## 85	49.4	13.4	29.0	31.0
## 86	45.5	15.0	25.0	33.0

```
## 87      44.9 16.4 30.5 36.0
## 88      44.5 14.7 32.0 36.0
## 89      47.0 14.4 26.0 30.0
## 90      46.8 15.0 26.0 36.0
## 91      43.5 16.0 25.5 31.5
## 92      46.8 14.5 25.5 31.0
## 93      53.6 14.2 30.0 32.0
## 94      53.2 14.7 29.0 31.0
## 95      53.4 13.0 25.0 25.0
## 96      42.4 14.1 25.5 30.0
## 97      44.7 16.0 23.5 28.0
## 98      44.3 13.2 23.0 28.0
## 99      43.0 14.8 22.0 28.5
## 100     44.8 15.0 23.5 30.5
## 101     46.8 14.8 23.0 27.0
## 102     48.0 14.0 25.0 33.0
## 103     52.6 14.4 23.0 27.0
## 104     51.2 14.4 24.5 35.0
```

What happens if we `arrange()` something that is not a number?

```
arrange(possum, X1)
```

```
##      X1 case site  Pop sex age hdlngth skullw totlngth taill footlngth
## 1     A1  29    1  Vic  f   3   92.8   56.0    88.0   35.0    74.9
## 2     A2  30    1  Vic  f   2   92.1   54.4    84.0   33.5    70.6
## 3     A3  31    1  Vic  m   3   92.8   54.1    93.0   37.0    68.0
## 4     A4  32    1  Vic  f   4   94.3   56.7    94.0   39.0    74.8
## 5    AD1  33    1  Vic  m   3   91.4   54.6    89.0   37.0    70.8
## 6    BB13 35    2  Vic  m   4   94.4   57.9    85.0   35.5    71.2
## 7    BB15 36    2  Vic  m   7   93.3   59.3    88.0   35.0    74.3
## 8    BB17 37    2  Vic  f   2   89.3   54.8    82.5   35.0    71.2
## 9    BB25 38    2  Vic  m   7   92.4   56.0    80.5   35.5    68.4
## 10   BB31 39    2  Vic  f   1   84.7   51.5    75.0   34.0    68.7
## 11   BB33 40    2  Vic  f   3   91.0   55.0    84.5   36.0    72.8
## 12   BB36 41    2  Vic  f   5   88.4   57.0    83.0   36.5     NA
## 13   BB38 42    2  Vic  m   3   85.3   54.1    77.0   32.0    62.7
## 14    BB4 34    2  Vic  m   2   90.6   55.7    85.5   36.5    73.1
## 15   BB40 43    2  Vic  f   2   90.0   55.5    81.0   32.0    72.0
## 16   BB41 44    2  Vic  m  NA   85.1   51.5    76.0   35.5    70.3
## 17   BB44 45    2  Vic  m   3   90.7   55.9    81.0   34.0    71.5
## 18   BB45 46    2  Vic  m  NA   91.4   54.4    84.0   35.0    72.8
## 19    BR1 54    4 other m   7   96.9   63.0    91.5   43.0    71.3
## 20    BR2 55    4 other m   2  103.1   63.2    92.5   38.0    72.5
## 21    BR3 56    4 other m   3   99.9   61.5    93.7   38.0    68.7
## 22    BR4 57    4 other f   4   95.1   59.4    93.0   41.0    67.2
## 23    BR5 58    4 other m   3   94.5   64.2    91.0   39.0    66.5
## 24    BR6 59    4 other m   2  102.5   62.8    96.0   40.0    73.2
## 25    BR7 60    4 other f   2   91.3   57.7    88.0   39.0    63.1
## 26   BSF1 74    6 other f   4   88.7   52.0    83.0   38.0    61.5
## 27  BSF10 83    6 other f   3   90.0   53.8    81.5   36.0    62.0
## 28  BSF11 84    6 other m   3   88.4   54.6    80.5   36.0    62.6
## 29  BSF12 85    6 other m   3   89.5   56.2    92.0   40.5    65.6
## 30  BSF13 86    6 other f   3   88.2   53.2    86.5   38.5    60.3
## 31   BSF2 75    6 other m   6   93.8   58.1    89.0   38.0    66.2
```

## 32	BSF3	76	6 other	m	5	92.4	56.8	89.0	41.0	64.5
## 33	BSF4	77	6 other	m	6	93.6	56.2	84.0	36.0	62.8
## 34	BSF5	78	6 other	m	1	86.5	51.0	81.0	36.5	63.0
## 35	BSF6	79	6 other	m	1	85.8	50.0	81.0	36.5	62.8
## 36	BSF7	80	6 other	m	1	86.7	52.6	84.0	38.0	62.3
## 37	BSF8	81	6 other	m	3	90.6	56.0	85.5	38.0	65.6
## 38	BSF9	82	6 other	f	4	86.0	54.0	82.0	36.5	60.7
## 39	BTP1	87	7 other	m	2	98.5	60.7	93.0	41.5	71.7
## 40	BTP10	95	7 other	m	4	91.6	56.6	88.5	37.5	64.5
## 41	BTP12	96	7 other	m	4	94.8	55.7	83.0	38.0	66.5
## 42	BTP13	97	7 other	m	3	91.0	53.1	86.0	38.0	63.8
## 43	BTP14	98	7 other	m	5	93.2	68.6	84.0	35.0	65.6
## 44	BTP15	99	7 other	f	3	93.3	56.2	86.5	38.5	64.8
## 45	BTP16	100	7 other	m	1	89.5	56.0	81.5	36.5	66.0
## 46	BTP17	101	7 other	m	1	88.6	54.7	82.5	39.0	64.4
## 47	BTP19	102	7 other	f	6	92.4	55.0	89.0	38.0	63.5
## 48	BTP20	103	7 other	m	4	91.5	55.2	82.5	36.5	62.9
## 49	BTP21	104	7 other	f	3	93.6	59.9	89.0	40.0	67.6
## 50	BTP3	88	7 other	f	2	89.6	58.0	87.5	38.0	66.7
## 51	BTP4	89	7 other	m	6	97.7	58.4	84.5	35.0	64.4
## 52	BTP5	90	7 other	m	3	92.6	54.6	85.0	38.5	69.8
## 53	BTP6	91	7 other	m	3	97.8	59.6	89.0	38.0	65.5
## 54	BTP7	92	7 other	m	2	90.7	56.3	85.0	37.0	67.6
## 55	BTP8	93	7 other	m	3	89.2	54.0	82.0	38.0	63.8
## 56	BTP9	94	7 other	m	7	91.8	57.6	84.0	35.5	64.2
## 57	C10	3	1 Vic	f	6	94.0	60.0	95.5	39.0	75.4
## 58	C15	4	1 Vic	f	6	93.2	57.1	92.0	38.0	76.1
## 59	C23	5	1 Vic	f	2	91.5	56.3	85.5	36.0	71.0
## 60	C24	6	1 Vic	f	1	93.1	54.8	90.5	35.5	73.2
## 61	C26	7	1 Vic	m	2	95.3	58.2	89.5	36.0	71.5
## 62	C27	8	1 Vic	f	6	94.8	57.6	91.0	37.0	72.7
## 63	C28	9	1 Vic	f	9	93.4	56.3	91.5	37.0	72.4
## 64	C3	1	1 Vic	m	8	94.1	60.4	89.0	36.0	74.5
## 65	C31	10	1 Vic	f	6	91.8	58.0	89.5	37.5	70.9
## 66	C32	11	1 Vic	f	9	93.3	57.2	89.5	39.0	77.2
## 67	C34	12	1 Vic	f	5	94.9	55.6	92.0	35.5	71.7
## 68	C36	13	1 Vic	m	5	95.1	59.9	89.5	36.0	71.0
## 69	C37	14	1 Vic	m	3	95.4	57.6	91.5	36.0	74.3
## 70	C39	15	1 Vic	m	5	92.9	57.6	85.5	34.0	69.7
## 71	C40	16	1 Vic	m	4	91.6	56.0	86.0	34.5	73.0
## 72	C45	17	1 Vic	f	1	94.7	67.7	89.5	36.5	73.2
## 73	C47	18	1 Vic	m	2	93.5	55.7	90.0	36.0	73.7
## 74	C48	19	1 Vic	f	5	94.4	55.4	90.5	35.0	73.4
## 75	C5	2	1 Vic	f	6	92.5	57.6	91.5	36.5	72.5
## 76	C50	20	1 Vic	f	4	94.8	56.3	89.0	38.0	73.8
## 77	C54	21	1 Vic	f	3	95.9	58.1	96.5	39.5	77.9
## 78	C55	22	1 Vic	m	3	96.3	58.5	91.0	39.5	73.5
## 79	C58	23	1 Vic	f	4	92.5	56.1	89.0	36.0	72.8
## 80	C59	24	1 Vic	m	2	94.4	54.9	84.0	34.0	75.0
## 81	C60	25	1 Vic	m	3	95.8	58.5	91.5	35.5	72.3
## 82	C61	26	1 Vic	m	7	96.0	59.0	90.0	36.0	73.6
## 83	C63	27	1 Vic	f	2	90.5	54.5	85.0	35.0	70.3
## 84	C64	28	1 Vic	m	4	93.8	56.8	87.0	34.5	73.2
## 85	CD1	61	5 other	m	7	95.7	59.0	86.0	38.0	63.1

## 86	CD10	70	5 other	f	7	91.9	56.4	87.0	38.0	65.4
## 87	CD11	71	5 other	m	6	92.8	57.6	90.0	40.0	65.7
## 88	CD12	72	5 other	m	1	85.9	52.4	80.5	35.0	62.0
## 89	CD13	73	5 other	m	1	82.5	52.3	82.0	36.5	65.7
## 90	CD2	62	5 other	f	3	91.3	58.0	90.5	39.0	65.5
## 91	CD3	63	5 other	f	6	92.0	56.4	88.5	38.0	64.1
## 92	CD4	64	5 other	f	3	96.9	56.5	89.5	38.5	63.0
## 93	CD5	65	5 other	f	5	93.5	57.4	88.5	38.0	68.2
## 94	CD6	66	5 other	f	3	90.4	55.8	86.0	36.5	63.2
## 95	CD7	67	5 other	m	4	93.3	57.6	85.0	36.5	64.7
## 96	CD8	68	5 other	m	5	94.1	56.0	88.5	38.0	65.9
## 97	CD9	69	5 other	m	5	98.0	55.6	88.0	37.5	65.0
## 98	WW1	47	3 other	m	2	90.1	54.8	89.0	37.5	66.0
## 99	WW2	48	3 other	m	5	98.6	63.2	85.0	34.0	66.9
## 100	WW3	49	3 other	m	4	95.4	59.2	85.0	37.0	69.0
## 101	WW4	50	3 other	f	5	91.6	56.4	88.0	38.0	65.0
## 102	WW5	51	3 other	f	5	95.6	59.6	85.0	36.0	64.0
## 103	WW6	52	3 other	m	6	97.6	61.0	93.5	40.0	67.9
## 104	WW7	53	3 other	f	3	93.1	58.1	91.0	38.0	67.4
##	earconch	eye	chest	belly						
## 1		51.8	14.0	24.0	32.0					
## 2		50.8	14.5	24.5	33.0					
## 3		52.5	14.5	27.0	31.0					
## 4		52.0	14.9	28.0	34.0					
## 5		51.8	14.8	24.0	30.0					
## 6		55.5	16.4	28.0	35.5					
## 7		52.0	14.9	25.5	36.0					
## 8		52.0	13.6	28.0	31.5					
## 9		49.5	15.9	27.0	30.0					
## 10		53.4	13.0	25.0	25.0					
## 11		51.4	13.6	27.0	30.0					
## 12		40.3	15.9	27.0	30.5					
## 13		51.2	13.8	25.5	33.0					
## 14		53.1	14.4	26.0	28.5					
## 15		49.4	13.4	29.0	31.0					
## 16		52.6	14.4	23.0	27.0					
## 17		54.0	14.6	27.0	31.5					
## 18		51.2	14.4	24.5	35.0					
## 19		46.0	17.5	30.0	36.5					
## 20		44.9	16.4	30.5	36.0					
## 21		46.8	16.4	27.5	31.5					
## 22		45.3	14.5	31.0	39.0					
## 23		46.4	14.4	30.5	33.0					
## 24		44.5	14.7	32.0	36.0					
## 25		47.0	14.4	26.0	30.0					
## 26		45.9	14.7	26.0	34.0					
## 27		43.3	14.0	25.0	29.0					
## 28		43.6	16.3	25.0	28.5					
## 29		43.5	14.5	27.0	31.5					
## 30		43.7	13.6	26.0	31.0					
## 31		45.6	16.9	26.0	33.5					
## 32		46.4	17.8	26.0	33.0					
## 33		42.9	16.2	25.0	35.0					
## 34		44.3	13.2	23.0	28.0					

## 35	43.0	14.8	22.0	28.5
## 36	44.8	15.0	23.5	30.5
## 37	41.7	17.0	27.5	35.0
## 38	42.9	15.4	26.0	32.0
## 39	46.8	15.0	26.0	36.0
## 40	45.4	14.9	27.0	31.0
## 41	47.7	14.0	25.0	33.0
## 42	46.0	14.5	25.0	31.5
## 43	44.3	14.5	28.5	32.0
## 44	43.8	14.0	28.0	35.0
## 45	46.8	14.8	23.0	27.0
## 46	48.0	14.0	25.0	33.0
## 47	45.4	13.0	25.0	30.0
## 48	45.9	15.4	25.0	29.0
## 49	46.0	14.8	28.5	33.5
## 50	43.5	16.0	25.5	31.5
## 51	46.2	14.4	29.0	30.5
## 52	44.8	14.5	25.5	32.5
## 53	48.0	15.0	26.0	32.0
## 54	46.8	14.5	25.5	31.0
## 55	44.9	12.8	24.0	31.0
## 56	45.1	14.4	29.0	35.0
## 57	51.9	15.5	30.0	34.0
## 58	52.2	15.2	28.0	34.0
## 59	53.2	15.1	28.5	33.0
## 60	53.6	14.2	30.0	32.0
## 61	52.0	14.2	30.0	34.5
## 62	53.9	14.5	29.0	34.0
## 63	52.9	15.5	28.0	33.0
## 64	54.5	15.2	28.0	36.0
## 65	53.4	14.4	27.5	32.0
## 66	51.3	14.9	31.0	34.0
## 67	51.0	15.3	28.0	33.0
## 68	49.8	15.8	27.0	32.0
## 69	53.7	15.1	28.0	31.5
## 70	51.8	15.7	28.0	35.0
## 71	51.4	14.4	28.0	32.0
## 72	53.2	14.7	29.0	31.0
## 73	55.4	15.3	28.0	32.0
## 74	53.9	15.2	28.0	32.0
## 75	51.2	16.0	28.5	33.0
## 76	52.4	15.5	27.0	36.0
## 77	52.9	14.2	30.0	40.0
## 78	52.1	16.2	28.0	36.0
## 79	53.3	15.4	28.0	35.0
## 80	53.5	16.2	27.0	32.0
## 81	51.6	14.9	31.0	35.0
## 82	56.2	15.0	29.0	38.0
## 83	50.8	14.2	23.0	28.0
## 84	53.0	15.3	27.0	30.0
## 85	44.9	15.0	26.5	31.0
## 86	44.1	13.0	27.0	34.0
## 87	42.8	15.0	27.5	34.0
## 88	42.4	14.1	25.5	30.0

```
## 89      44.7 16.0 23.5 28.0
## 90      41.3 16.0 27.0 32.0
## 91      46.3 15.2 25.5 28.5
## 92      45.1 17.1 25.5 33.0
## 93      41.7 14.0 29.0 38.5
## 94      44.2 15.7 26.5 34.0
## 95      44.1 16.5 27.5 29.5
## 96      43.1 17.4 27.0 30.0
## 97      45.6 15.0 28.5 34.0
## 98      45.5 15.0 25.0 33.0
## 99      44.9 17.0 28.0 35.0
## 100     45.0 15.9 29.5 35.5
## 101     47.2 14.9 28.0 36.0
## 102     43.9 17.4 28.0 38.5
## 103     44.3 15.8 28.5 32.5
## 104     46.0 16.5 26.0 33.5
```

`arrange()` can alphabetize for us too. This is useful in combination with `head()` when surveying our data.

Next up, `filter()` and `select()`. Filtering means choosing *rows* while selecting means choosing *columns*. You can filter rows based on what the rows contain, but you can only select columns by name or position.

```
filter(possum, sex == "m")
```

```
##      X1 case site  Pop sex age hdlngth skullw totlngth taill footlgth
## 1    C3      1   1   Vic  m   8   94.1   60.4    89.0  36.0    74.5
## 2   C26      7   1   Vic  m   2   95.3   58.2    89.5  36.0    71.5
## 3   C36     13   1   Vic  m   5   95.1   59.9    89.5  36.0    71.0
## 4   C37     14   1   Vic  m   3   95.4   57.6    91.5  36.0    74.3
## 5   C39     15   1   Vic  m   5   92.9   57.6    85.5  34.0    69.7
## 6   C40     16   1   Vic  m   4   91.6   56.0    86.0  34.5    73.0
## 7   C47     18   1   Vic  m   2   93.5   55.7    90.0  36.0    73.7
## 8   C55     22   1   Vic  m   3   96.3   58.5    91.0  39.5    73.5
## 9   C59     24   1   Vic  m   2   94.4   54.9    84.0  34.0    75.0
## 10  C60     25   1   Vic  m   3   95.8   58.5    91.5  35.5    72.3
## 11  C61     26   1   Vic  m   7   96.0   59.0    90.0  36.0    73.6
## 12  C64     28   1   Vic  m   4   93.8   56.8    87.0  34.5    73.2
## 13   A3     31   1   Vic  m   3   92.8   54.1    93.0  37.0    68.0
## 14  AD1     33   1   Vic  m   3   91.4   54.6    89.0  37.0    70.8
## 15  BB4     34   2   Vic  m   2   90.6   55.7    85.5  36.5    73.1
## 16 BB13     35   2   Vic  m   4   94.4   57.9    85.0  35.5    71.2
## 17 BB15     36   2   Vic  m   7   93.3   59.3    88.0  35.0    74.3
## 18 BB25     38   2   Vic  m   7   92.4   56.0    80.5  35.5    68.4
## 19 BB38     42   2   Vic  m   3   85.3   54.1    77.0  32.0    62.7
## 20 BB41     44   2   Vic  m  NA   85.1   51.5    76.0  35.5    70.3
## 21 BB44     45   2   Vic  m   3   90.7   55.9    81.0  34.0    71.5
## 22 BB45     46   2   Vic  m  NA   91.4   54.4    84.0  35.0    72.8
## 23  WW1     47   3 other  m   2   90.1   54.8    89.0  37.5    66.0
## 24  WW2     48   3 other  m   5   98.6   63.2    85.0  34.0    66.9
## 25  WW3     49   3 other  m   4   95.4   59.2    85.0  37.0    69.0
## 26  WW6     52   3 other  m   6   97.6   61.0    93.5  40.0    67.9
## 27  BR1     54   4 other  m   7   96.9   63.0    91.5  43.0    71.3
## 28  BR2     55   4 other  m   2  103.1   63.2    92.5  38.0    72.5
```

## 29	BR3	56	4 other	m	3	99.9	61.5	93.7	38.0	68.7
## 30	BR5	58	4 other	m	3	94.5	64.2	91.0	39.0	66.5
## 31	BR6	59	4 other	m	2	102.5	62.8	96.0	40.0	73.2
## 32	CD1	61	5 other	m	7	95.7	59.0	86.0	38.0	63.1
## 33	CD7	67	5 other	m	4	93.3	57.6	85.0	36.5	64.7
## 34	CD8	68	5 other	m	5	94.1	56.0	88.5	38.0	65.9
## 35	CD9	69	5 other	m	5	98.0	55.6	88.0	37.5	65.0
## 36	CD11	71	5 other	m	6	92.8	57.6	90.0	40.0	65.7
## 37	CD12	72	5 other	m	1	85.9	52.4	80.5	35.0	62.0
## 38	CD13	73	5 other	m	1	82.5	52.3	82.0	36.5	65.7
## 39	BSF2	75	6 other	m	6	93.8	58.1	89.0	38.0	66.2
## 40	BSF3	76	6 other	m	5	92.4	56.8	89.0	41.0	64.5
## 41	BSF4	77	6 other	m	6	93.6	56.2	84.0	36.0	62.8
## 42	BSF5	78	6 other	m	1	86.5	51.0	81.0	36.5	63.0
## 43	BSF6	79	6 other	m	1	85.8	50.0	81.0	36.5	62.8
## 44	BSF7	80	6 other	m	1	86.7	52.6	84.0	38.0	62.3
## 45	BSF8	81	6 other	m	3	90.6	56.0	85.5	38.0	65.6
## 46	BSF11	84	6 other	m	3	88.4	54.6	80.5	36.0	62.6
## 47	BSF12	85	6 other	m	3	89.5	56.2	92.0	40.5	65.6
## 48	BTP1	87	7 other	m	2	98.5	60.7	93.0	41.5	71.7
## 49	BTP4	89	7 other	m	6	97.7	58.4	84.5	35.0	64.4
## 50	BTP5	90	7 other	m	3	92.6	54.6	85.0	38.5	69.8
## 51	BTP6	91	7 other	m	3	97.8	59.6	89.0	38.0	65.5
## 52	BTP7	92	7 other	m	2	90.7	56.3	85.0	37.0	67.6
## 53	BTP8	93	7 other	m	3	89.2	54.0	82.0	38.0	63.8
## 54	BTP9	94	7 other	m	7	91.8	57.6	84.0	35.5	64.2
## 55	BTP10	95	7 other	m	4	91.6	56.6	88.5	37.5	64.5
## 56	BTP12	96	7 other	m	4	94.8	55.7	83.0	38.0	66.5
## 57	BTP13	97	7 other	m	3	91.0	53.1	86.0	38.0	63.8
## 58	BTP14	98	7 other	m	5	93.2	68.6	84.0	35.0	65.6
## 59	BTP16	100	7 other	m	1	89.5	56.0	81.5	36.5	66.0
## 60	BTP17	101	7 other	m	1	88.6	54.7	82.5	39.0	64.4
## 61	BTP20	103	7 other	m	4	91.5	55.2	82.5	36.5	62.9
##	earconch	eye	chest	belly						
## 1		54.5	15.2	28.0	36.0					
## 2		52.0	14.2	30.0	34.5					
## 3		49.8	15.8	27.0	32.0					
## 4		53.7	15.1	28.0	31.5					
## 5		51.8	15.7	28.0	35.0					
## 6		51.4	14.4	28.0	32.0					
## 7		55.4	15.3	28.0	32.0					
## 8		52.1	16.2	28.0	36.0					
## 9		53.5	16.2	27.0	32.0					
## 10		51.6	14.9	31.0	35.0					
## 11		56.2	15.0	29.0	38.0					
## 12		53.0	15.3	27.0	30.0					
## 13		52.5	14.5	27.0	31.0					
## 14		51.8	14.8	24.0	30.0					
## 15		53.1	14.4	26.0	28.5					
## 16		55.5	16.4	28.0	35.5					
## 17		52.0	14.9	25.5	36.0					
## 18		49.5	15.9	27.0	30.0					
## 19		51.2	13.8	25.5	33.0					
## 20		52.6	14.4	23.0	27.0					

```
## 21      54.0 14.6 27.0 31.5
## 22      51.2 14.4 24.5 35.0
## 23      45.5 15.0 25.0 33.0
## 24      44.9 17.0 28.0 35.0
## 25      45.0 15.9 29.5 35.5
## 26      44.3 15.8 28.5 32.5
## 27      46.0 17.5 30.0 36.5
## 28      44.9 16.4 30.5 36.0
## 29      46.8 16.4 27.5 31.5
## 30      46.4 14.4 30.5 33.0
## 31      44.5 14.7 32.0 36.0
## 32      44.9 15.0 26.5 31.0
## 33      44.1 16.5 27.5 29.5
## 34      43.1 17.4 27.0 30.0
## 35      45.6 15.0 28.5 34.0
## 36      42.8 15.0 27.5 34.0
## 37      42.4 14.1 25.5 30.0
## 38      44.7 16.0 23.5 28.0
## 39      45.6 16.9 26.0 33.5
## 40      46.4 17.8 26.0 33.0
## 41      42.9 16.2 25.0 35.0
## 42      44.3 13.2 23.0 28.0
## 43      43.0 14.8 22.0 28.5
## 44      44.8 15.0 23.5 30.5
## 45      41.7 17.0 27.5 35.0
## 46      43.6 16.3 25.0 28.5
## 47      43.5 14.5 27.0 31.5
## 48      46.8 15.0 26.0 36.0
## 49      46.2 14.4 29.0 30.5
## 50      44.8 14.5 25.5 32.5
## 51      48.0 15.0 26.0 32.0
## 52      46.8 14.5 25.5 31.0
## 53      44.9 12.8 24.0 31.0
## 54      45.1 14.4 29.0 35.0
## 55      45.4 14.9 27.0 31.0
## 56      47.7 14.0 25.0 33.0
## 57      46.0 14.5 25.0 31.5
## 58      44.3 14.5 28.5 32.0
## 59      46.8 14.8 23.0 27.0
## 60      48.0 14.0 25.0 33.0
## 61      45.9 15.4 25.0 29.0
```

```
filter(possum, taill>38)
```

```
##      X1 case site  Pop sex age hdlngth skullw totlngth taill footlngth
## 1    C10   3    1   Vic  f   6    94.0   60.0     95.5  39.0     75.4
## 2    C32  11    1   Vic  f   9    93.3   57.2     89.5  39.0     77.2
## 3    C54  21    1   Vic  f   3    95.9   58.1     96.5  39.5     77.9
## 4    C55  22    1   Vic  m   3    96.3   58.5     91.0  39.5     73.5
## 5     A4  32    1   Vic  f   4    94.3   56.7     94.0  39.0     74.8
## 6    WW6  52    3 other m   6    97.6   61.0     93.5  40.0     67.9
## 7    BR1  54    4 other m   7    96.9   63.0     91.5  43.0     71.3
## 8    BR4  57    4 other f   4    95.1   59.4     93.0  41.0     67.2
## 9    BR5  58    4 other m   3    94.5   64.2     91.0  39.0     66.5
```

## 10	BR6	59	4 other	m	2	102.5	62.8	96.0	40.0	73.2
## 11	BR7	60	4 other	f	2	91.3	57.7	88.0	39.0	63.1
## 12	CD2	62	5 other	f	3	91.3	58.0	90.5	39.0	65.5
## 13	CD4	64	5 other	f	3	96.9	56.5	89.5	38.5	63.0
## 14	CD11	71	5 other	m	6	92.8	57.6	90.0	40.0	65.7
## 15	BSF3	76	6 other	m	5	92.4	56.8	89.0	41.0	64.5
## 16	BSF12	85	6 other	m	3	89.5	56.2	92.0	40.5	65.6
## 17	BSF13	86	6 other	f	3	88.2	53.2	86.5	38.5	60.3
## 18	BTP1	87	7 other	m	2	98.5	60.7	93.0	41.5	71.7
## 19	BTP5	90	7 other	m	3	92.6	54.6	85.0	38.5	69.8
## 20	BTP15	99	7 other	f	3	93.3	56.2	86.5	38.5	64.8
## 21	BTP17	101	7 other	m	1	88.6	54.7	82.5	39.0	64.4
## 22	BTP21	104	7 other	f	3	93.6	59.9	89.0	40.0	67.6
##	earconch	eye	chest	belly						
## 1		51.9	15.5	30.0	34.0					
## 2		51.3	14.9	31.0	34.0					
## 3		52.9	14.2	30.0	40.0					
## 4		52.1	16.2	28.0	36.0					
## 5		52.0	14.9	28.0	34.0					
## 6		44.3	15.8	28.5	32.5					
## 7		46.0	17.5	30.0	36.5					
## 8		45.3	14.5	31.0	39.0					
## 9		46.4	14.4	30.5	33.0					
## 10		44.5	14.7	32.0	36.0					
## 11		47.0	14.4	26.0	30.0					
## 12		41.3	16.0	27.0	32.0					
## 13		45.1	17.1	25.5	33.0					
## 14		42.8	15.0	27.5	34.0					
## 15		46.4	17.8	26.0	33.0					
## 16		43.5	14.5	27.0	31.5					
## 17		43.7	13.6	26.0	31.0					
## 18		46.8	15.0	26.0	36.0					
## 19		44.8	14.5	25.5	32.5					
## 20		43.8	14.0	28.0	35.0					
## 21		48.0	14.0	25.0	33.0					
## 22		46.0	14.8	28.5	33.5					

```
select(possum, case, site, age)
```

##	case	site	age
## 1	1	1	8
## 2	2	1	6
## 3	3	1	6
## 4	4	1	6
## 5	5	1	2
## 6	6	1	1
## 7	7	1	2
## 8	8	1	6
## 9	9	1	9
## 10	10	1	6
## 11	11	1	9
## 12	12	1	5
## 13	13	1	5
## 14	14	1	3

## 15	15	1	5
## 16	16	1	4
## 17	17	1	1
## 18	18	1	2
## 19	19	1	5
## 20	20	1	4
## 21	21	1	3
## 22	22	1	3
## 23	23	1	4
## 24	24	1	2
## 25	25	1	3
## 26	26	1	7
## 27	27	1	2
## 28	28	1	4
## 29	29	1	3
## 30	30	1	2
## 31	31	1	3
## 32	32	1	4
## 33	33	1	3
## 34	34	2	2
## 35	35	2	4
## 36	36	2	7
## 37	37	2	2
## 38	38	2	7
## 39	39	2	1
## 40	40	2	3
## 41	41	2	5
## 42	42	2	3
## 43	43	2	2
## 44	44	2	NA
## 45	45	2	3
## 46	46	2	NA
## 47	47	3	2
## 48	48	3	5
## 49	49	3	4
## 50	50	3	5
## 51	51	3	5
## 52	52	3	6
## 53	53	3	3
## 54	54	4	7
## 55	55	4	2
## 56	56	4	3
## 57	57	4	4
## 58	58	4	3
## 59	59	4	2
## 60	60	4	2
## 61	61	5	7
## 62	62	5	3
## 63	63	5	6
## 64	64	5	3
## 65	65	5	5
## 66	66	5	3
## 67	67	5	4
## 68	68	5	5

```
## 69    69    5    5
## 70    70    5    7
## 71    71    5    6
## 72    72    5    1
## 73    73    5    1
## 74    74    6    4
## 75    75    6    6
## 76    76    6    5
## 77    77    6    6
## 78    78    6    1
## 79    79    6    1
## 80    80    6    1
## 81    81    6    3
## 82    82    6    4
## 83    83    6    3
## 84    84    6    3
## 85    85    6    3
## 86    86    6    3
## 87    87    7    2
## 88    88    7    2
## 89    89    7    6
## 90    90    7    3
## 91    91    7    3
## 92    92    7    2
## 93    93    7    3
## 94    94    7    7
## 95    95    7    4
## 96    96    7    4
## 97    97    7    3
## 98    98    7    5
## 99    99    7    3
## 100   100    7    1
## 101   101    7    1
## 102   102    7    6
## 103   103    7    4
## 104   104    7    3
```

```
select(possum, 1:5)
```

```
##      X1 case site  Pop sex
## 1     C3    1    1  Vic   m
## 2     C5    2    1  Vic   f
## 3    C10    3    1  Vic   f
## 4    C15    4    1  Vic   f
## 5    C23    5    1  Vic   f
## 6    C24    6    1  Vic   f
## 7    C26    7    1  Vic   m
## 8    C27    8    1  Vic   f
## 9    C28    9    1  Vic   f
## 10   C31   10    1  Vic   f
## 11   C32   11    1  Vic   f
## 12   C34   12    1  Vic   f
## 13   C36   13    1  Vic   m
## 14   C37   14    1  Vic   m
```

## 15	C39	15	1	Vic	m
## 16	C40	16	1	Vic	m
## 17	C45	17	1	Vic	f
## 18	C47	18	1	Vic	m
## 19	C48	19	1	Vic	f
## 20	C50	20	1	Vic	f
## 21	C54	21	1	Vic	f
## 22	C55	22	1	Vic	m
## 23	C58	23	1	Vic	f
## 24	C59	24	1	Vic	m
## 25	C60	25	1	Vic	m
## 26	C61	26	1	Vic	m
## 27	C63	27	1	Vic	f
## 28	C64	28	1	Vic	m
## 29	A1	29	1	Vic	f
## 30	A2	30	1	Vic	f
## 31	A3	31	1	Vic	m
## 32	A4	32	1	Vic	f
## 33	AD1	33	1	Vic	m
## 34	BB4	34	2	Vic	m
## 35	BB13	35	2	Vic	m
## 36	BB15	36	2	Vic	m
## 37	BB17	37	2	Vic	f
## 38	BB25	38	2	Vic	m
## 39	BB31	39	2	Vic	f
## 40	BB33	40	2	Vic	f
## 41	BB36	41	2	Vic	f
## 42	BB38	42	2	Vic	m
## 43	BB40	43	2	Vic	f
## 44	BB41	44	2	Vic	m
## 45	BB44	45	2	Vic	m
## 46	BB45	46	2	Vic	m
## 47	WW1	47	3	other	m
## 48	WW2	48	3	other	m
## 49	WW3	49	3	other	m
## 50	WW4	50	3	other	f
## 51	WW5	51	3	other	f
## 52	WW6	52	3	other	m
## 53	WW7	53	3	other	f
## 54	BR1	54	4	other	m
## 55	BR2	55	4	other	m
## 56	BR3	56	4	other	m
## 57	BR4	57	4	other	f
## 58	BR5	58	4	other	m
## 59	BR6	59	4	other	m
## 60	BR7	60	4	other	f
## 61	CD1	61	5	other	m
## 62	CD2	62	5	other	f
## 63	CD3	63	5	other	f
## 64	CD4	64	5	other	f
## 65	CD5	65	5	other	f
## 66	CD6	66	5	other	f
## 67	CD7	67	5	other	m
## 68	CD8	68	5	other	m

```

## 69    CD9    69    5 other    m
## 70   CD10   70    5 other    f
## 71   CD11   71    5 other    m
## 72   CD12   72    5 other    m
## 73   CD13   73    5 other    m
## 74   BSF1   74    6 other    f
## 75   BSF2   75    6 other    m
## 76   BSF3   76    6 other    m
## 77   BSF4   77    6 other    m
## 78   BSF5   78    6 other    m
## 79   BSF6   79    6 other    m
## 80   BSF7   80    6 other    m
## 81   BSF8   81    6 other    m
## 82   BSF9   82    6 other    f
## 83  BSF10   83    6 other    f
## 84  BSF11   84    6 other    m
## 85  BSF12   85    6 other    m
## 86  BSF13   86    6 other    f
## 87   BTP1   87    7 other    m
## 88   BTP3   88    7 other    f
## 89   BTP4   89    7 other    m
## 90   BTP5   90    7 other    m
## 91   BTP6   91    7 other    m
## 92   BTP7   92    7 other    m
## 93   BTP8   93    7 other    m
## 94   BTP9   94    7 other    m
## 95  BTP10   95    7 other    m
## 96  BTP12   96    7 other    m
## 97  BTP13   97    7 other    m
## 98  BTP14   98    7 other    m
## 99  BTP15   99    7 other    f
## 100 BTP16  100    7 other    m
## 101 BTP17  101    7 other    m
## 102 BTP19  102    7 other    f
## 103 BTP20  103    7 other    m
## 104 BTP21  104    7 other    f

```

```
select(possum, -X1)
```

```

##      case site  Pop sex age hdlngth skullw totlngth taill footlngth earconch
## 1         1    1  Vic  m  8   94.1   60.4      89.0  36.0     74.5    54.5
## 2         2    1  Vic  f  6   92.5   57.6      91.5  36.5     72.5    51.2
## 3         3    1  Vic  f  6   94.0   60.0      95.5  39.0     75.4    51.9
## 4         4    1  Vic  f  6   93.2   57.1      92.0  38.0     76.1    52.2
## 5         5    1  Vic  f  2   91.5   56.3      85.5  36.0     71.0    53.2
## 6         6    1  Vic  f  1   93.1   54.8      90.5  35.5     73.2    53.6
## 7         7    1  Vic  m  2   95.3   58.2      89.5  36.0     71.5    52.0
## 8         8    1  Vic  f  6   94.8   57.6      91.0  37.0     72.7    53.9
## 9         9    1  Vic  f  9   93.4   56.3      91.5  37.0     72.4    52.9
## 10        10    1  Vic  f  6   91.8   58.0      89.5  37.5     70.9    53.4
## 11        11    1  Vic  f  9   93.3   57.2      89.5  39.0     77.2    51.3
## 12        12    1  Vic  f  5   94.9   55.6      92.0  35.5     71.7    51.0
## 13        13    1  Vic  m  5   95.1   59.9      89.5  36.0     71.0    49.8
## 14        14    1  Vic  m  3   95.4   57.6      91.5  36.0     74.3    53.7

```

## 15	15	1	Vic	m	5	92.9	57.6	85.5	34.0	69.7	51.8
## 16	16	1	Vic	m	4	91.6	56.0	86.0	34.5	73.0	51.4
## 17	17	1	Vic	f	1	94.7	67.7	89.5	36.5	73.2	53.2
## 18	18	1	Vic	m	2	93.5	55.7	90.0	36.0	73.7	55.4
## 19	19	1	Vic	f	5	94.4	55.4	90.5	35.0	73.4	53.9
## 20	20	1	Vic	f	4	94.8	56.3	89.0	38.0	73.8	52.4
## 21	21	1	Vic	f	3	95.9	58.1	96.5	39.5	77.9	52.9
## 22	22	1	Vic	m	3	96.3	58.5	91.0	39.5	73.5	52.1
## 23	23	1	Vic	f	4	92.5	56.1	89.0	36.0	72.8	53.3
## 24	24	1	Vic	m	2	94.4	54.9	84.0	34.0	75.0	53.5
## 25	25	1	Vic	m	3	95.8	58.5	91.5	35.5	72.3	51.6
## 26	26	1	Vic	m	7	96.0	59.0	90.0	36.0	73.6	56.2
## 27	27	1	Vic	f	2	90.5	54.5	85.0	35.0	70.3	50.8
## 28	28	1	Vic	m	4	93.8	56.8	87.0	34.5	73.2	53.0
## 29	29	1	Vic	f	3	92.8	56.0	88.0	35.0	74.9	51.8
## 30	30	1	Vic	f	2	92.1	54.4	84.0	33.5	70.6	50.8
## 31	31	1	Vic	m	3	92.8	54.1	93.0	37.0	68.0	52.5
## 32	32	1	Vic	f	4	94.3	56.7	94.0	39.0	74.8	52.0
## 33	33	1	Vic	m	3	91.4	54.6	89.0	37.0	70.8	51.8
## 34	34	2	Vic	m	2	90.6	55.7	85.5	36.5	73.1	53.1
## 35	35	2	Vic	m	4	94.4	57.9	85.0	35.5	71.2	55.5
## 36	36	2	Vic	m	7	93.3	59.3	88.0	35.0	74.3	52.0
## 37	37	2	Vic	f	2	89.3	54.8	82.5	35.0	71.2	52.0
## 38	38	2	Vic	m	7	92.4	56.0	80.5	35.5	68.4	49.5
## 39	39	2	Vic	f	1	84.7	51.5	75.0	34.0	68.7	53.4
## 40	40	2	Vic	f	3	91.0	55.0	84.5	36.0	72.8	51.4
## 41	41	2	Vic	f	5	88.4	57.0	83.0	36.5	NA	40.3
## 42	42	2	Vic	m	3	85.3	54.1	77.0	32.0	62.7	51.2
## 43	43	2	Vic	f	2	90.0	55.5	81.0	32.0	72.0	49.4
## 44	44	2	Vic	m	NA	85.1	51.5	76.0	35.5	70.3	52.6
## 45	45	2	Vic	m	3	90.7	55.9	81.0	34.0	71.5	54.0
## 46	46	2	Vic	m	NA	91.4	54.4	84.0	35.0	72.8	51.2
## 47	47	3	other	m	2	90.1	54.8	89.0	37.5	66.0	45.5
## 48	48	3	other	m	5	98.6	63.2	85.0	34.0	66.9	44.9
## 49	49	3	other	m	4	95.4	59.2	85.0	37.0	69.0	45.0
## 50	50	3	other	f	5	91.6	56.4	88.0	38.0	65.0	47.2
## 51	51	3	other	f	5	95.6	59.6	85.0	36.0	64.0	43.9
## 52	52	3	other	m	6	97.6	61.0	93.5	40.0	67.9	44.3
## 53	53	3	other	f	3	93.1	58.1	91.0	38.0	67.4	46.0
## 54	54	4	other	m	7	96.9	63.0	91.5	43.0	71.3	46.0
## 55	55	4	other	m	2	103.1	63.2	92.5	38.0	72.5	44.9
## 56	56	4	other	m	3	99.9	61.5	93.7	38.0	68.7	46.8
## 57	57	4	other	f	4	95.1	59.4	93.0	41.0	67.2	45.3
## 58	58	4	other	m	3	94.5	64.2	91.0	39.0	66.5	46.4
## 59	59	4	other	m	2	102.5	62.8	96.0	40.0	73.2	44.5
## 60	60	4	other	f	2	91.3	57.7	88.0	39.0	63.1	47.0
## 61	61	5	other	m	7	95.7	59.0	86.0	38.0	63.1	44.9
## 62	62	5	other	f	3	91.3	58.0	90.5	39.0	65.5	41.3
## 63	63	5	other	f	6	92.0	56.4	88.5	38.0	64.1	46.3
## 64	64	5	other	f	3	96.9	56.5	89.5	38.5	63.0	45.1
## 65	65	5	other	f	5	93.5	57.4	88.5	38.0	68.2	41.7
## 66	66	5	other	f	3	90.4	55.8	86.0	36.5	63.2	44.2
## 67	67	5	other	m	4	93.3	57.6	85.0	36.5	64.7	44.1
## 68	68	5	other	m	5	94.1	56.0	88.5	38.0	65.9	43.1

## 69	69	5 other	m	5	98.0	55.6	88.0	37.5	65.0	45.6
## 70	70	5 other	f	7	91.9	56.4	87.0	38.0	65.4	44.1
## 71	71	5 other	m	6	92.8	57.6	90.0	40.0	65.7	42.8
## 72	72	5 other	m	1	85.9	52.4	80.5	35.0	62.0	42.4
## 73	73	5 other	m	1	82.5	52.3	82.0	36.5	65.7	44.7
## 74	74	6 other	f	4	88.7	52.0	83.0	38.0	61.5	45.9
## 75	75	6 other	m	6	93.8	58.1	89.0	38.0	66.2	45.6
## 76	76	6 other	m	5	92.4	56.8	89.0	41.0	64.5	46.4
## 77	77	6 other	m	6	93.6	56.2	84.0	36.0	62.8	42.9
## 78	78	6 other	m	1	86.5	51.0	81.0	36.5	63.0	44.3
## 79	79	6 other	m	1	85.8	50.0	81.0	36.5	62.8	43.0
## 80	80	6 other	m	1	86.7	52.6	84.0	38.0	62.3	44.8
## 81	81	6 other	m	3	90.6	56.0	85.5	38.0	65.6	41.7
## 82	82	6 other	f	4	86.0	54.0	82.0	36.5	60.7	42.9
## 83	83	6 other	f	3	90.0	53.8	81.5	36.0	62.0	43.3
## 84	84	6 other	m	3	88.4	54.6	80.5	36.0	62.6	43.6
## 85	85	6 other	m	3	89.5	56.2	92.0	40.5	65.6	43.5
## 86	86	6 other	f	3	88.2	53.2	86.5	38.5	60.3	43.7
## 87	87	7 other	m	2	98.5	60.7	93.0	41.5	71.7	46.8
## 88	88	7 other	f	2	89.6	58.0	87.5	38.0	66.7	43.5
## 89	89	7 other	m	6	97.7	58.4	84.5	35.0	64.4	46.2
## 90	90	7 other	m	3	92.6	54.6	85.0	38.5	69.8	44.8
## 91	91	7 other	m	3	97.8	59.6	89.0	38.0	65.5	48.0
## 92	92	7 other	m	2	90.7	56.3	85.0	37.0	67.6	46.8
## 93	93	7 other	m	3	89.2	54.0	82.0	38.0	63.8	44.9
## 94	94	7 other	m	7	91.8	57.6	84.0	35.5	64.2	45.1
## 95	95	7 other	m	4	91.6	56.6	88.5	37.5	64.5	45.4
## 96	96	7 other	m	4	94.8	55.7	83.0	38.0	66.5	47.7
## 97	97	7 other	m	3	91.0	53.1	86.0	38.0	63.8	46.0
## 98	98	7 other	m	5	93.2	68.6	84.0	35.0	65.6	44.3
## 99	99	7 other	f	3	93.3	56.2	86.5	38.5	64.8	43.8
## 100	100	7 other	m	1	89.5	56.0	81.5	36.5	66.0	46.8
## 101	101	7 other	m	1	88.6	54.7	82.5	39.0	64.4	48.0
## 102	102	7 other	f	6	92.4	55.0	89.0	38.0	63.5	45.4
## 103	103	7 other	m	4	91.5	55.2	82.5	36.5	62.9	45.9
## 104	104	7 other	f	3	93.6	59.9	89.0	40.0	67.6	46.0
##	eye chest belly									
## 1	15.2	28.0	36.0							
## 2	16.0	28.5	33.0							
## 3	15.5	30.0	34.0							
## 4	15.2	28.0	34.0							
## 5	15.1	28.5	33.0							
## 6	14.2	30.0	32.0							
## 7	14.2	30.0	34.5							
## 8	14.5	29.0	34.0							
## 9	15.5	28.0	33.0							
## 10	14.4	27.5	32.0							
## 11	14.9	31.0	34.0							
## 12	15.3	28.0	33.0							
## 13	15.8	27.0	32.0							
## 14	15.1	28.0	31.5							
## 15	15.7	28.0	35.0							
## 16	14.4	28.0	32.0							
## 17	14.7	29.0	31.0							

##	18	15.3	28.0	32.0
##	19	15.2	28.0	32.0
##	20	15.5	27.0	36.0
##	21	14.2	30.0	40.0
##	22	16.2	28.0	36.0
##	23	15.4	28.0	35.0
##	24	16.2	27.0	32.0
##	25	14.9	31.0	35.0
##	26	15.0	29.0	38.0
##	27	14.2	23.0	28.0
##	28	15.3	27.0	30.0
##	29	14.0	24.0	32.0
##	30	14.5	24.5	33.0
##	31	14.5	27.0	31.0
##	32	14.9	28.0	34.0
##	33	14.8	24.0	30.0
##	34	14.4	26.0	28.5
##	35	16.4	28.0	35.5
##	36	14.9	25.5	36.0
##	37	13.6	28.0	31.5
##	38	15.9	27.0	30.0
##	39	13.0	25.0	25.0
##	40	13.6	27.0	30.0
##	41	15.9	27.0	30.5
##	42	13.8	25.5	33.0
##	43	13.4	29.0	31.0
##	44	14.4	23.0	27.0
##	45	14.6	27.0	31.5
##	46	14.4	24.5	35.0
##	47	15.0	25.0	33.0
##	48	17.0	28.0	35.0
##	49	15.9	29.5	35.5
##	50	14.9	28.0	36.0
##	51	17.4	28.0	38.5
##	52	15.8	28.5	32.5
##	53	16.5	26.0	33.5
##	54	17.5	30.0	36.5
##	55	16.4	30.5	36.0
##	56	16.4	27.5	31.5
##	57	14.5	31.0	39.0
##	58	14.4	30.5	33.0
##	59	14.7	32.0	36.0
##	60	14.4	26.0	30.0
##	61	15.0	26.5	31.0
##	62	16.0	27.0	32.0
##	63	15.2	25.5	28.5
##	64	17.1	25.5	33.0
##	65	14.0	29.0	38.5
##	66	15.7	26.5	34.0
##	67	16.5	27.5	29.5
##	68	17.4	27.0	30.0
##	69	15.0	28.5	34.0
##	70	13.0	27.0	34.0
##	71	15.0	27.5	34.0

```
## 72 14.1 25.5 30.0
## 73 16.0 23.5 28.0
## 74 14.7 26.0 34.0
## 75 16.9 26.0 33.5
## 76 17.8 26.0 33.0
## 77 16.2 25.0 35.0
## 78 13.2 23.0 28.0
## 79 14.8 22.0 28.5
## 80 15.0 23.5 30.5
## 81 17.0 27.5 35.0
## 82 15.4 26.0 32.0
## 83 14.0 25.0 29.0
## 84 16.3 25.0 28.5
## 85 14.5 27.0 31.5
## 86 13.6 26.0 31.0
## 87 15.0 26.0 36.0
## 88 16.0 25.5 31.5
## 89 14.4 29.0 30.5
## 90 14.5 25.5 32.5
## 91 15.0 26.0 32.0
## 92 14.5 25.5 31.0
## 93 12.8 24.0 31.0
## 94 14.4 29.0 35.0
## 95 14.9 27.0 31.0
## 96 14.0 25.0 33.0
## 97 14.5 25.0 31.5
## 98 14.5 28.5 32.0
## 99 14.0 28.0 35.0
## 100 14.8 23.0 27.0
## 101 14.0 25.0 33.0
## 102 13.0 25.0 30.0
## 103 15.4 25.0 29.0
## 104 14.8 28.5 33.5
```

6.3.1 Exercise

- From the possum data, create a table which only has the site, sex, and age data for female possums under the age of 5. Arrange this table from youngest to oldest.

6.4 join() ME!



The tidyverse comes with a number of very useful functions to join related data frames. These functions all work with two data frames at a time and fall into two main categories:

Mutating Joins

- `left_join()`, `right_join()`
- `inner_join()`
- `full_join()`

Filtering Joins

- `semi_join()`
- `anti_join()`

To demonstrate how joins work, we will need some relational data sets. Relational data is data which shares some overlapping ‘key’ variable (or variables) but each data set has some data which the other does not. Joining is a way of combining these data sets - but there are a surprising number of ways to do this.

Here are two very small data sets we can start with:

```
##      name smoker gender diagnosis dose
## 1  Magneto    yes  male  diabetes  250
## 2   Storm    yes female diabetes  500
## 3 Mystique   no female      IBD  400
## 4   Batman    yes  male arthritis 325
## 5    Joker   no  male arthritis 650
## 6 Catwoman   yes female      IBD  400
## 7  Hellboy   no  male      lupus   50

##      diagnosis medication
## 1    diabetes metformin
## 2   arthritis  tylenol
## 3         IBD    5-ASA
## 4 Hypertension  ramipril
```

We have doses for medications in the first table, but we don’t know which medications each patient is taking!

Let’s join the two tables using `left_join()`:

```
left_join(patients, medications)
```

```
##      name smoker gender diagnosis dose medication
## 1  Magneto   yes  male  diabetes  250  metformin
## 2   Storm   yes female diabetes  500  metformin
## 3 Mystique  no  female      IBD  400    5-ASA
## 4   Batman   yes  male arthritis 325   tylenol
## 5    Joker   no  male arthritis 650   tylenol
## 6 Catwoman  yes female      IBD  400    5-ASA
## 7  Hellboy  no   male    lupus   50     <NA>
```

Notice that we get a message - the function has figured out which column is the 'key' column - the column which is present in both data sets. Sometimes, though, we don't want to leave this to chance. Better to specify this using the 'by' argument.

```
left_join(patients, medications, by = "diagnosis")
```

We get NA in our output because we don't have a full set of medication data for each diagnosis.

`left_join()` keeps all the rows in the first (left hand) table and adds columns from the second. `right_join()` does the opposite:

```
right_join(patients, medications, by = "diagnosis")
```

```
##      name smoker gender      diagnosis dose medication
## 1  Magneto   yes  male    diabetes  250  metformin
## 2   Storm   yes female    diabetes  500  metformin
## 3 Mystique  no  female      IBD  400    5-ASA
## 4   Batman   yes  male    arthritis 325   tylenol
## 5    Joker   no  male    arthritis 650   tylenol
## 6 Catwoman  yes female      IBD  400    5-ASA
## 7    <NA>  <NA>  <NA> Hypertension  NA   ramipril
```

Notice now that we lost some information about Hellboy because his diagnosis wasn't in the medications table.

Usually we use `left_join()`, but sometimes when we are piping (we'll get to that soon), `right_join()` comes in handy.

`full_join()` keeps all the rows from both data sets, again introducing NA as needed:

```
full_join(patients, medications, by = "diagnosis")
```

```
##      name smoker gender      diagnosis dose medication
## 1  Magneto   yes  male    diabetes  250  metformin
## 2   Storm   yes female    diabetes  500  metformin
## 3 Mystique  no  female      IBD  400    5-ASA
## 4   Batman   yes  male    arthritis 325   tylenol
## 5    Joker   no  male    arthritis 650   tylenol
## 6 Catwoman  yes female      IBD  400    5-ASA
## 7  Hellboy  no   male    lupus   50     <NA>
## 8    <NA>  <NA>  <NA> Hypertension  NA   ramipril
```

And finally, `inner_join()` keeps only rows which are common to both data sets:

```
inner_join(patients, medications, by = "diagnosis")
```

```
##      name smoker gender diagnosis dose medication
## 1  Magneto   yes  male  diabetes  250  metformin
```

```
## 2    Storm    yes female diabetes 500 metformin
## 3 Mystique    no female      IBD  400      5-ASA
## 4    Batman    yes  male arthritis 325   tylenol
## 5     Joker    no  male arthritis 650   tylenol
## 6 Catwoman    yes female      IBD  400      5-ASA
```

These four functions are called mutating joins because they add columns. We'll meet `mutate()` soon and you will see why that makes sense.

The other main type of join are filtering joins. We just met `filter()` so we remember that filtering is a way of choosing rows. Filtering joins only ever remove rows. No new columns get introduced.

`semi_join()` keeps all the rows in the first table which have a match in the second table:

```
semi_join(patients, medications, by = "diagnosis")
```

```
##      name smoker gender diagnosis dose
## 1  Magneto    yes  male  diabetes  250
## 2   Storm    yes female diabetes  500
## 3 Mystique    no female      IBD  400
## 4   Batman    yes  male arthritis  325
## 5    Joker    no  male arthritis  650
## 6 Catwoman    yes female      IBD  400
```

Whereas `anti_join()` *drops* all the rows in the first table which have a match in the second table:

```
anti_join(patients, medications, by = "diagnosis")
```

```
##      name smoker gender diagnosis dose
## 1 Hellboy    no  male      lupus    50
```

Remember, these two functions only ever *remove* data.

There is a third category of two table functions. These are the set operations, and you might find them useful for the exercises. These functions assume you have two data frames with all the same variables (columns). They are pretty self-explanatory:

- `intersect(x, y)`: return only observations in both x and y
- `union(x, y)`: return unique observations in x and y
- `setdiff(x, y)`: return observations in x, but not in y.

6.4.1 Exercises

1. Is the output from `right_join(patients, medications)` the same as `left_join(patients, medications)`? Decide what you think the answer is before trying it out in your R session.
2. Here's some more data:

```
##      patient script_ID license
## 1  Magneto         15   23546
## 2   Storm         22   23546
## 3 Mystique         5    6536
## 4 Catwoman         4    6536
## 5 Hellboy          5   23546
## 6 Hellboy         12   67589

##      patient script_ID license
## 1  Magneto         17   32567
## 2   Storm         22   23546
## 3 Mystique         55   6536
```

```
## 4 Catwoman      24    6536
## 5 Hellboy       51   23546
## 6 Hellboy       12   67589
## 7 Storm        22   23546
## 8 Mystique      5    6536
## 9 Catwoman      4    6536
## 10 Hellboy      5    23546

## physician license      speciality
## 1 Prof.X    23546    endocrinology
## 2 Grey      6536    gastroenterology
## 3 Frost     67589    family.practice
## 4 Agent     32567    family.practice
```

Use mutating and filtering joins to piece all the data together and figure out:

- What meds are over the counter? (No prescriptions)
- Which specialities prescribe metformin?

Note that the two scripts table contain some overlapping data - make sure to only look at each script_ID once!

You'll also need to know that if you are trying to join by columns which have different names, you can use `by = c("x" = "a")`.

6.5 A tidy way to handle strings

In lesson four we learned some useful tricks for handling strings using `str_replace()`, `str_detect()`, `str_to_upper()`, `str_to_lower()`, and `str_split()`. These are all in the `stringr` package (included in the tidyverse) and you may find some of these come in handy when working to clean your data:

- `str_c()`
- `str_length()`
- `str_pad()`, `str_trim()`
- `str_to_upper()`, `str_to_lower()`, `str_to_title()`
- `str_order()`, `str_sort()`
- `str_detect()`, `str_sub()`, `str_replace()`
- `str_count()`
- `str_split()`

These are fairly self explanatory based on the name. We can jump right into trying them and seeing how they work.

6.5.1 Exercises

Take another crack at the exercises from lesson four and see if you can perform the same task using tidyverse functions. Notice any differences in behavior or ease of use?

1. Convert a full name (let's start with `name <- "William Osler"`) to an abbreviated name made up of the first three letters of the first name and the first the letters of the last name, all in lower case. For example "Stephen Master" would become "stemas".
2. Read in `starwars_menu.csv`. The data is the pre-selected menu for a banquet. Have a look at the data. The caterer needs to know how many individuals want "whey" but the "whey" is buried in things like "the whey, the shoots and the rice".
 - How many people want "whey"?
 - Produce a data frame of all the rows containing the word whey.
 - Which individuals chose "whey"?

- Which individuals did not choose “whey”?
3. Use this character vector to experiment with the functions `str_order()`, `str_sort()`, `str_count()`. Can you figure out what each of these functions does? What do `str_pad()`, `str_trim()`, `str_squish()` do?

```
letters <- c("e", "r", "q", "a", "a", "f", "c", "x", "z")
```

7 Piping, Mutating, and Summarizing

7.1 Pipes

Before we go further, we are going to employ a function called the pipe which is denoted `%>%`. This is analogous to the OSX and Linux command line pipe `|` used (constantly) in shell programming. What it does is “push” the output of one function to another function which avoids repeatedly nested functions.

Here is an example. This can be hard to read when many functions are nested:

```
# traditional nested functions
possum <- read.csv("Data_Files/possum.csv")
summary(select(filter(possum, sex == "m"), matches("hdlngth")))
```

```
##      hdlngth
## Min.      : 82.50
## 1st Qu.: 90.70
## Median : 93.20
## Mean   : 92.92
## 3rd Qu.: 95.40
## Max.    :103.10
```

But it’s easier to read with pipes:

```
# compare with piped functions
possum %>% filter(sex == "m") %>%
  select(hdlngth) %>%
  summary()
```

```
##      hdlngth
## Min.      : 82.50
## 1st Qu.: 90.70
## Median : 93.20
## Mean   : 92.92
## 3rd Qu.: 95.40
## Max.    :103.10
```

When we pipe, we start with the data we want to work with. All the functions which come next in the ‘pipeline’ inherit the data as it is fed to them. So we don’t need to specify what data we want each function to use, as the default is to use whatever has come ‘down the pipe’ to that function.

If this is confusing, imagine the `%>%` operator as being the adverb ‘then’, with the functions being the verbs, and the data being the noun (or subject). So the above pipe can be ‘read’ as the sentence “Start with the possum data, then filter rows with sex equal to ‘m’, then select the column ‘hdlngth’, then provide a summary.”

7.2 `mutate()` and `transmute()`

The `mutate()` function permits the calculation and addition of a new column. First let’s use the electrolyte results from earlier and add a column with the AG the traditional way:

```
#traditional way
lytes.long <- read.csv("Data_Files/lytes.csv")
lytes.wide <- pivot_wider(lytes.long, names_from = "Test", values_from = "Result" )
lytes.wide$Anion.Gap <- lytes.wide$Sodium + lytes.wide$Potassium - lytes.wide$Chloride - lytes.wide$Total.CO2
lytes.wide
```

```
## # A tibble: 4 x 6
##   Patient Sodium Potassium Chloride `Total CO2` Anion.Gap
##   <int>   <dbl>     <dbl>   <dbl>     <dbl>     <dbl>
## 1     1     143       4.2     110       25      12.2
## 2     2     129       4.4     100       18      15.4
## 3     3     142       4.6     105       27      14.6
## 4     4     148       5       111       25      17
```

Now try with pipes and mutate()

```
Anion_Gaps <- lytes.wide %>%
  mutate(Anion.Gap = Sodium + Potassium - Chloride - `Total CO2`)
Anion_Gaps
```

```
## # A tibble: 4 x 6
##   Patient Sodium Potassium Chloride `Total CO2` Anion.Gap
##   <int>   <dbl>     <dbl>   <dbl>     <dbl>     <dbl>
## 1     1     143       4.2     110       25      12.2
## 2     2     129       4.4     100       18      15.4
## 3     3     142       4.6     105       27      14.6
## 4     4     148       5       111       25      17
```

See how piping keeps our code cleaner and easier to type?

The `transmute()` function does the same as `mutate()` but discards all the columns in the original data frame and only returns the newly calculated column:

```
Anion.Gaps.2 <- lytes.wide %>%
  transmute(Anion.Gap = Sodium + Potassium - Chloride - `Total CO2`)
Anion.Gaps.2
```

```
## # A tibble: 4 x 1
##   Anion.Gap
##   <dbl>
## 1    12.2
## 2    15.4
## 3    14.6
## 4     17
```

If you want to retain columns with transmute, you can just name them and they will stay:

```
#keep the patient IDs
Anion.Gaps.3 <- lytes.wide %>%
  transmute(Patient,
            Anion.Gap = Sodium + Potassium - Chloride - `Total CO2`)
Anion.Gaps.3
```

```
## # A tibble: 4 x 2
##   Patient Anion.Gap
##   <int>     <dbl>
## 1     1     12.2
## 2     2     15.4
```

```
## 3      3      14.6
## 4      4      17
```

7.2.1 Exercise

- Load the starwars dataset and use `mutate()` to add a column to the starwars dataframe with the BMI of the character. You can simply type `data(starwars)` to get the starwars data to load.
- What is Jabba's BMI?

$$BMI = \frac{mass(kg)}{[height(cm)]^2} \times 10000$$

7.3 separate() and unite()

These are pretty easy functions to understand. The `separate()` function breaks columns up based on a separator.

```
names <- c("Dan", "Janet", "Andre", "Angela", "Mari") %>% factor()
startdate <- c("04-Jul-2006", "01-Jul-2016", "01-Sep-2009", "01-Jul-2016", "01-Jul-2012")
staff <- data.frame(names, startdate)
staff %>%
  separate(startdate,
            into = c("StartDay", "StartMonth", "StartYear"),
            sep = "-") -> staff.sep
staff.sep
```

```
##   names StartDay StartMonth StartYear
## 1   Dan      04      Jul      2006
## 2 Janet      01      Jul      2016
## 3 Andre      01      Sep      2009
## 4 Angela      01      Jul      2016
## 5  Mari      01      Jul      2012
```

And `unite()` does the opposite.

```
staff.sep %>% unite(StartDate, StartDay, StartMonth, StartYear, sep = "|")
```

```
##   names StartDate
## 1   Dan 04|Jul|2006
## 2 Janet 01|Jul|2016
## 3 Andre 01|Sep|2009
## 4 Angela 01|Jul|2016
## 5  Mari 01|Jul|2012
```

The first argument to `unite` is the name of the new column you are creating, and then list the columns you want to unite.

###Exercise Using the 'staff' data frame we created above, can you separate the 'startdate' column into columns for day, month and year while keeping the original column?

Desired output:

```
##   names startdate StartDay StartMonth StartYear
## 1   Dan 04-Jul-2006      04      Jul      2006
## 2 Janet 01-Jul-2016      01      Jul      2016
## 3 Andre 01-Sep-2009      01      Sep      2009
## 4 Angela 01-Jul-2016      01      Jul      2016
## 5  Mari 01-Jul-2012      01      Jul      2012
```

Hint - to see what else a function can do, type '?' and then the function name to bring up the documentation. Often functions have hidden arguments which can do useful things.

7.4 group_by() %>% summarize()

The `summarize()` function allows you to rapidly produce summary statistics and it is usually (but not always) used with another function called `group_by()`

```
summarize(possum, m.skullw = mean(skullw), sd.skullw = sd(skullw), count = n())
```

```
##   m.skullw sd.skullw count
## 1 56.88365  3.113426   104
```

Used by itself, this is not all that useful or interesting but when paired with `group_by()` it is very useful.

7.4.1 Using group_by() %>% summarize()

When `summarize()` is combined with grouping by factors, we get very rapid summaries that would be traditionally more cumbersome.

They can be used to produce counts:

```
#group_by summarize
iris %>%
  group_by(Species) %>%
  summarize(count = n())
```

```
## # A tibble: 3 x 2
##   Species    count
##   <fct>      <int>
## 1 setosa         50
## 2 versicolor    50
## 3 virginica     50
```

They can be used to calculate summary statistics:

```
#demonstrate group_by summarize with iris in data wide
iris %>%
  group_by(Species) %>%
  summarize(m.Sepal.Length = mean(Sepal.Length),
            m.Sepal.Width = mean(Sepal.Width),
            m.Petal.Length = mean(Petal.Length),
            m.Petal.Width = mean(Petal.Width))
```

```
## # A tibble: 3 x 5
##   Species    m.Sepal.Length m.Sepal.Width m.Petal.Length m.Petal.Width
##   <fct>          <dbl>          <dbl>          <dbl>          <dbl>
## 1 setosa         5.01            3.43            1.46            0.246
## 2 versicolor    5.94            2.77            4.26            1.33
## 3 virginica     6.59            2.97            5.55            2.03
```

There are special summarize functions to do the same thing in one line:

```
#demonstrate the compactness of summarize_all()
iris %>%
  group_by( Species) %>%
  summarize_all(funs(mean)) -> iris.res1
iris.res1
```

```
## # A tibble: 3 x 5
##   Species    Sepal.Length Sepal.Width Petal.Length Petal.Width
##   <fct>         <dbl>         <dbl>         <dbl>         <dbl>
## 1 setosa         5.01           3.43           1.46           0.246
## 2 versicolor    5.94           2.77           4.26           1.33
## 3 virginica     6.59           2.97           5.55           2.03

#multiple functions
iris %>%
  group_by(Species) %>%
  summarize_all(funs(mean, median, sd)) -> iris.res2
iris.res2

## # A tibble: 3 x 13
##   Species    Sepal.Length_mean Sepal.Width_mean Petal.Length_mean Petal.Width_mean
##   <fct>         <dbl>         <dbl>         <dbl>         <dbl>
## 1 setosa         5.01           3.43           1.46           0.246
## 2 versico~     5.94           2.77           4.26           1.33
## 3 virgini~     6.59           2.97           5.55           2.03
## # i 8 more variables: Sepal.Length_median <dbl>, Sepal.Width_median <dbl>,
## #   Petal.Length_median <dbl>, Petal.Width_median <dbl>, Sepal.Length_sd <dbl>,
## #   Sepal.Width_sd <dbl>, Petal.Length_sd <dbl>, Petal.Width_sd <dbl>
```

The summary can be applied to specific columns:

```
#specific columns
iris.res3 <- iris %>%
  group_by(Species) %>%
  summarize_at(c("Petal.Length", "Petal.Width"),
    list(mean = mean, median = median, sd = sd))
iris.res3

## # A tibble: 3 x 7
##   Species    Petal.Length_mean Petal.Width_mean Petal.Length_median
##   <fct>         <dbl>         <dbl>         <dbl>
## 1 setosa         1.46           0.246           1.5
## 2 versicolor    4.26           1.33           4.35
## 3 virginica     5.55           2.03           5.55
## # i 3 more variables: Petal.Width_median <dbl>, Petal.Length_sd <dbl>,
## #   Petal.Width_sd <dbl>
```

7.4.2 Using group_by() %>% summarize() with data in long format

But if we put the data in a datalong format, we get real horsepower and this can also be used to generate quick visualizations by factors as we will show later.

```
iris.long <- iris %>%
  pivot_longer(1:4,
    names_to = "Flower.Part",
    values_to = "Length")

#find the means
iris.res4 <- iris.long %>%
  group_by(Species, Flower.Part) %>%
  summarize(mean = mean(Length))
iris.res4
```

```
## # A tibble: 12 x 3
## # Groups:   Species [3]
##   Species    Flower.Part    mean
##   <fct>      <chr>      <dbl>
## 1 setosa     Petal.Length 1.46
## 2 setosa     Petal.Width  0.246
## 3 setosa     Sepal.Length 5.01
## 4 setosa     Sepal.Width  3.43
## 5 versicolor Petal.Length 4.26
## 6 versicolor Petal.Width  1.33
## 7 versicolor Sepal.Length 5.94
## 8 versicolor Sepal.Width  2.77
## 9 virginica  Petal.Length 5.55
## 10 virginica Petal.Width  2.03
## 11 virginica Sepal.Length 6.59
## 12 virginica Sepal.Width  2.97
```

#find a bunch of summary stats

```
iris.res5 <- iris.long %>%
  group_by(Species, Flower.Part) %>%
  summarize_all(list(min = min, mean = mean, sd = sd, med = median, IQR = IQR, max = max))
iris.res5
```

```
## # A tibble: 12 x 8
## # Groups:   Species [3]
##   Species    Flower.Part    min mean    sd    med    IQR    max
##   <fct>      <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 setosa     Petal.Length 1    1.46 0.174 1.5 0.175 1.9
## 2 setosa     Petal.Width 0.1 0.246 0.105 0.2 0.1    0.6
## 3 setosa     Sepal.Length 4.3 5.01 0.352 5   0.400 5.8
## 4 setosa     Sepal.Width 2.3 3.43 0.379 3.4 0.475 4.4
## 5 versicolor Petal.Length 3    4.26 0.470 4.35 0.600 5.1
## 6 versicolor Petal.Width 1    1.33 0.198 1.3 0.3    1.8
## 7 versicolor Sepal.Length 4.9 5.94 0.516 5.9 0.7    7
## 8 versicolor Sepal.Width 2    2.77 0.314 2.8 0.475 3.4
## 9 virginica  Petal.Length 4.5 5.55 0.552 5.55 0.775 6.9
## 10 virginica Petal.Width 1.4 2.03 0.275 2   0.5    2.5
## 11 virginica Sepal.Length 4.9 6.59 0.636 6.5 0.675 7.9
## 12 virginica Sepal.Width 2.2 2.97 0.322 3   0.375 3.8
```

Note that all the summarise/summarize verbs work with either the US or UK/Canadian spelling.

7.5 Putting all the pipes together

7.5.1 Exercise

Using pipes and the various tidyverse ‘verbs’ (aka functions) you’ve learned in the last two lessons, write a pipe which starts with the dataframe ‘possum’ from earlier and returns a dataframe which gives us the number of female possums and the mean age and median foot to tail length ratio for just the female possums, grouped by research site and population? Can you then make the resulting table ‘tidy’?

Start with this:

```
possum.f.sum <- possum %>%
  filter() %>%
  mutate() %>%
```

And add what you need.

Desired output:

```
## # A tibble: 21 x 4
## # Groups:   site [7]
##   site Pop  Metric      Value
##   <int> <chr> <chr>    <dbl>
## 1     1   Vic mean_age    4.42
## 2     2   Vic mean_age    2.6
## 3     3 other mean_age    4.33
## 4     4 other mean_age     3
## 5     5 other mean_age    4.5
## 6     6 other mean_age    3.5
## 7     7 other mean_age    3.5
## 8     1   Vic median_foot_to_tail 1.99
## 9     2   Vic median_foot_to_tail 2.03
## 10    3 other median_foot_to_tail 1.77
## # i 11 more rows
```

8 Lubridate

8.1 A sign of the times: lubridate()

The `lubridate()` package allows us to deal with dates and times and do algebra on them as we would with other vectors. This represents a major advantage over the handling of dates using base R packages.

Lubridate makes logical assumptions about what you probably mean based on typical date formats.

The functions that lubridate uses are `mdy()`, `ymd()` and `dmy()`. They have very predictable behavior.

```
library(lubridate)
# May need to change locale to English US if errors are seen
# Sys.setlocale("LC_TIME", "en_US.UTF-8") #OSX
# Sys.setlocale("LC_TIME", "English") #Windows
mdy("Aug-20,1755")
```

```
## [1] "1755-08-20"
```

```
mdy("Aug/20/1755")
```

```
## [1] "1755-08-20"
```

```
mdy("08-20-1755")
```

```
## [1] "1755-08-20"
```

```
mdy("08201755")
```

```
## [1] "1755-08-20"
```

```
mdy("August 20, 1755")
```

```
## [1] "1755-08-20"
```

```
mdy("August 20 1755")
```

```
## [1] "1755-08-20"
```

```
#all work perfectly without any explicit statements about format.
```

You can also change the language to suit your preferences. For example, if we want to work in French dates.

```
dmy("Lundi le 12 Septembre, 2016") #gives us an error
```

```
## [1] NA
```

```
Sys.setlocale("LC_TIME", "fr_CA.UTF-8") #change settings to French Canadian
```

```
## [1] "fr_CA.UTF-8"
```

```
dmy("Lundi le 12 Septembre, 2016")
```

```
## [1] "2016-09-12"
```

```
Sys.setlocale("LC_TIME", "en_US.UTF-8") #change back to English USA
```

```
## [1] "en_US.UTF-8"
```

The real gold with lubridate is how it can handle times too!

```
#calculating the number of days since the Declaration of Independence  
then <- mdy_hms("July 04, 1776 14:32:45")  
then
```

```
## [1] "1776-07-04 14:32:45 UTC"
```

```
now <- ymd_hms(Sys.time())  
now
```

```
## [1] "2025-08-11 17:58:31 UTC"
```

```
delta <- difftime(now, then, units = "days")  
delta #wow
```

```
## Time difference of 90983.14 days
```

```
#calculating the number of days Canada has been a country  
then <- mdy_hms("July 01, 1867 11:17:21")  
then
```

```
## [1] "1867-07-01 11:17:21 UTC"
```

```
now <- ymd_hms(Sys.time())  
now
```

```
## [1] "2025-08-11 17:58:31 UTC"
```

```
delta <- difftime(now, then, units = "days")  
delta
```

```
## Time difference of 57750.28 days
```

8.1.1 Exercise

1. Read the file Potassium.csv into a variable called `K.data`. This file contains real K^+ data extracted from SunQuest from Dan's lab for one month from the two ER bays. It contains order time, collection time, receive time and result time. Use the `head()` function to get an idea of how the dates and times are formatted. All the K^+ results presented were run on an ABL800 whole blood analyzer directly from an unspun PST tube.

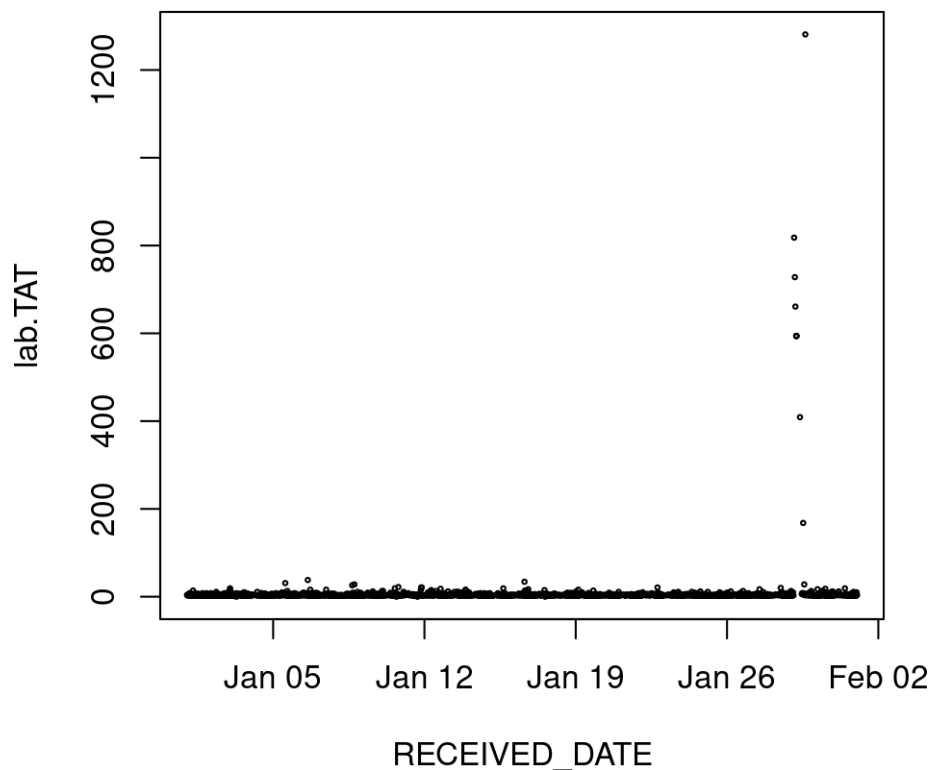
- Convert the order, receive, and result times into dates
- Calculate the order-to-result times and store it a column called TAT (“turn around time”). Use the function `difftime()` to calculate the time difference.
 - What is the median and IQR of the TAT?
 - What is the 90th and 99th percentiles of the TAT?
 - What is the maximum value of TAT?
- Calculate the receive-to-result times and store it a column called lab.TAT (the analysis time).
 - What is the median and IQR of the lab.TAT?
 - What is the 90th and 99th percentiles of the lab.TAT?
 - What is the maximum value of lab.TAT? What day did it occur?
 - What strange finding have you discovered?

2. Produce a histogram of TAT and lab.TAT on the same histogram. Recall: `hist(my.vector, breaks = 20, add = TRUE)`

Start with your hist of TAT so that the x axis is long and can display both TAT and lab.TAT. In this particular, `breaks = 20` works nicely for TAT and `breaks = 200` works nicely for lab.TAT. This was just trial and error.

From this kind of data we can plot the daily turn around time, which can be really helpful. We won’t do it now but you have the code here so you can see what is possible.

```
plot(lab.TAT ~ RECEIVED_DATE, data = K.data, cex = 0.3)
```

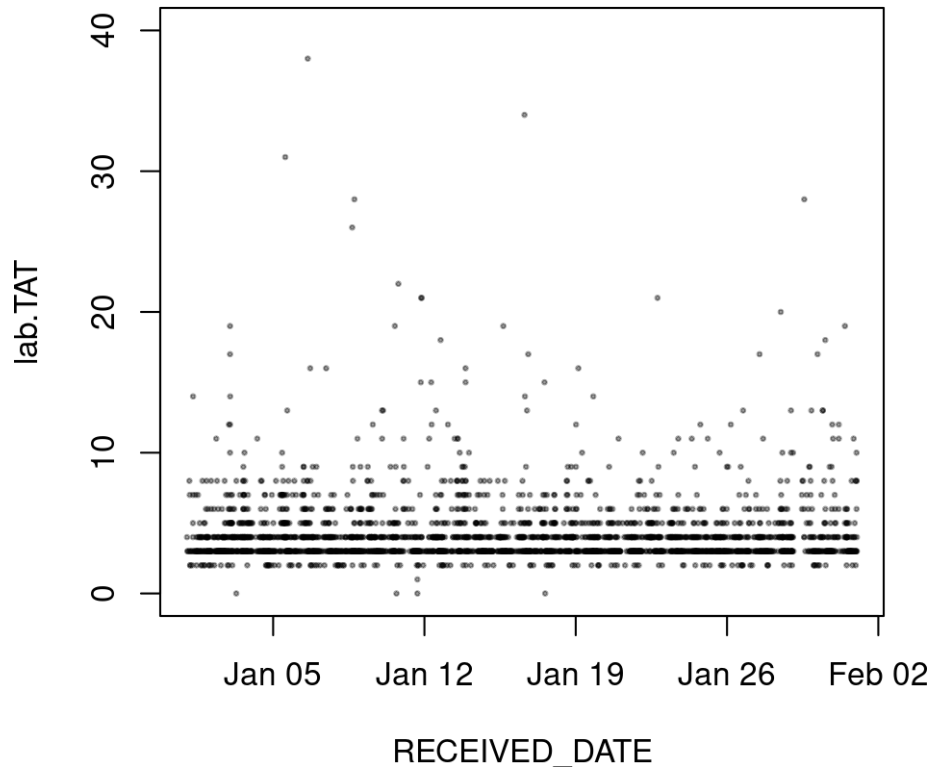


```
#the LIS downtime on Jan 29 is evident
```

Making it a little prettier:

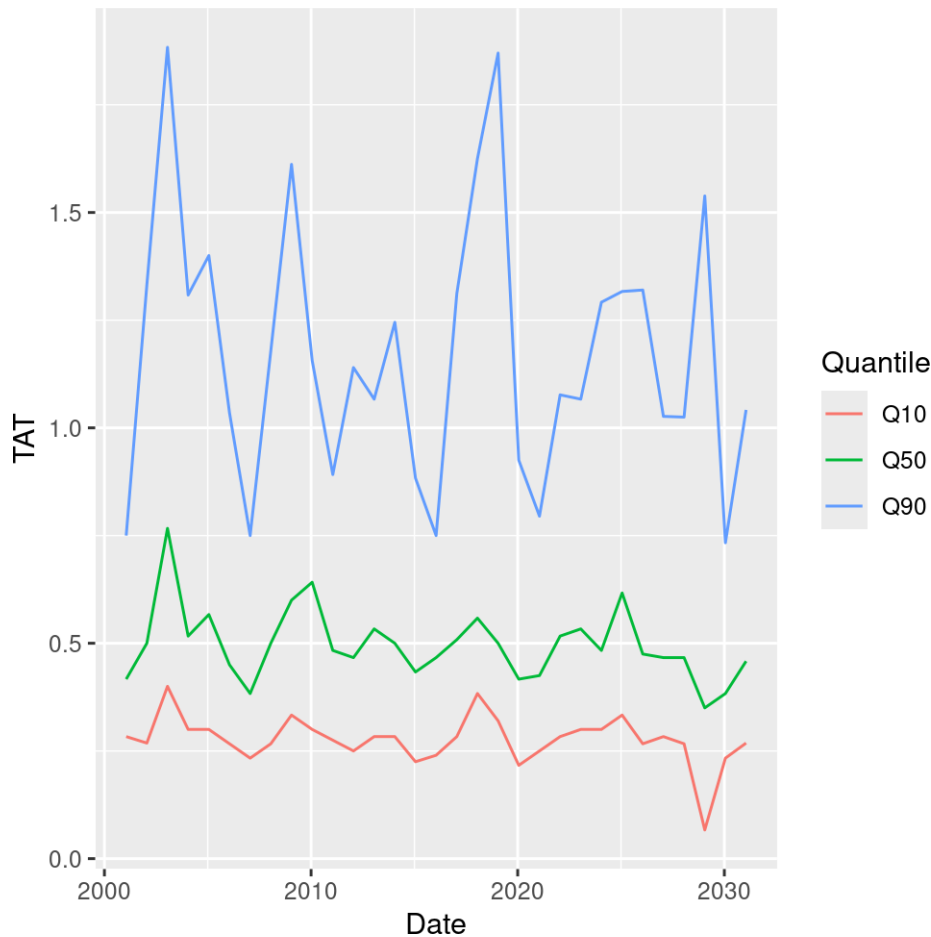
```
plot(lab.TAT ~ RECEIVED_DATE,
     data = K.data,
     ylim = c(0,40),
     col = "#00000060",
```

```
pch = 19,
cex = 0.3) #as it happens individual TATs.
```



And you can do lots of other things with this data. For demonstration purposes only, in future you will be able to plot daily statistics with some very compact syntax. Plunk this in and see what you get.

```
library(ggplot2)
library(tidyr)
K.data <- read.csv("Data_Files/Potassium.csv")
K.data %>%
  mutate(across(COLLECTION_DATE:RECEIVED_DATE, ymd_hms)) %>%
  mutate(TAT = difftime(RECEIVED_DATE, ORDERED_DATE, units = "min")) %>%
  mutate(Date = as.Date(COLLECTION_DATE)) %>%
  group_by(Date) %>%
  summarise(Q10 = quantile(TAT, probs = 0.1), Q50 = median(TAT), Q90 = quantile(TAT, probs = 0.9))
  pivot_longer(Q10:Q90, names_to = "Quantile", values_to = "TAT") %>%
  ggplot(aes(x = Date, y = TAT, col = Quantile)) +
  geom_line()
```



9 Functions, Conditional Responses and Loops

9.1 Making your own functions

It is frequently necessary to create a function that does a specific calculation so you do not need to repeatedly write out the same code. We are going to create some functions, save them, and reload them for future convenient use. This way, we are going to save ourselves *so much* work.

The syntax of the custom function looks like this:

```
function.name <- function(arg1,arg2,arg3 - ie input variables) {
  statements to generate an R object
  return(object)
}
```

A simple function might look like this:

```
square <- function(x) { # x is just a variable name. Any allowable variable name is OK
  result <- x^2         # calculate the square and assign to a variable "result"
  return(result)        # return the value of the result variable
}
square(5)
```

```
## [1] 25
```

Note that variables used in functions are local to the function and cannot be called outside the function.

In other words, you can use the variable `x` anywhere else in your script and it will not interfere with your `square()` function. Note that if you define a variable inside your function, you will not be able to inquire as to its value after the function has been executed.

Let's look at a more complex example. We will go back to handling strings, first by using `substr()` to take a word and remove the first and last letters.

```
chopper <- function(word) {  
  return(substr(word, 2, nchar(word) - 1))  
}  
  
# now try it out on some words  
# will it work on a sentence?  
  
# stringr() library functions can do the same  
chopper_tidy <- function(word) {  
  return(str_sub(word, 2, str_count(word) - 1))  
}
```

9.1.1 Exercise

1. Write a function called `cv` that takes a vector of numbers and calculates the CV of those numbers in %. Recall that $%CV = sd/mean \times 100$.
2. Write a function called `clean` that removes any less than or greater than signs from your data but preserves the number that follows and returns a numeric vector of your data.

Things you should be aware of when making your own functions:

- You are not limited to having variables as the thing a function does. The function can generate graphs, read or write files, and do anything that R can do.
- You can have as many input variables and output variables as you want. If you are returning multiple output variables, you can return them as a dataframe or a list.
- You can save your function as a text file and then read it back in at another time using the `source()` function.

9.2 Conditional Responses

It is very common to have to provide responses based on conditions that are not known *a priori*. For example, you might have to do something different if you have an even number of components to a vector than if you had an odd number of components. Or you might have to apply a different calculation for eGFR for men than for women. In these cases, we need to use “if” statements to make the right decision. The structure goes like this:

```
if (condition 1) {  
  do something  
} else if (condition 2) {  
  do something else  
} else {  
  do something with everything that fails conditions 1 and 2  
}
```

Here's a useless function that shows how `if` statements work. It takes a name and tells you if the name corresponds to either of the instructors.

```
is.teaching.now <- function(name) {  
  if (name == "Dan" | name == "Daniel") {  
    response <- TRUE  
  } else if (name == "Nicholas" | name == "Nick") {
```

```

    response <- TRUE
  } else {
    response <- FALSE
  }
  return(response)
}
#try it
is.teaching.now("Wolfgang")

```

```
## [1] FALSE
```

```
is.teaching.now("Nick")
```

```
## [1] TRUE
```

Here is another function that tells you if a number is even.

```

iseven <- function(x) {
  if (floor(x/2) == (x/2)) {
    return(TRUE)
  } else {
    return(FALSE)
  }
}
#try it
iseven(8)

```

```
## [1] TRUE
```

```
iseven(19)
```

```
## [1] FALSE
```

```
iseven(pi)
```

```
## [1] FALSE
```

```
iseven(round(2*pi,0))
```

```
## [1] TRUE
```

9.2.1 Exercise

1. Write a function that tells you whether a number is an integer or not by using the `floor()` function. If it's an integer, have the function return the string "This number has integrity". Otherwise have it return "This number lacks integrity".
2. Write a function that takes a sentence and determines if the word "schnitzel" is in the sentence. It should return "This sentence contains schnitzel" or "This sentence is schnitzel-free". You may find a function called `%in%` helpful - this tells you if quantity is in a vector. For example:

```

sentence <- c("It's", "a", "beautiful", "day", "in", "Salzburg")
"Paris" %in% sentence

```

```
## [1] FALSE
```

```
"Salzburg" %in% sentence
```

```
## [1] TRUE
```

9.3 Loops—When you need to say the same thing over and over and over

Very often we need to do the same thing over and over and perform a calculation or inquiry over an entire vector. (See also the `apply()` family of functions or the `purrr` package). Loops help us to accomplish things like this, and they are an essential tool in many things we will want to do.

There are two kinds of loops: the “for” loop and the “while” loop. In general we’ll use “for” more often but we will cover both.

The syntax of the `for` loop goes like this:

```
for (counter in sequence) {  
  do something  
}
```

So, in reality, that might look like this:

```
for (i in 1:5) {  
  print(c("Hi number",i))  
}
```

```
## [1] "Hi number" "1"  
## [1] "Hi number" "2"  
## [1] "Hi number" "3"  
## [1] "Hi number" "4"  
## [1] "Hi number" "5"
```

The `iseven()` function that we created earlier in this chapter did not function on a vector. It only worked for single numbers. Let’s make a loop to apply it to a vector of numbers.

```
x <- c(1,3,4,2,5,6,7,3,10) # defines a vector to operate on  
result <- vector() # creates an empty vector to store our result in  
for (i in 1:length(x)) {  
  result[i] <- iseven(x[i])  
}  
result
```

```
## [1] FALSE FALSE TRUE TRUE FALSE TRUE FALSE FALSE TRUE
```

```
# Seems to work. Now let's use it to pull out the numbers.  
x[result]
```

```
## [1] 4 2 6 10
```

You can also embed loops inside of loops and, where necessary, call the indices of the outer loop from within the inner loop. You may better appreciate the value of `group_by()` and `summarize()` after realizing that the code authors are doing this nested looping for you with their commands.

```
# Examine features of irises  
# Loop over the different species and find the mean of the  
# 4 different petal measurements.  
iris <- read.csv("Data_Files/iris.csv")  
species <- unique(iris$Species)  
features <- names(iris)[1:4]  
mean.results <- data.frame(species = rep(NA,3),  
                           m.Sepal.Length = rep(NA,3),  
                           m.Sepal.Width = rep(NA,3),  
                           m.Petal.Length = rep(NA,3),  
                           m.Petal.Width = rep(NA,3))  
for (i in 1:length(species)){
```

```

flower.data <- subset(iris, Species == species[i])
mean.results[i,1]<- as.character(species[i])
for (j in 1:length(features)){
  mean.results[i,j + 1] <- mean(flower.data[,j], na.rm = TRUE)
}
}
mean.results

##      species m.Sepal.Length m.Sepal.Width m.Petal.Length m.Petal.Width
## 1      setosa          5.006          3.428          1.462          0.246
## 2 versicolor          5.936          2.770          4.260          1.326
## 3 virginica          6.588          2.974          5.552          2.026

# compare
iris %>%
  group_by(Species) %>%
  summarise_all(mean)

## # A tibble: 3 x 5
##   Species    Sepal.Length Sepal.Width Petal.Length Petal.Width
##   <chr>          <dbl>         <dbl>         <dbl>         <dbl>
## 1 setosa          5.01           3.43           1.46           0.246
## 2 versicolor          5.94           2.77           4.26           1.33
## 3 virginica          6.59           2.97           5.55           2.03

```

Sometimes you don't want to loop to perform a specific (pre-defined) number of tasks. Sometimes you want to loop until an event occurs. For example, you might stop when the data point of interest is identified, or when a specific time is reached, or when until the user clicks "quit" (yes—you can make a GUI in R). In situations like this when the stopping point is not well-defined, you might want to use a while loop in R.

It's pretty straight-forward syntax:

```

while (condition) {
  do something
}

```

Here's a simple example that is essentially identical to a for loop:

```

i <- 1
while (i <= 5) {
  print(c("This is loop number", i))
  i <- i + 1
}

```

```

## [1] "This is loop number" "1"
## [1] "This is loop number" "2"
## [1] "This is loop number" "3"
## [1] "This is loop number" "4"
## [1] "This is loop number" "5"

```

Now let's try an example where a while loop makes more sense. The loop generates Gaussian random numbers with a mean of 0 and a standard deviation of 1. The loop quits when a number is generated that is more than 4 standard deviations from the mean. On average, this will occur about 1 in 10,000 attempts.

```

i <- 1
z <- 0

```

```
while (abs(z) < 4) { #abs means "absolute value" & takes care of neg or pos numbers
  z <- rnorm(1, 0, 1) #generates a single random number of mean = 0 and sd = 1
  i <- i + 1
}
print(paste("It took",i,"replicates to find a Gaussian random number more than",
            "4 standard deviations from the mean"))
```

```
## [1] "It took 1259 replicates to find a Gaussian random number more than 4 standard deviations fr
```

9.3.1 Exercise

- List all possible 3 card hands from a standard card deck with no jokers. Cards can be annotated with character vectors. For example the hearts would be `hearts <- c(paste0(c(2:10), "H"), "JH", "QH", "KH", "AH")`.

10 ggplot

So far, we've seen how to do some 'quick and dirty' plots with plotting functions like `hist()` and `plot()` which are built in to base R. The tidyverse has its own paradigm for creating graphics called `ggplot`. The advantage to using `ggplot` over base R functions is that the `ggplot` paradigm comes with many built in defaults to make your plots look nice without having to code too much customization. Overall, the degree to which you can customize `ggplot` graphics is less than in base R, but it's soo much easier. As we go through some examples, note how `ggplot` has automatically chosen colour schemes, scales, and axis labels for us, without us specifying any of this. Of course, we can override these, but having some usable defaults built in makes it very fast to produce nice plots.

10.1 What is ggplot?

The "gg" in "ggplot" stands for "grammar of graphics", and the basic idea is this: when you plot data, you are creating a visual representation of numeric or categorical information. The most basic example is a scatterplot; the position of a point on the x axis reflects one variable, and the position on the y axis reflects another variable. That works well for simple examples, but often we have a large number of parameters that we'd like to display. Ideally, we want a clear, flexible framework that maps arbitrary variables to arbitrary visual elements such as x position, y position, size, color, shape, transparency, etc. This would let us rapidly explore different ways of looking at our data to see what is the most helpful. `ggplot` provides this sort of framework, with a clean mapping of variables to output.

Let's illustrate this using one of R's built in data sets, `mpg` which has a variety of data about different models of cars:

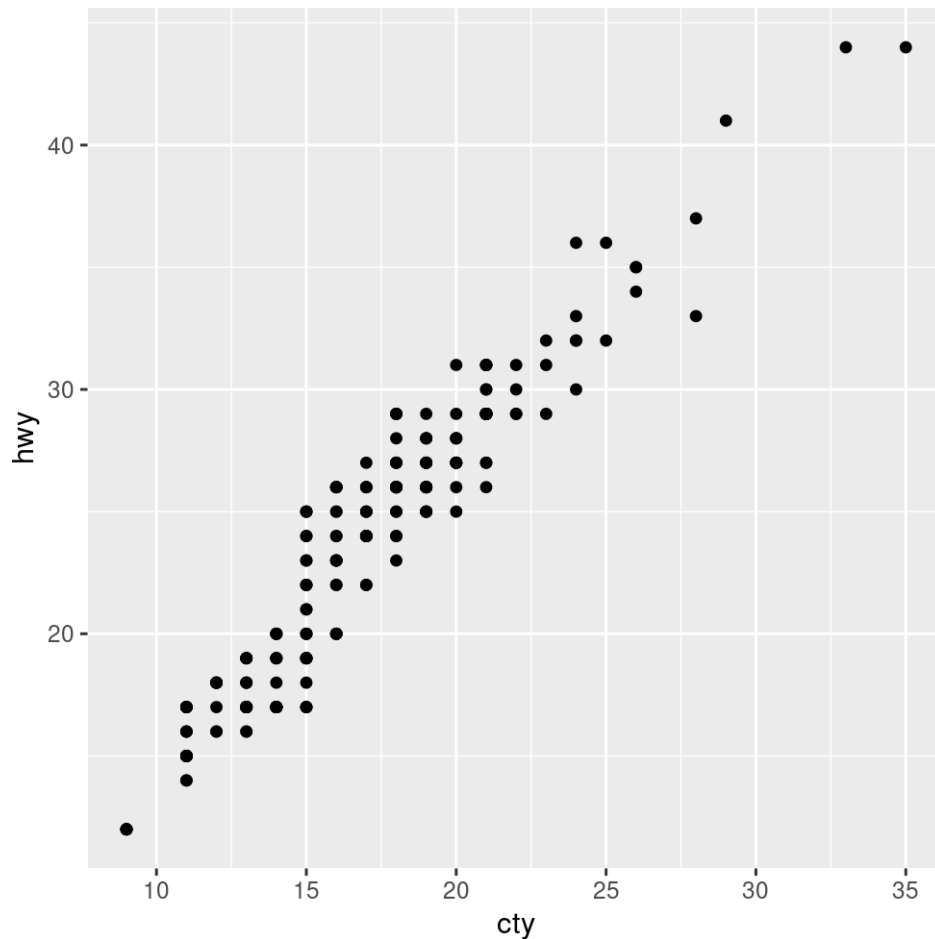
```
glimpse(mpg)
```

```
## Rows: 234
## Columns: 11
## $ manufacturer <chr> "audi", "audi", "audi", "audi", "audi", "audi", "audi", "~
## $ model        <chr> "a4", "a4", "a4", "a4", "a4", "a4", "a4", "a4 quattro", "~
## $ displ       <dbl> 1.8, 1.8, 2.0, 2.0, 2.8, 2.8, 3.1, 1.8, 1.8, 2.0, 2.0, 2.~
## $ year        <int> 1999, 1999, 2008, 2008, 1999, 1999, 2008, 1999, 1999, 200~
## $ cyl         <int> 4, 4, 4, 4, 6, 6, 6, 4, 4, 4, 4, 6, 6, 6, 6, 6, 8, 8, ~
## $ trans       <chr> "auto(l5)", "manual(m5)", "manual(m6)", "auto(av)", "auto~
## $ drv         <chr> "f", "f", "f", "f", "f", "f", "f", "4", "4", "4", "4", "4~
## $ cty         <int> 18, 21, 20, 21, 16, 18, 18, 18, 16, 20, 19, 15, 17, 17, 1~
## $ hwy         <int> 29, 29, 31, 30, 26, 26, 27, 26, 25, 28, 27, 25, 25, 25, 2~
## $ fl          <chr> "p", "p", "p", "p", "p", "p", "p", "p", "p", "p", "p", "p", "p~
## $ class       <chr> "compact", "compact", "compact", "compact", "compact", "c~
```

10.2 Scatterplots

We can start by plotting city versus highway fuel economy. `ggplot()` wants us to provide some data, a mapping of the data onto parameters, and a geometry with which to render that data.

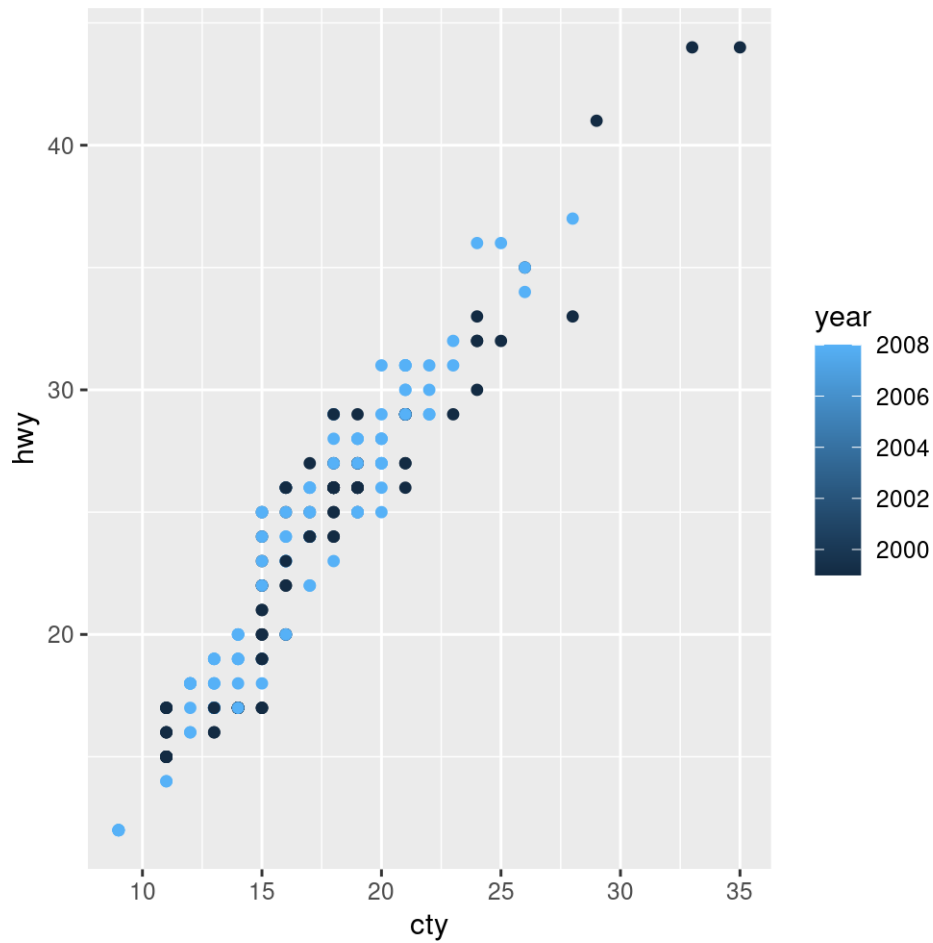
```
ggplot(mpg, aes(x = cty, y = hwy)) +  
  geom_point()
```



Notice that our `ggplot()` command had three parts: data, a set of “mapping” aesthetics (x and y in this case), and a way of rendering the geometry (`geom_point`). This doesn’t look like our ordinary function calls, but you can think of the + as saying “OK, add this geometry rendering to the plot that I just made”.

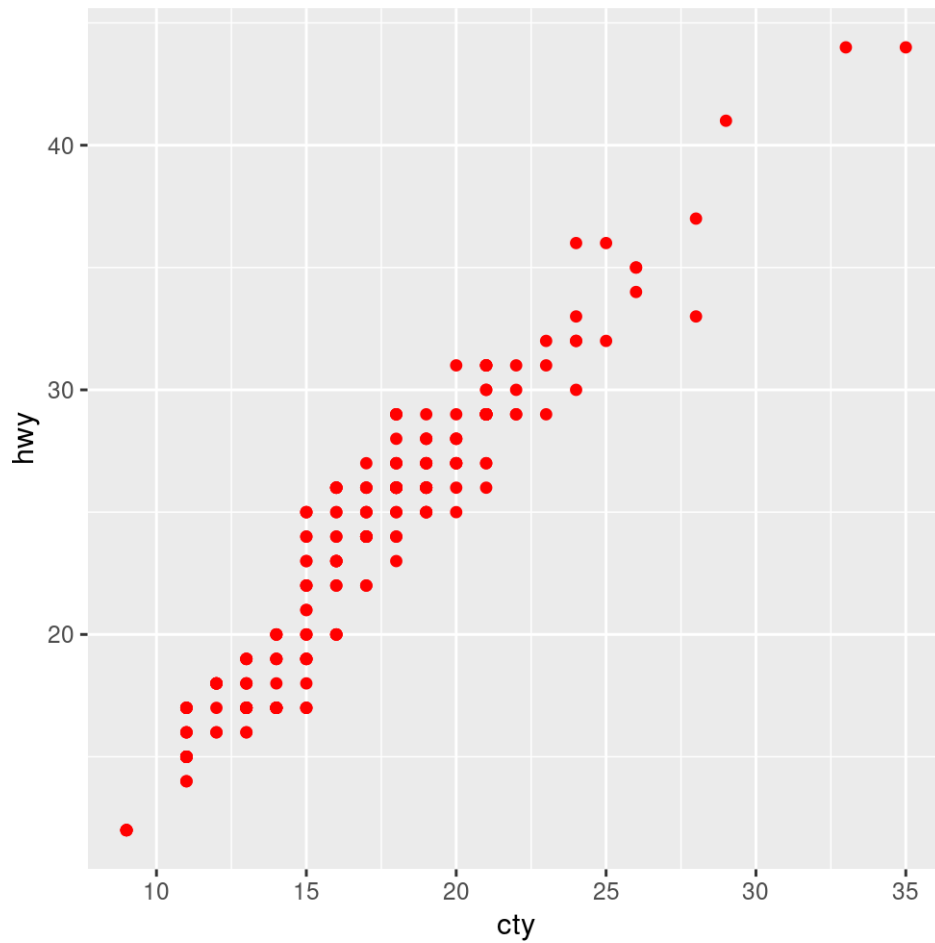
The “aesthetics” that are specified within the `aes()` call are where the real fun starts. The aesthetics for x and y can be specified in a global `aes()` call in the main `ggplot()` call, and further customization for colour, size, shape etc can be specified inside the geometry call. For example, let’s look at city vs highway fuel economy again, and map the year of the model to color.

```
ggplot(mpg, aes(x = cty, y = hwy)) +  
  geom_point(aes(colour = year))
```



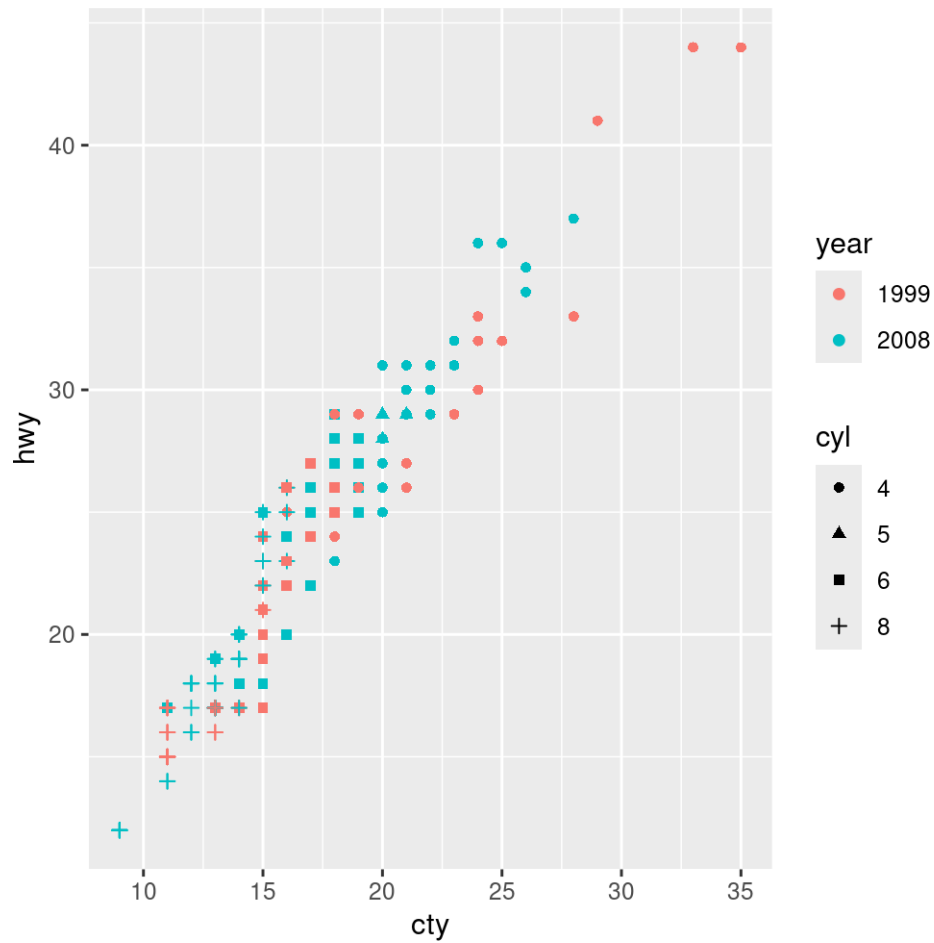
Notice how that is different than specifying colour *outside* of the `aes()` call:

```
ggplot(mpg, aes(x = cty, y = hwy)) +  
  geom_point(colour = "red")
```



Notice that `ggplot()` has made the incorrect inference that the vehicle year should be considered a continuous variable. We can fix this by explicitly making the car year number a factor. We can also combine multiple aesthetics:

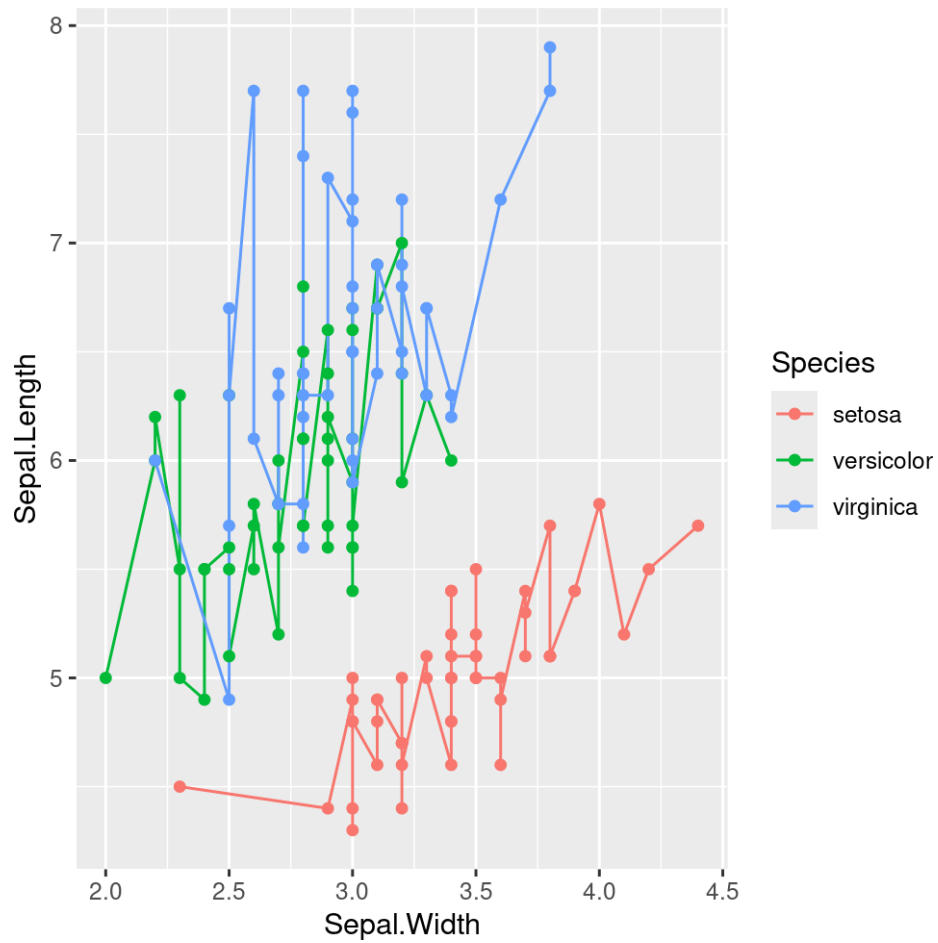
```
mpg %>%
  mutate(year = factor(year), cyl = factor(cyl)) %>%
  ggplot(aes(x = cty, y = hwy, col = year, shape = cyl)) +
  geom_point()
```



10.3 Line Plots

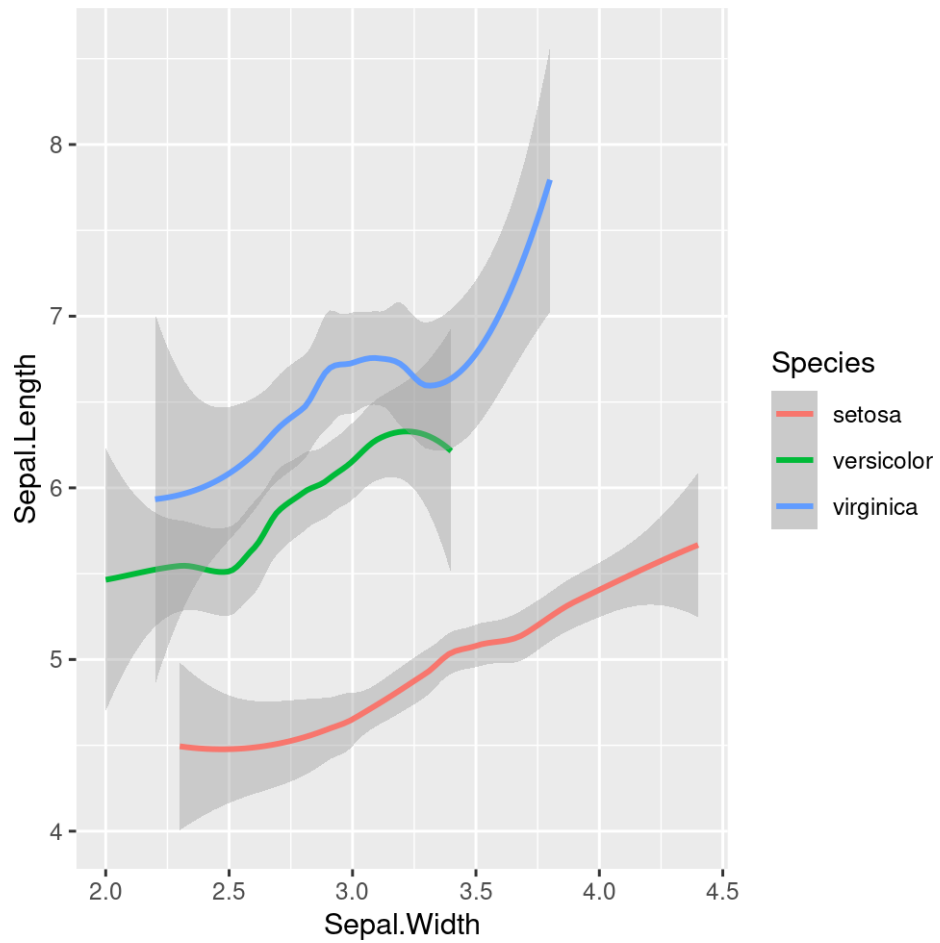
There are dozens of geometries at your disposal. Some other common useful ones are `geom_line` and `geom_smooth`.

```
iris %>%
  ggplot(aes(x = Sepal.Width, y = Sepal.Length, col = Species)) +
  geom_line() +
  geom_point()
```



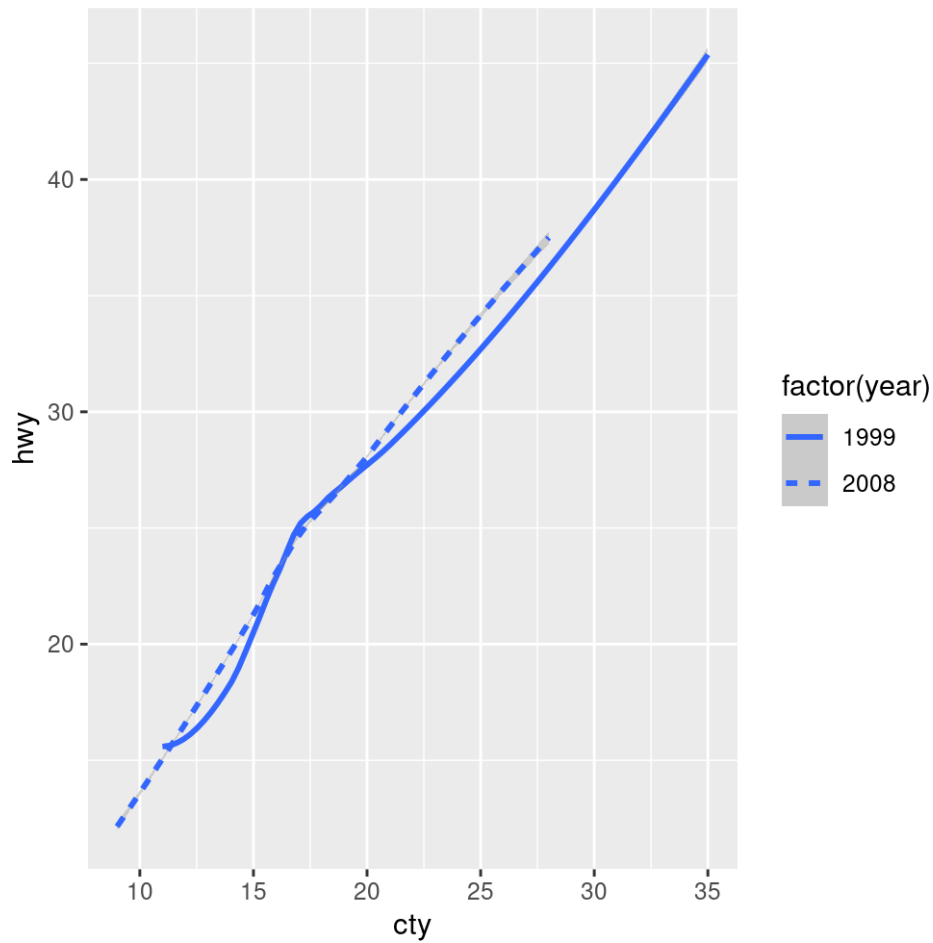
We can `geom_smooth()` a line if it looks too choppy:

```
iris %>%
  ggplot(aes(x = Sepal.Width, y = Sepal.Length, col = Species)) +
  geom_smooth()
```



Of note, we can map more than one aesthetic:

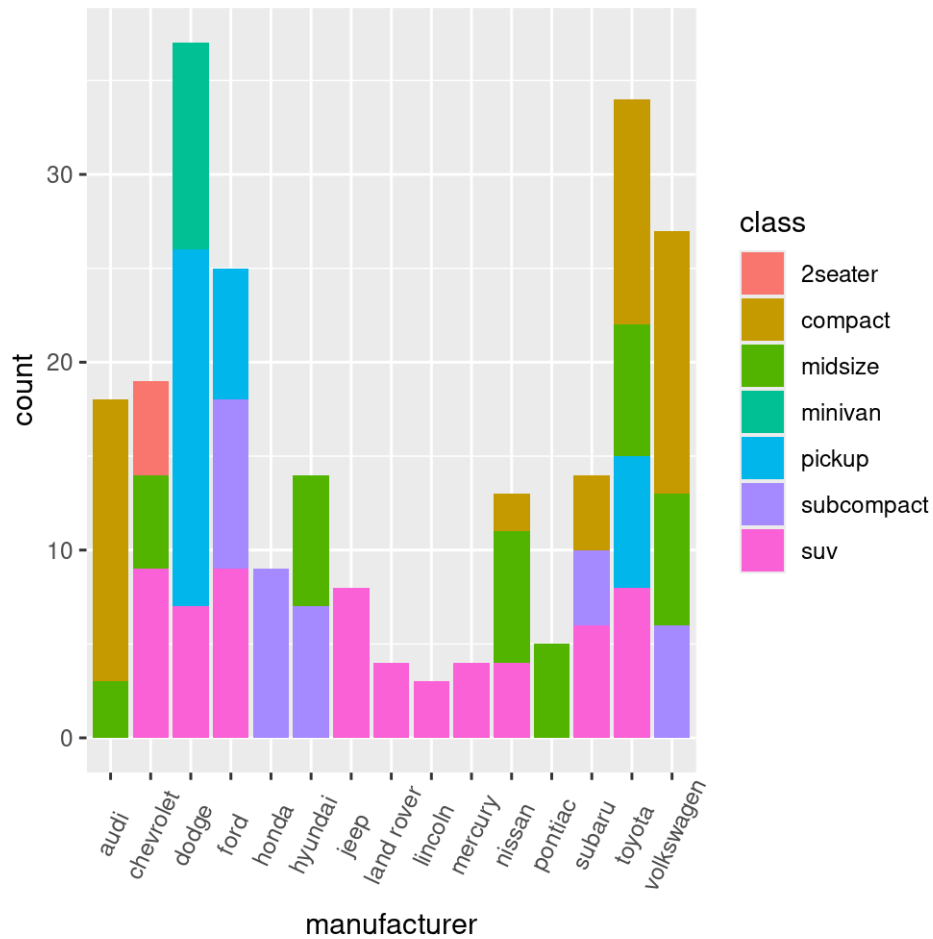
```
ggplot(mpg, aes(x = cty, y = hwy)) +  
  geom_smooth(aes(linetype = factor(year)), level = 0.2)
```



10.4 Bar Plots

`geom_bar` has built in stats, with the default being ‘count’, for which no y aesthetic needs to be specified, as the y axis will display the number of observations for each x axis value:

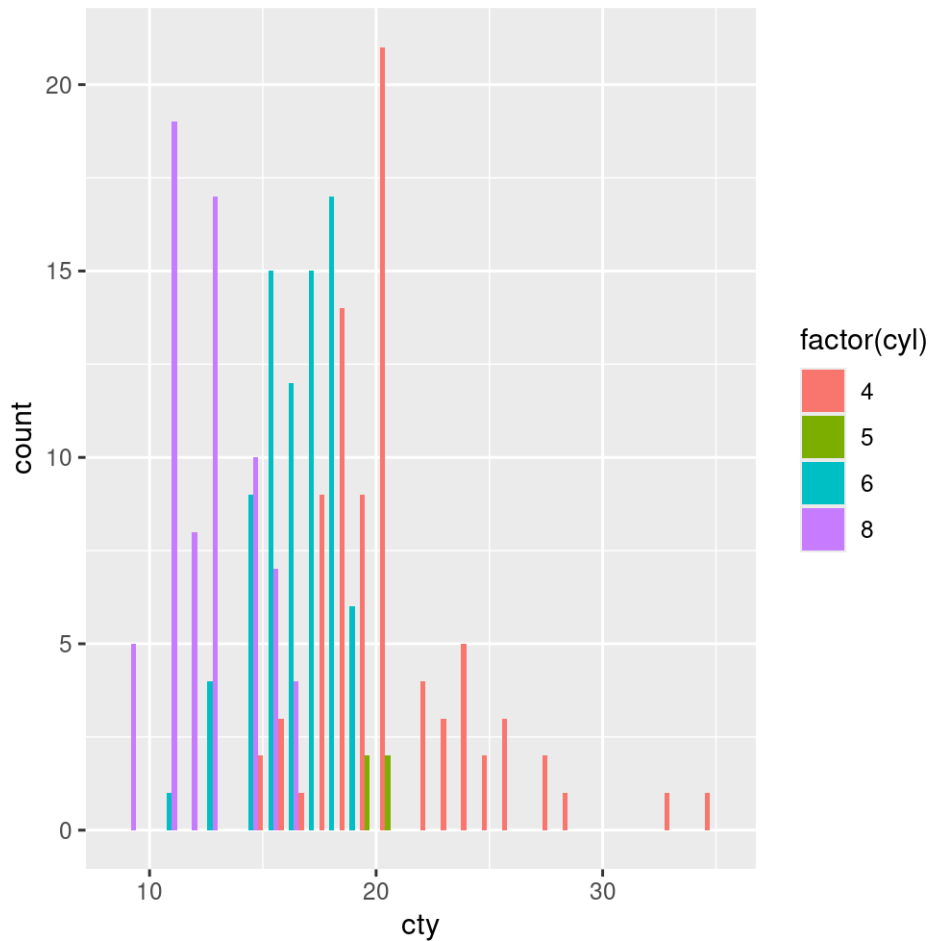
```
ggplot(mpg, aes(x = manufacturer)) +  
  geom_bar(aes(fill = class)) +  
  theme(axis.text.x = element_text(angle = 65, vjust = 0.6))
```



Note we used 'fill' here to colour the bars rather than 'colour' - what happens if you use 'colour' instead?

And of course, good old histograms get their own geom:

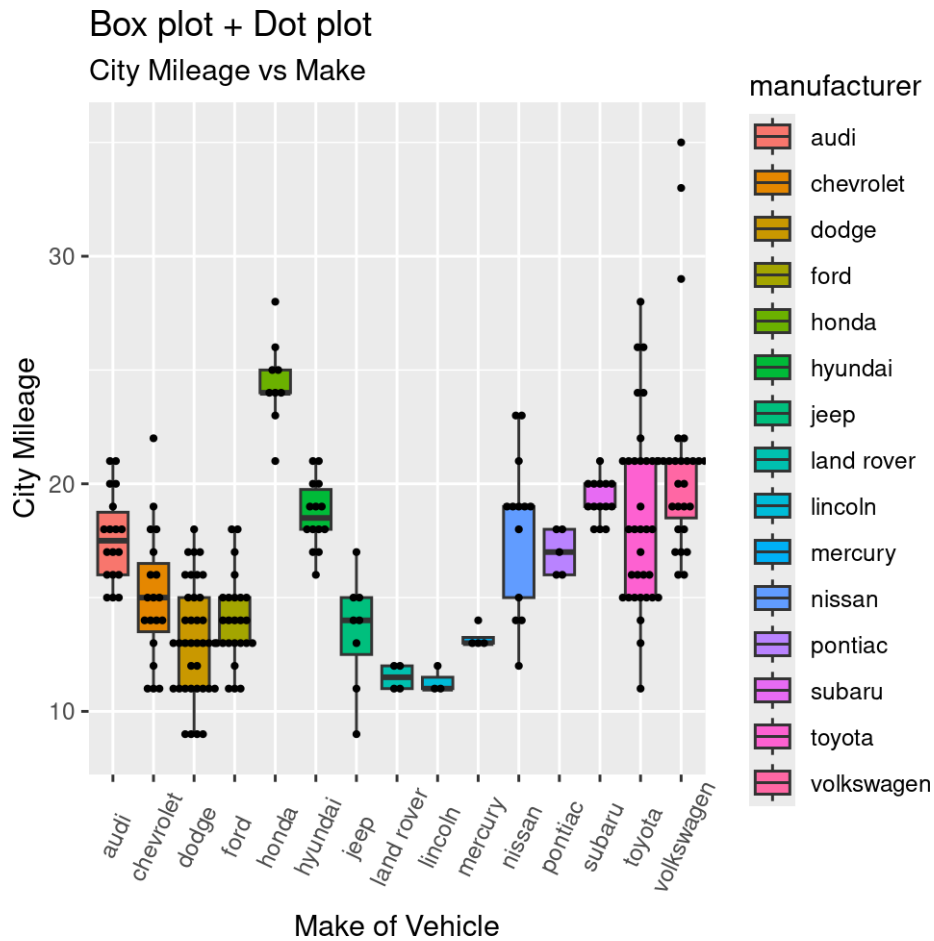
```
ggplot(mpg, aes(x = cty)) +  
  geom_histogram(aes(fill = factor(cyl)), position = "dodge")
```



10.5 And more!

`ggplot()` objects can be layered and layered upon, including multiple geoms, labels, custom scales, and more. Here's just one example:

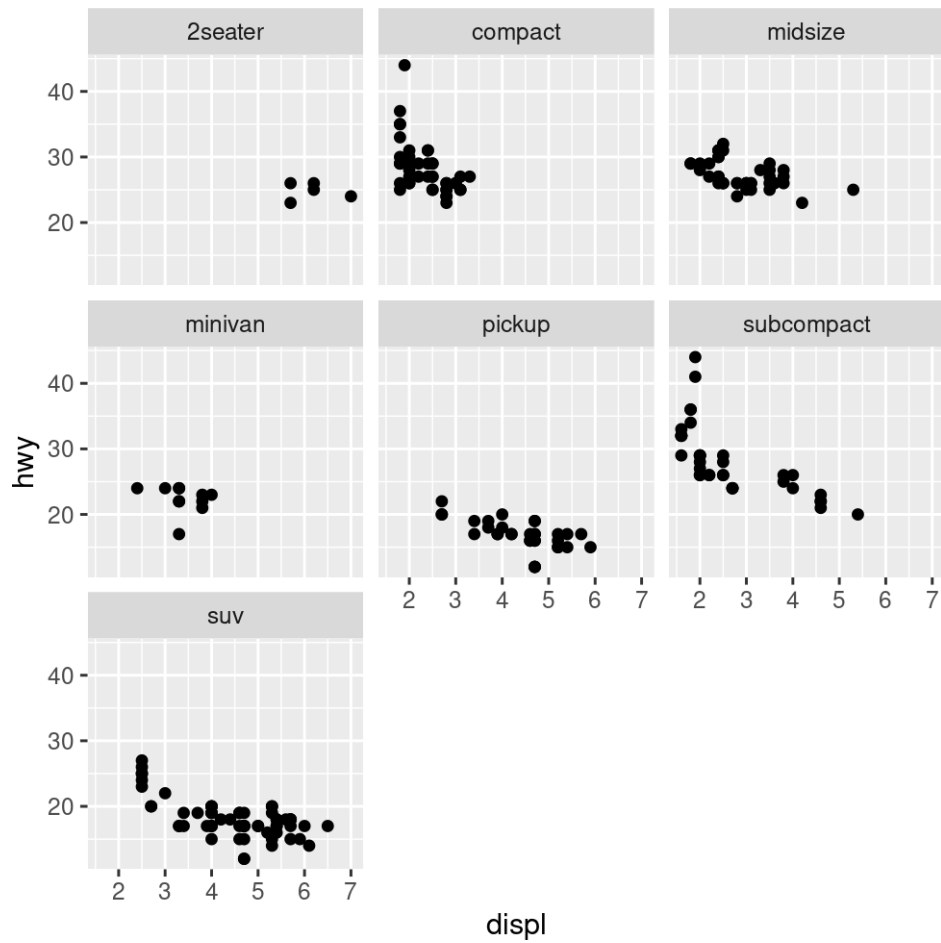
```
ggplot(mpg, aes(manufacturer, cty)) +
  geom_boxplot(aes(fill = manufacturer),
    outlier.shape = NA) +
  geom_dotplot(binaxis = 'y',
    stackdir = 'center',
    dotsize = .3,
    fill = "black",
    col = "black") +
  theme(axis.text.x = element_text(angle = 65, vjust = 0.6)) +
  labs(title="Box plot + Dot plot",
    subtitle="City Mileage vs Make",
    x = "Make of Vehicle",
    y = "City Mileage",
    legend = "")
```



10.6 Multipanel Plots

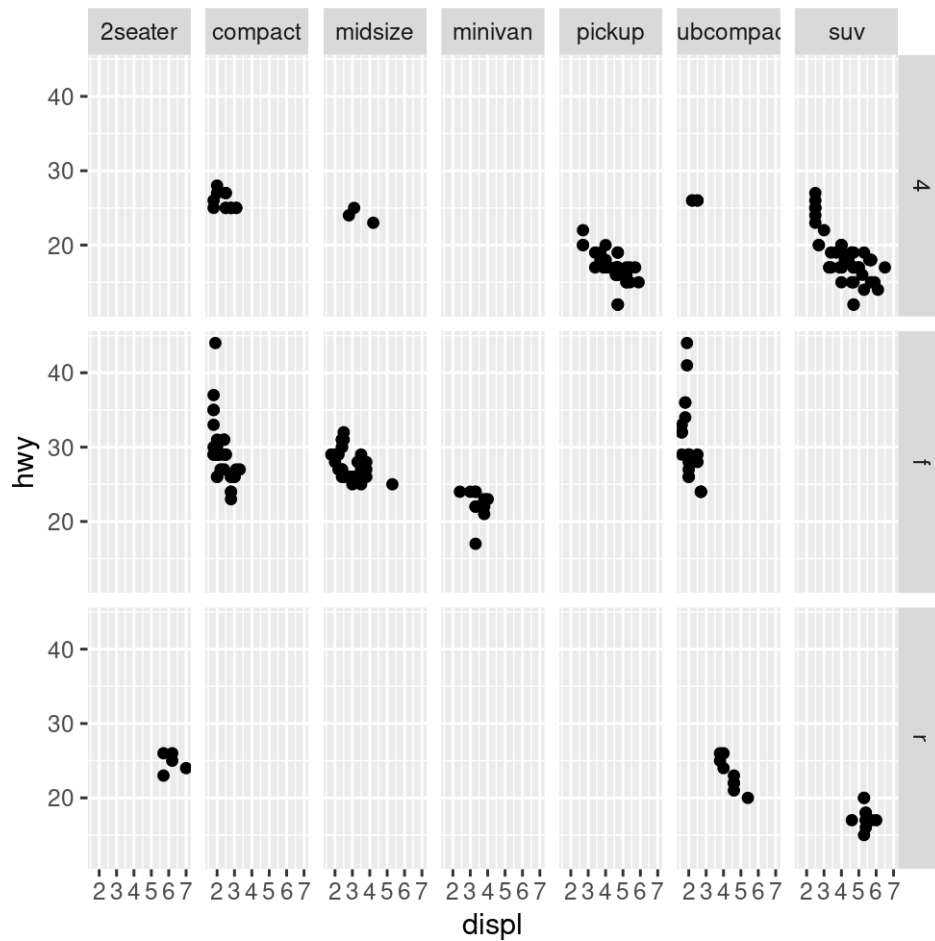
If you want to produce multi-panel plots that are based on a factor within your data set, this can be accomplished using the `facet_wrap()` function:

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() +  
  facet_wrap(~ class)
```



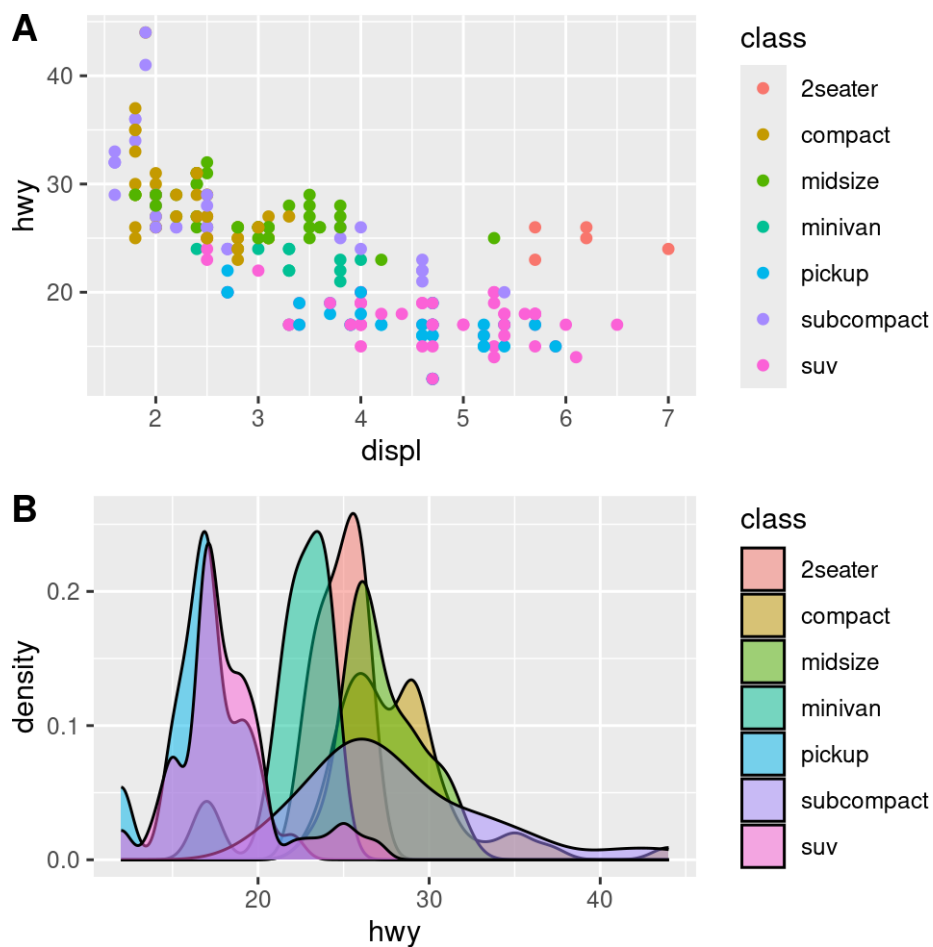
You can use `facet_grid()` if you want to produce all possible combinations of two factors in your data:

```
ggplot(mpg, aes(x = displ, y = hwy)) +  
  geom_point() +  
  facet_grid(drv~class)
```



And if you just want to glue images together that may not originate from a single `ggplot()` command, this can be achieved with the `plot_grid()` function from the `cowplot` package.

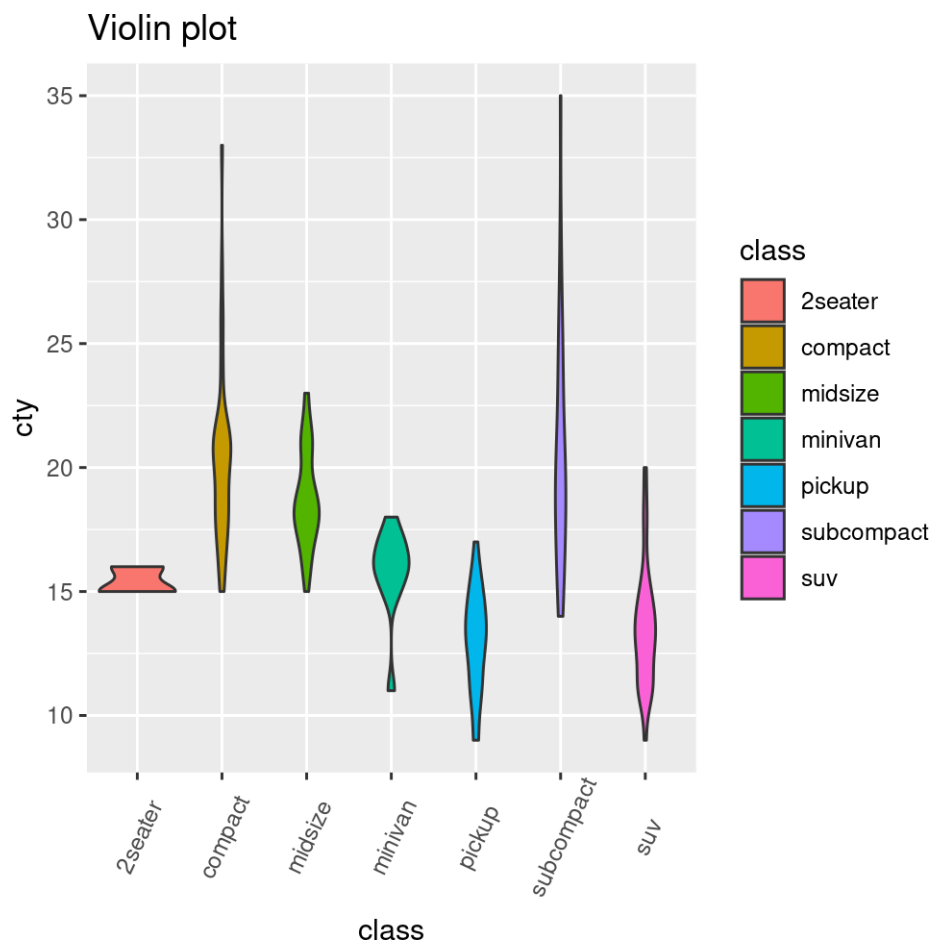
```
library(cowplot)
p1 <- ggplot(mpg, aes(x = displ, y = hwy, colour = class)) +
  geom_point()
p2 <- ggplot(mpg, aes(x = hwy, fill = class)) +
  geom_density(alpha = 0.5)
plot_grid(p1, p2, nrow=2, labels=c('A', 'B'))
```



There is so much more you can do with `ggplot()` that we won't have time to cover. If you are looking for more ways to display data and customize your plots, check out one of the many ggplot 'cheat sheets' available online. You can also start by going through the data visualization chapter of Hadley Wickham's book *R for Data Science* or see the *R Graphics Cookbook* by Winston Chan.

10.6.1 Exercise

- Using the `mpg` dataset or any of the other datasets we've used so far, spend some time playing around with various geoms and aesthetics. Remember you can type `?geom_smooth` or similar to bring up the documentation in RStudio. Can you figure out how to make a "violin" plot?



10.7 Saving a plot

We have a lot of flexibility about saving our plots. We can save in bmp, jpeg, png, tiff and in vector formats of svg, ps and pdf. This is very good.

Unfortunately, the syntax between Mac and Windows is a little different.

10.7.1 Saving as a jpeg , png, tiff or bmp.

Execute the following:

```
jpeg(filename = "violin_plot.jpg", width = 10, height = 10, units = "in", res = 200)
ggplot(mpg, aes(class, cty)) +
  geom_violin(aes(fill=class)) +
  labs(title="Violin plot") +
  theme(axis.text.x = element_text(angle=65, vjust=0.6))
dev.off()
```

On Macs this translates to:

```
quartz(file = "violin_plot.jpg", width = 10, height = 10, type = "jpg", dpi = 200)
ggplot(mpg, aes(class, cty)) +
  geom_violin(aes(fill=class)) +
  labs(title="Violin plot") +
  theme(axis.text.x = element_text(angle=65, vjust=0.6))
dev.off()
```

If you want output that is “vector” in nature:

```
postsript(file = "violin_plot.ps", width = 10, height = 10)
ggplot(mpg, aes(class, cty)) +
  geom_violin(aes(fill=class)) +
  labs(title="Violin plot") +
  theme(axis.text.x = element_text(angle=65, vjust=0.6))
dev.off()
```

You will notice that the aspect ratio needs adjustment for the ps output. You can do pdf also for example:

```
pdf(file = "violin_plot.pdf", width = 10, height = 10)
ggplot(mpg, aes(class, cty)) +
  geom_violin(aes(fill=class)) +
  labs(title="Violin plot") +
  theme(axis.text.x = element_text(angle=65, vjust=0.6))
dev.off()
```

11 File Operations

You are going to need to read files in, as you have done with `read.csv()`. You are going to need to write files—whether it is the statistical output or perhaps the content of a dataframe that you have produced through your efforts. Sometimes you need to remove files or move files too. All of these file operations are possible from within R. You can also both read and write xls or xlsx files, which we will touch on also. This stuff is dry but it is necessary.

What files are in the current working directory? This does the trick, as we have discussed.

```
list.files()
```

If you want the figures in a specific directory, you can put it in the parentheses as in `dir_ls(path = "Name_of_Subdirectory")`. You can also provide the “full path” of a directory too.

```
list.files(path = "/")
# should give you all the files in the C: directory in windows
# and the root directory in Linux and Mac OSX
```

Let’s suppose we want to see if a file is present—so that we don’t try to perform an operation on a non-existent file, or to ensure that some other event is complete. We can ask as follows:

```
file.exists("Data_Files/Potassium.csv")
```

```
## [1] TRUE
```

```
file.exists("Data_Files/Data_Set_3.1459.csv")
```

```
## [1] FALSE
```

We might want to create a directory and then move or copy some files into it.

```
dir.create("junk_here")

if (file.exists("junk_here")) {
  file.copy("Data_Files/Potassium.csv", "junk_here/bla.csv")
} else {
  print("Hey -- That directory did not get created")
}
```

Note that if you provide `file_copy()` a vector of file names, `file_copy()` will copy all the files in the vector.

11.0.1 Exercise

1. You have a number of .csv files used in this course in the directory “Data_Files”. Create a new directory called “Data_Files_Copy” and use `file_copy()` to copy all of the .csv files into the new “Data_Files_Copy” directory. Make sure you copy *only* the .csv files.
2. Rename the files created in (1) appending today’s date to the file name so that the filenames look like this name-yyyy-mm-dd.csv. Make sure you do this in a generic fashion so that it will work regardless of how many files there are and what their initial names are. Use `Sys.Date()` to get the date. Use the `gsub()` command to replace “.jpg” with “yyyy-mm-dd.jpg”.

11.1 Writing a data frame to a csv file

Suppose you have done a bunch of work and you want to write it to a file so that you can share it or open it in a spreadsheet program.

For the sake of showing you what we mean, let’s create a fake-o dataframe.

```
subjects <- 1:10
secret_identity <- c("Clark", "Peter", "Bruce",
                    "David", "Orin", "Dick",
                    "Diana", "Steve", "Tony", "Barry")
hero <- c("Superman", "Spiderman", "Batman",
         "Hulk", "Aquaman", "Robin the Boy Wonder",
         "Wonder Woman", "Captain America", "Iron Man", "Flash")
hero_data <- data.frame(subjects, secret_identity, hero)
write_csv(hero_data, "hero_data.csv")
```

Now go confirm that the file is there and open it in a spreadsheet program.

But what if you forgot one superhero and you needed to append? You can append the individual record to the existing file. Note that `col.names` is turned off and `append` is turned on.

```
subjects <- 1:10
secret_identity <- c("Clark", "Peter", "Bruce",
                    "David", "Orin", "Dick",
                    "Diana", "Steve", "Tony", "Barry")
hero <- c("Superman", "Spiderman", "Batman",
         "Hulk", "Aquaman", "Robin the Boy Wonder",
         "Wonder Woman", "Captain America", "Iron Man", "Flash")
hero_data <- data.frame(subjects, secret_identity, hero)
write_csv(hero_data, "hero_data.csv")

forgotten_hero <- data.frame(subjects = 11,
                             secret_identity = "Kara",
                             hero = "Supergirl")
#you have to create it in an object of 1 column and 3 rows
write_csv(forgotten_hero, "hero_data.csv", append = TRUE)
```

Using this technique, you can add data to a file from a loop as the data is found to meet specific predefined criteria. For example, a resident and DH pulled all the hyponatremia occurrences from the LIS in 2014 and then looked for samples showing correction of Na^+ that is clinically too rapid (> 8 mmol/L in less than 24h). As these records were found, they were written to a file.

11.2 Writing xlsx files

Writing xlsx files with multiple sheets is quite easy also using the package `xlsx`. Note that there are multiple packages that can do this. You can also create fancy Excel files with formatting, bold, colored fonts and fill colors. This would only be of interest if you were writing automated reports for those folks for whom the appearance matters more than the content.

```
install.packages("openxlsx")

library(openxlsx)
hero.list <- list(Main_Heros = hero_data,
                  Forgotten_Heros = forgotten_hero)
write.xlsx(hero.list, file = "hero_data.xlsx", row.names = FALSE)
```

The same package allows you to read in a specific sheet from an xlsx file starting and finishing at a specific row.

12 Projects

12.1 Option 1: Lab Utilization Investigation

This file contains some very nice clean data about lab tests at a *totally fictional* hospital:

```
lab.data <- read.csv("Data_Files/lab_data.csv")
glimpse(lab.data)

## Rows: 54,406
## Columns: 14
## $ ID                <int> 3945, 3945, 3222, 2489, 1423, 1423, 1423, 1423, 142~
## $ SAMPLE            <int> 15416, 15416, 8687, 10888, 8097, 8097, 8097, 8097, ~
## $ COLLECTION_DATE   <chr> "2018-10-12T08:05:00Z", "2018-10-12T08:05:00Z", "20~
## $ ORDERED_DATE      <chr> "2018-10-12T07:53:00Z", "2018-10-12T07:53:00Z", "20~
## $ RESULT_DATE       <chr> "2018-10-12T08:25:58Z", "2018-10-12T08:33:38Z", "20~
## $ RECEIVED_DATE     <chr> "2018-10-12T08:13:00Z", "2018-10-12T08:13:00Z", "20~
## $ SCHEDULED_DATE    <chr> "2018-10-12T07:53:00Z", "2018-10-12T07:53:00Z", "20~
## $ BATTERY           <chr> "Complete Blood Count(CBC)", "INR PTT Battery(COAGB~
## $ TEST              <chr> "Hemoglobin(HB)", "INR(INR)", "Hemoglobin(HB)", "He~
## $ ABNORMAL_FLAG     <chr> "L", NA, NA, NA, NA, "L", NA, NA, NA, NA, "L", "H", ~
## $ RESULT            <dbl> 111.93, 1.04, 205.99, 126.01, 126.95, 0.82, 53.98, ~
## $ LOCATION_TYPE     <chr> "Inpatient", "Inpatient", "Inpatient", "Inpatient", ~
## $ ORDERING_PHYSICIAN <chr> "Leach, Mario", "Leach, Mario", "al-Rehman, Muthann~
## $ PRIORITY          <chr> "S", "S", "TD", "S", "S", "S", "S", "S", "S", "S", ~
```

Load it into your R session and see if you can produce a report on some KPIs (key performance indicators) for the lab:

- Start by surveying your data - what time period does it cover? How many tests do you have information on?
- What is the median and 90th percentile for Scheduled to Result turn around time for each test?
- Break it up by location type and create a nice plot (perhaps boxplot or violin plot or time-series plots) of TAT for the Emergency room versus Inpatients.
- Are there certain hours of the day when the turn around time suffers? Days of the week?

Then see if you can find some utilization insights:

- From what location type do the bulk of orders for each test originate?
- Which doctors order the most tests overall? Which doctors order the most tests per patient seen?

- How many tests are ordered on each patient?

12.2 Option 2: Clean Up Some Research Data – Bad Medicine

This is raw data from a study published on a familial hypercholesterolemia cohort from 2005. The birthdates are not real. The data were hand-transcribed by a resident into Excel.

12.2.1 Birthday Blues

If you would rather jump straight to analyzing cleaned data, run the solution code in the .Rmd file and then jump down to the next section.

- Read in the xlsx file entitled “badly_entered_data.xlsx”.
- Look at the DOB format using `head()` or `glimpse()`
- Parse the birthdays using the appropriate lubridate function and append them as a new column.
- What does the error message mean?
- Have a look at what lubridate has done. Why did this happen? How could this be avoided? Can you suggest a fix? Consider `str_c()` or `paste()`.
- Perform the parsing again. What has happened this time?
- Determine what is still wrong with the problematic birthdates.
- Can you come up with a programmatic way of salvaging the data other than fixing it by hand?

12.2.2 Descriptive Analysis

- In this data set 1 codes as Male and 2 codes as Female. How many males and females are there?
- If the study analysis was performed on 30-Aug-2003, calculate the ages of the patients in years on that date and append it as a new column.
- Determine the median and IQR of the patients in this study by sex.
- Is there a significant difference in the distribution of the ages of males and females? You can look at the functions `t.test()` and `wilcox.test()` as appropriate.
- Compare the distribution of cholesterol results between males and females and determine if there is a significant difference.
- Calculate the LDL-cholesterol and append as a new column.

12.2.3 A Physiology Question

There is a body of literature that says that lipoprotein(a) is unrelated to LDL cholesterol concentration because it is regulated through an independent pathway.

- Produce a correlation plot of Lp(a) versus LDL-C.
- Is there a statistically significant correlation?
- What is the regression equation of Lp(a) as a function of LDL-C? Look at `lm()`
- Does the data support or refute the traditional claim?

12.3 Option 3: Flat File Formatting

We are going to take flat file export data from an SCIEX API 5000 mass spectrometer and reformat the flat file for upload to the SunQuest lab information system. The process will be nearly identical for any instrument and LIS combination. The file you want to import is called `API_Flat_File.csv`. Before you do, open the file in a text editor or Excel. You will notice that the first 26 lines contain useless information, which you want to exclude when you read the file in. In fact, if you don’t exclude them, R won’t interpret the shape of the dataframe properly and this will cause confusion and delay, as Sir Topham Hatt says. The other thing to note is that this is a tab-delimited text file, not a comma-delimited file. So, `sep = “\t”` is a required parameter for `read_delim()`, not `sep = “,”`. Have a look at `help(read_delim)` to learn how to exclude the initial lines of a file—you want to look at the example of the skip option.

You want to pull out three relevant columns, which are named “Sample Name”, “Analyte Peak Name” and “Calculated Concentration (pmol/L).” It is better to address them by their name rather than their number because changes in AB SCIEX software could change which column number you are pulling... but not likely the name. Build a new dataframe containing these three columns.

This new dataframe contains extraneous information. You only need to upload the results of the quantifier ion (“Aldo 1”), not the qualifier ion (“Aldo 2”). Use `str_which()` to identify the rows containing “Aldo 1” data. Filter out the rows with “Aldo 1” data and assign this to a new dataframe.

Your dataframe still contains stuff that the LIS cannot digest. The patient samples have “Sample Name” in the format E followed by 10 integer digits. We only want these rows. Use `str_which()` to identify the rows, filter them out, and assign them to a new variable. In this simple example, it is sufficient to search based on the pattern “E”, but in general this would not be a good idea. In general you would use a regular expression to look for the pattern of E followed by 10 repetitions of the pattern 0-9. The search term, in this case, would be `^E[0-9]{10}` (Starts with E followed by 0-9 exactly 10 times). Filter out the patient data and assign to a new dataframe.

Use `str()` to examine the “Calculated Concentration” column. You will see that we have a problem. It is not numeric because to the undetectable results which come across as “No Peak”. We need to force this column to be numeric while assigning a concentration of 0 to the “No Peak” results. Sometimes the instrument sends other non-numeric terms over for undetectable results (like “< 0”). So, the safest thing to do is to convert the whole column to numeric and then replace the NA results with 0.

If you do not convert to character first, terrible things will befall you ... you will create results corresponding to the integer of the level R assigned to the column when it was read it. Oh, the horror. Apply `as.numeric(as.character())` to the Calculated.Concentration column to generate numeric data, assigning it to a temporary vector. This vector will contain NA results. Replace the NAs with 0 in this manner:

```
temp <- replace_na(temp,0)
```

Now overwrite the Calculated Concentration (pmol/L) field of your dataframe with the numeric data in your temp vector. Now we are almost done.

SunQuest wants its upload format like this:

```
API,E2660178393,ALD,230
```

```
API,E2660172438,ALD,95
```

```
API,E2660174731,ALD,634
```

```
API,E2660174643,ALD,217
```

API is the instrument name, the E-numbers are the patient IDs, ALD is the testcode and the numbers afterward are the results. Two of these columns are built already in your dataframe. Build the other two with `rep("API",n)` and `rep("ALD",n)` where `n` is the number of rows. Build a final dataframe corresponding to the required format (immediately above) and write it to a file with a comma as separator. Name the file “processed_flat_file.txt” so that when you click on it, it does not open in Excel, only in a text editor.

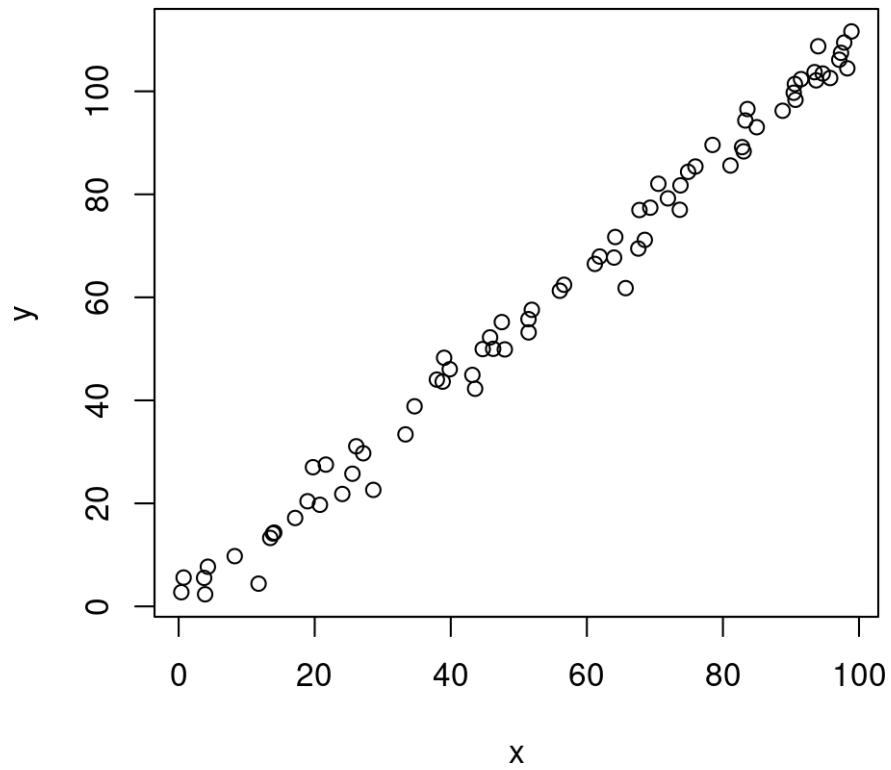
13 Appendix: Old Skool Base R Plotting

We are going to learn how to make a very high quality publication-read figures that are completely customizable and in any image format. While the techniques described in this lesson are not as powerful in and of themselves than those of the tidyverse, they are far more customizable.

```
x <- runif(75, 0, 100) # produces 75 random numbers between 0 and 100.
y <- 1.1 * x + rnorm(75,0,3) #add some bias and some scatter
```

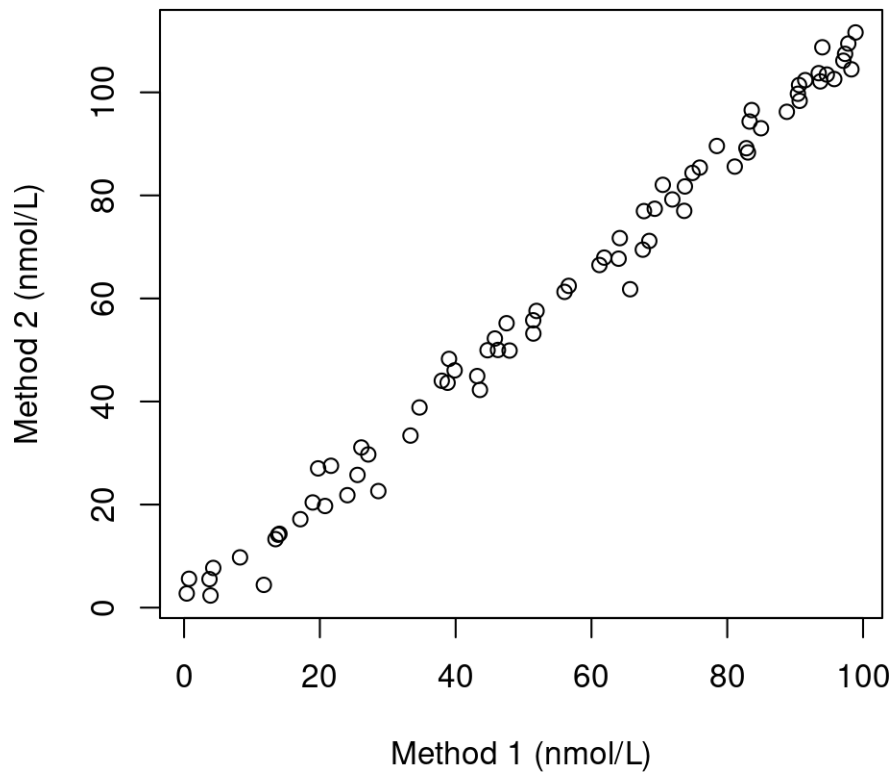
The most rudimentary default plot is created by

```
plot(x,y)
```



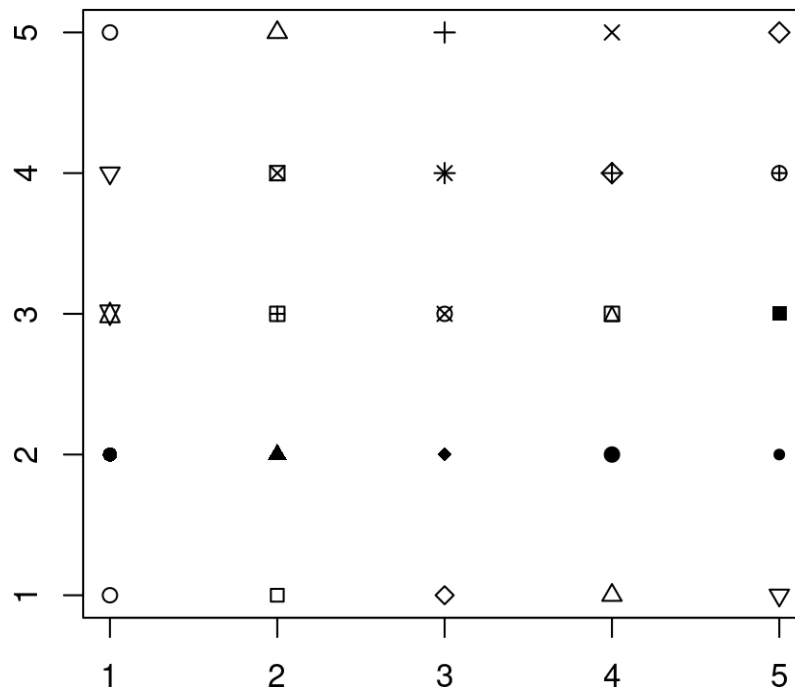
First, we will want to make better axis labels. This is easy. It is accomplished, for example, by adding `xlab = "Method 1 (nmol/L)"` to the x-axis and corresponding text to the y-axis.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)")
```



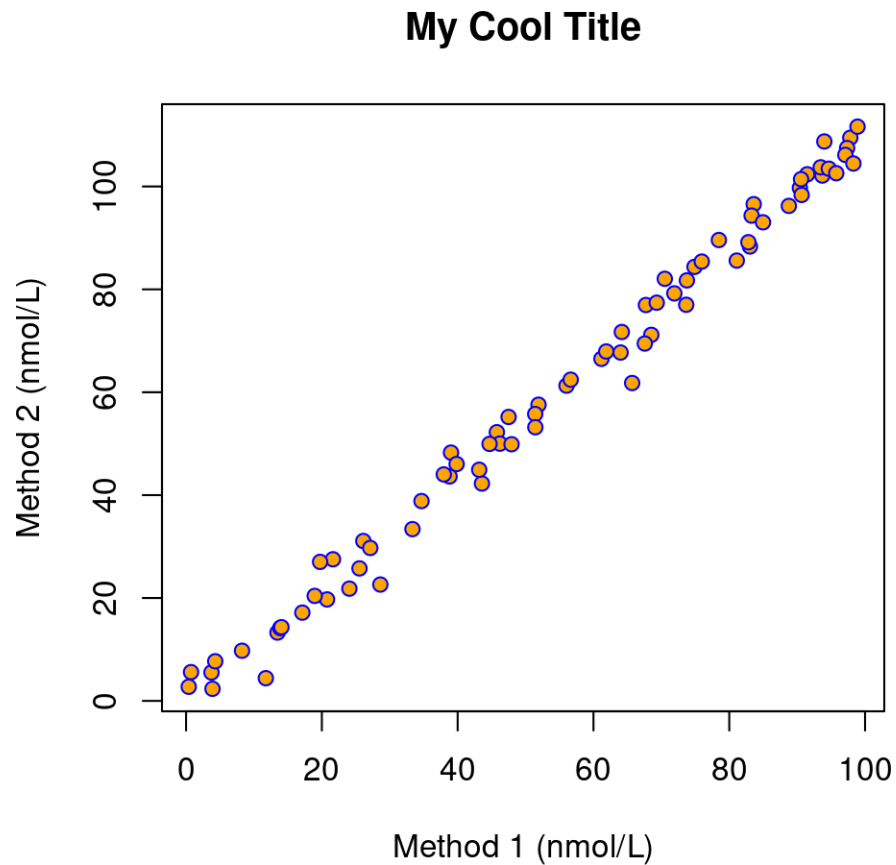
The title can be altered with the parameter `main = "My Cool Title"`.

```
plot(rep((1:5),5),c(rep(5,5),rep(4,5),rep(3,5),rep(2,5),rep(1,5)),pch = 1:25,
     xlab = "",ylab = "")
```



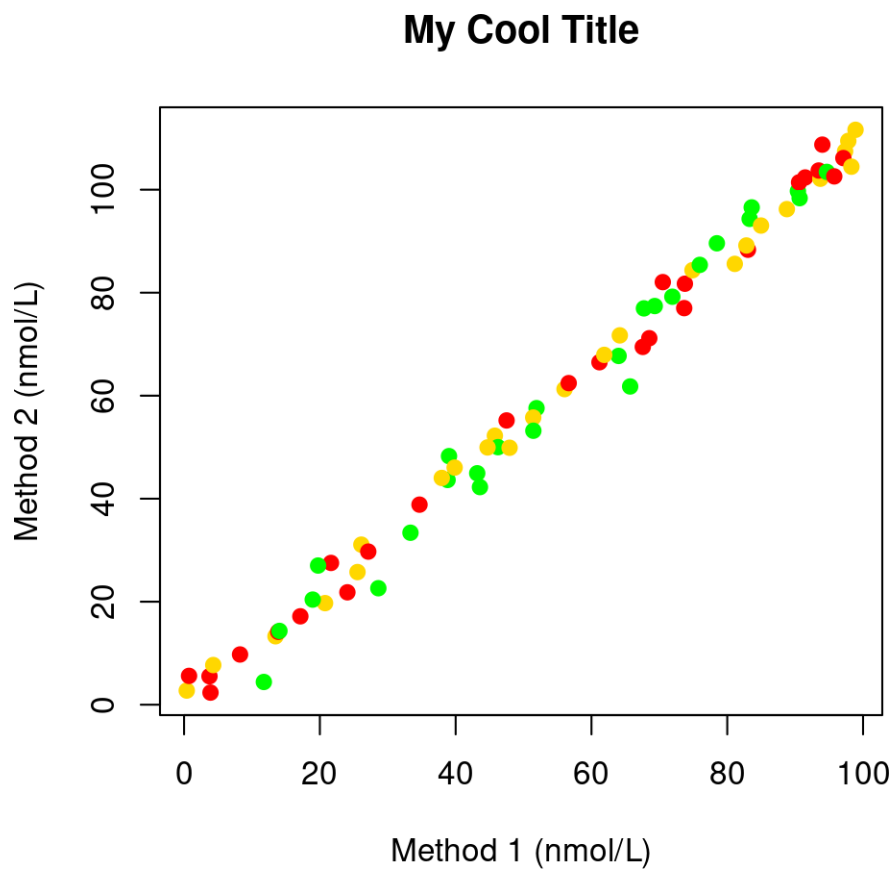
The parameter that alters the point character is called `pch` and you can set its value between 1 and 25. The colour of the points is set with `col = "the color I want"`. Characters 21-25 have fill colors in addition to outline colours and these can be set with the parameter `bg`.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 21, col = "blue", bg = "orange")
```



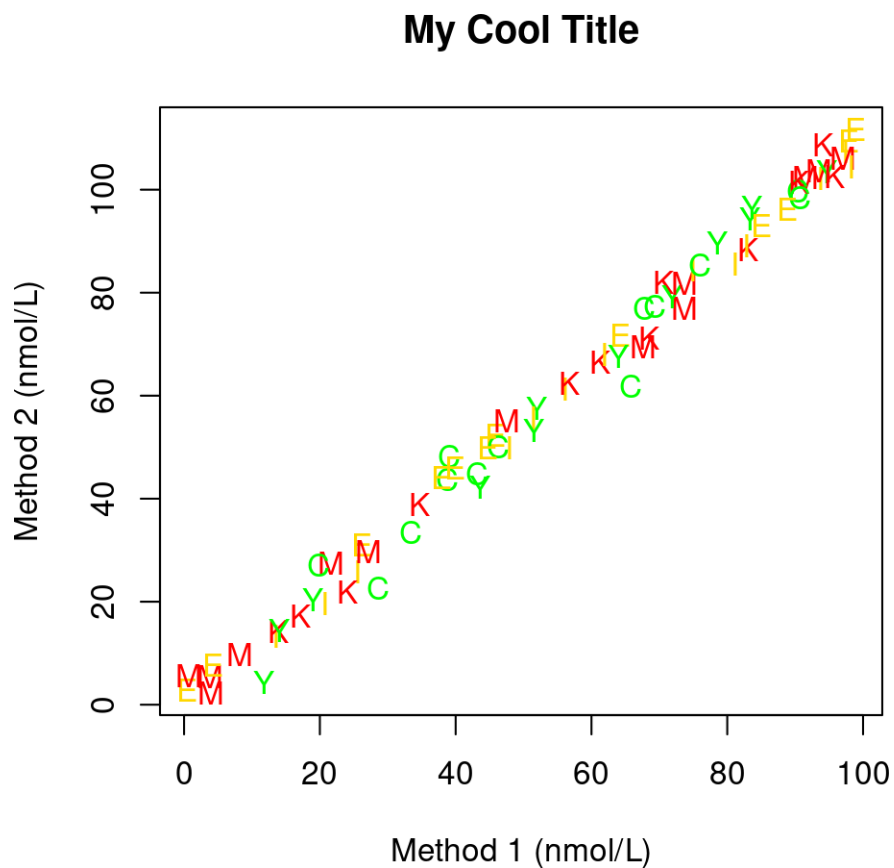
Your point colors can be in a vector if you want and the colours will cycle through the vector:

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 19, col = c("red", "gold", "green") )
```



In the event that you want the pch to be a letter... feel free!

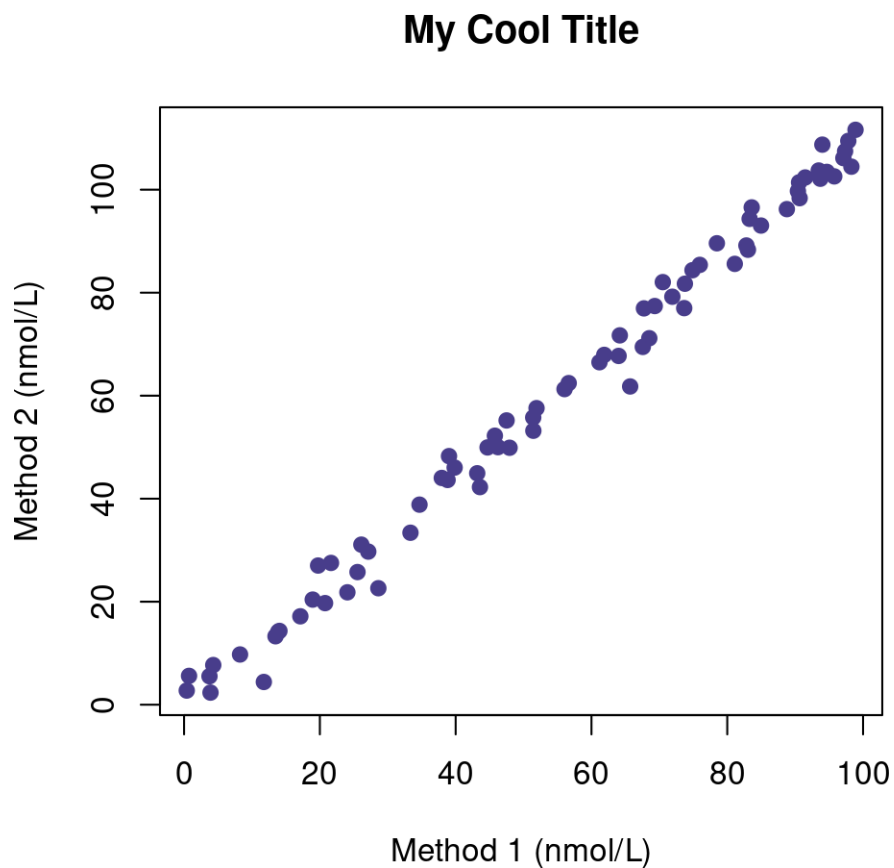
```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = c("M","I","C","K","E","Y"), col = c("red", "gold","green") )
```



You can see a full chart of R's colours here: research.stowers-institute.org/efg/R/Color/Chart/ColorChart.pdf

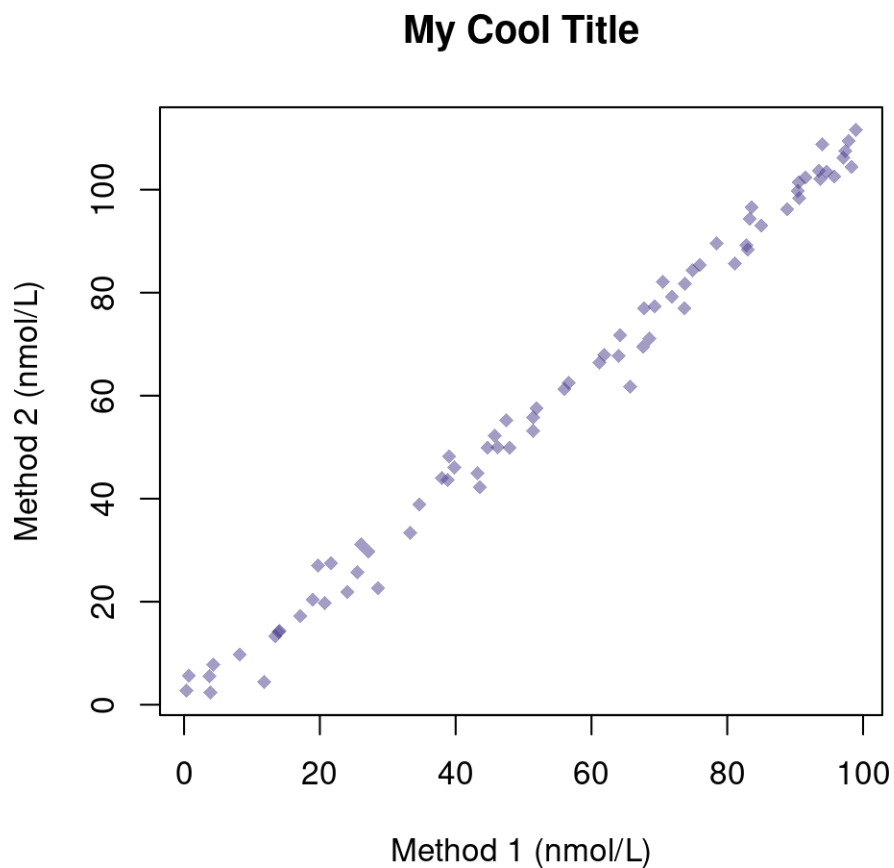
You can address colours using their hexadecimal code if you want.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",
     pch = 19, col = "#483D8B") #slateblue
```



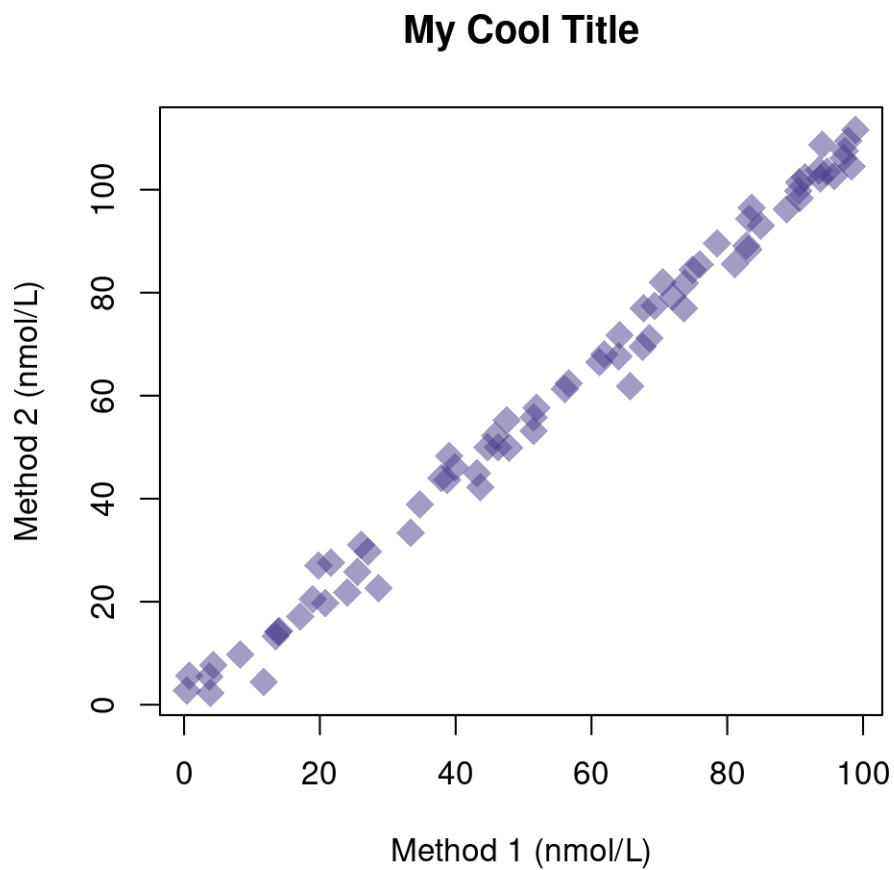
Another cool thing you can do is make the points semitransparent by appending a number in hexadecimal after the hex colour code from 00 to FF (256). A 00 makes the point invisible and FF makes it full transparent. A value of 80 makes a nice semitransparent look for high density plots.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 18, col = "#483D8B80") #slateblue
```



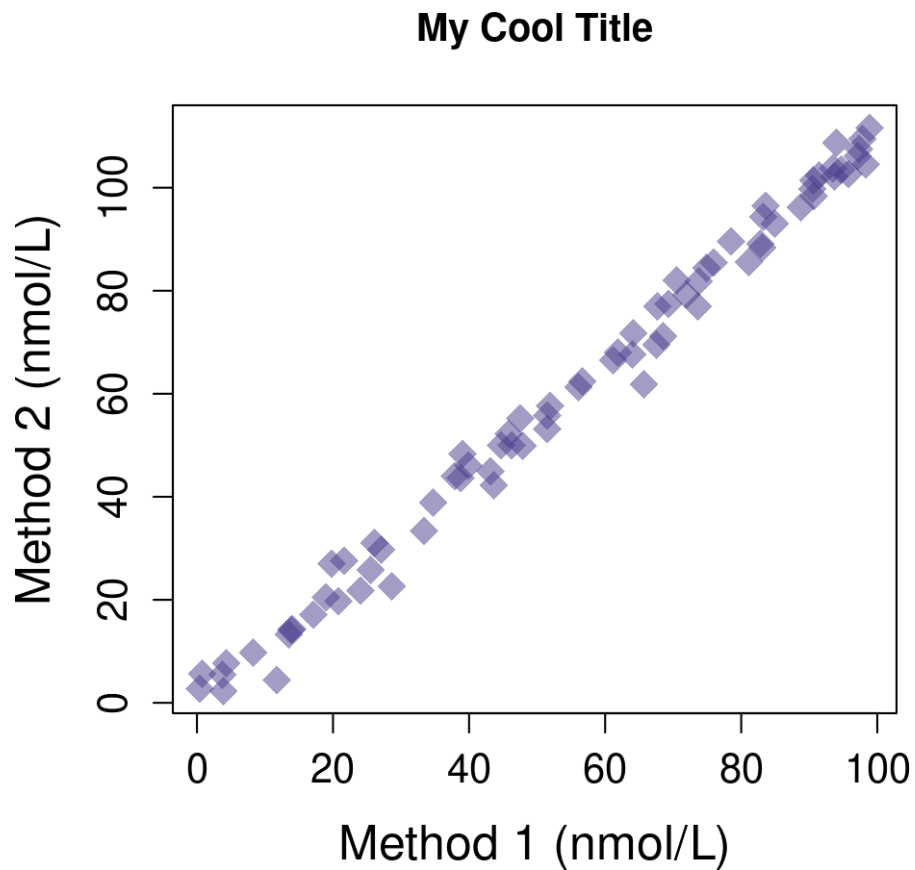
You can make your points bigger with the parameter `cex`. A value of 1 is default. 2 is double sized and 0.5 is half-sized etc.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 18, col = "#483D8B80", cex = 2) #slateblue
```



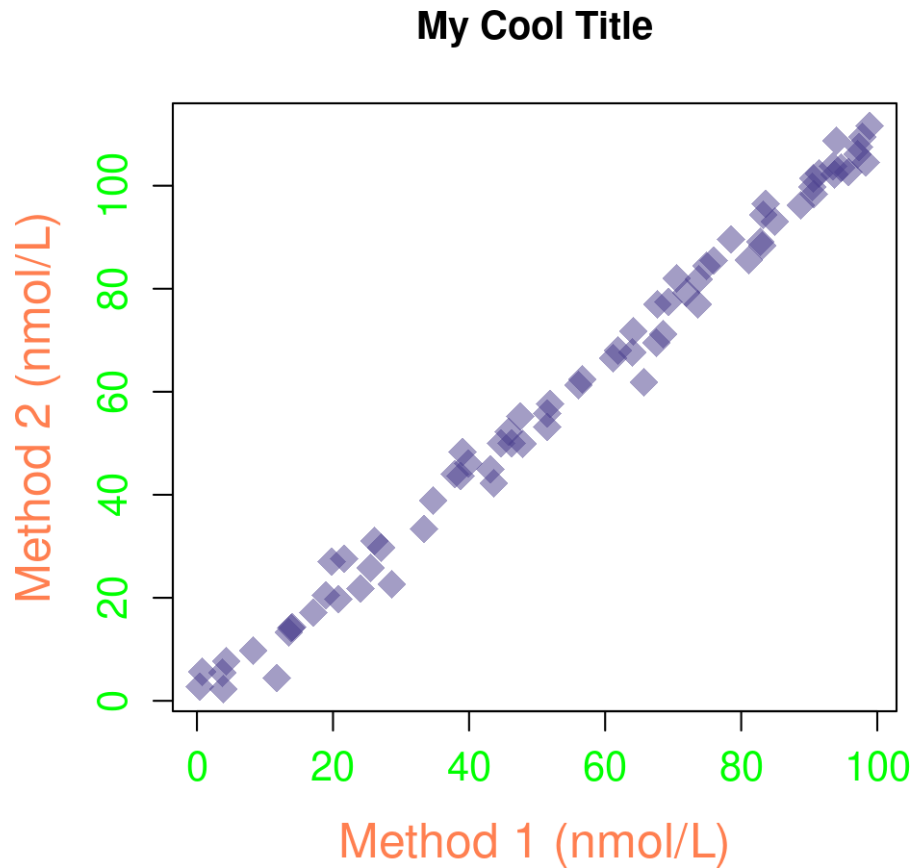
The size of the axis annotation is adjusted by `cex.axis` and the axis labels with `cex.lab`

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 18, col = "#483D8B80", cex = 2, cex.axis = 1.25, cex.lab = 1.5) #slateblue
```



Not surprisingly, there are `axis.col` and `lab.col` parameters to.

```
plot(x,y,xlab = "Method 1 (nmol/L)", ylab = "Method 2 (nmol/L)", main = "My Cool Title",  
     pch = 18, col = "#483D8B80", cex = 2, cex.axis = 1.25, cex.lab = 1.5, col.axis = "green",  
     col.lab = "coral") #slateblue
```



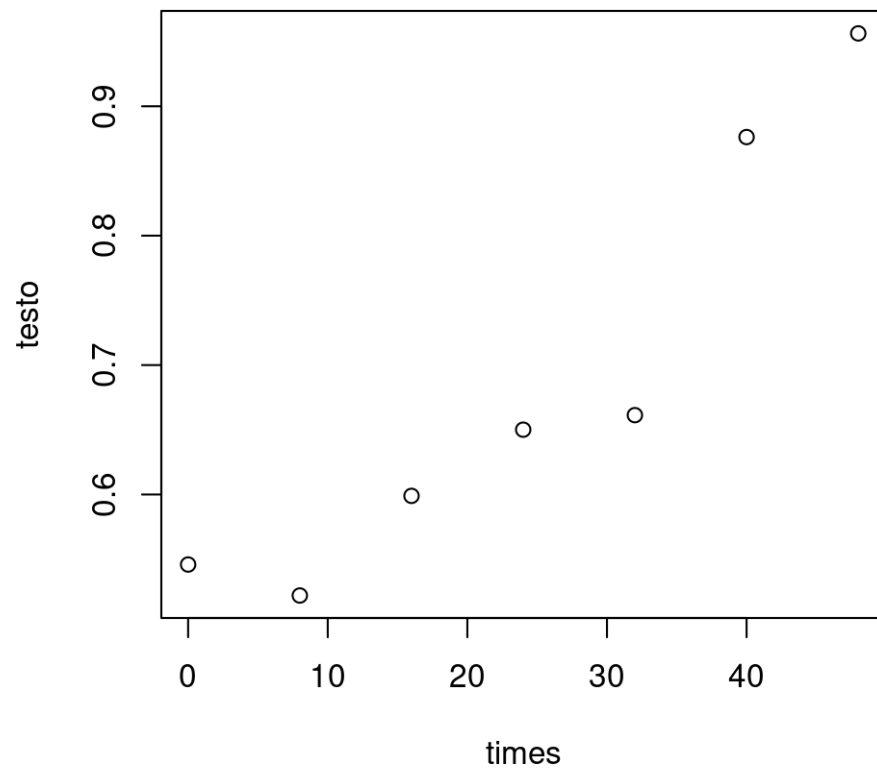
For now we will stop, but you get the picture that all plot parameters are adjustable.

13.1 Connecting the dots

In other kinds of plots, we might want our dots connected—like in a stability study perhaps.

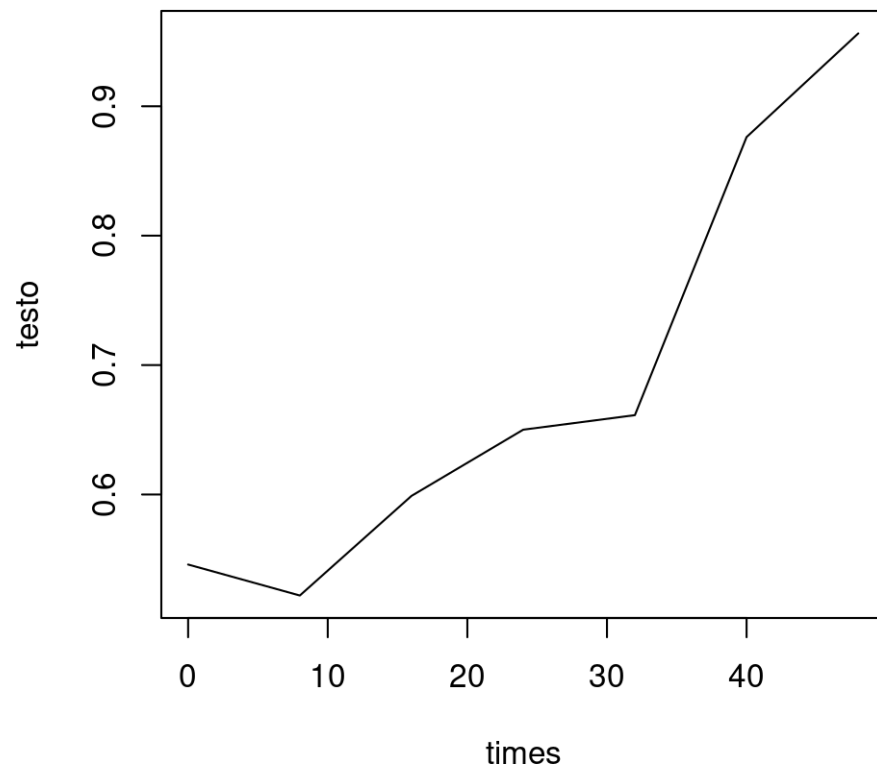
Suppose you are looking at how testosterone increases with time exposed to the gel in a gel separator tube. This is mock data, but the phenomenon is real.

```
times <- seq(from = 0, to = 48, by = 8)
testo <- seq(from = 0.5, to = 0.9, length.out = 7) + rnorm(7,0,0.05)
plot(times,testo)
```



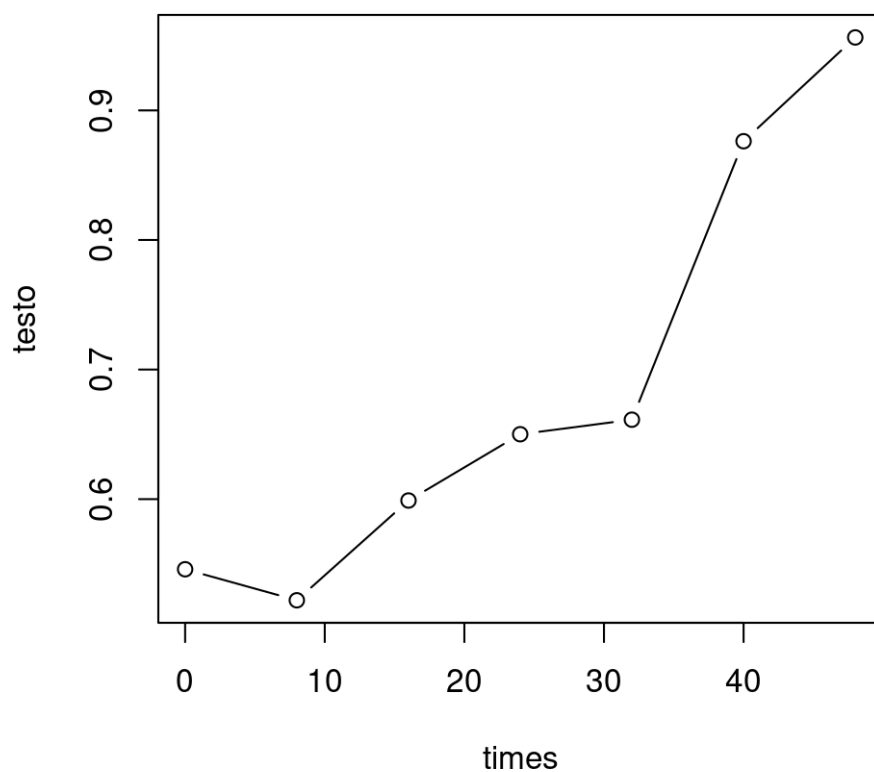
But to make a line use “l”

```
plot(times, testo, type = "l")
```



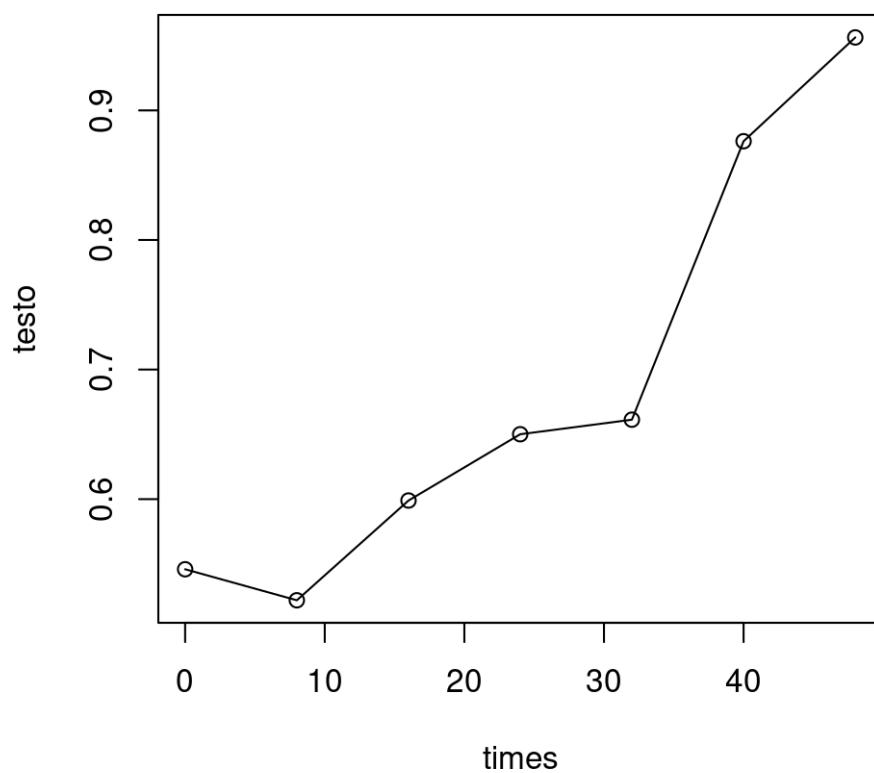
or both “b”

```
plot(times, testo, type = "b")
```



or overwritten "o"

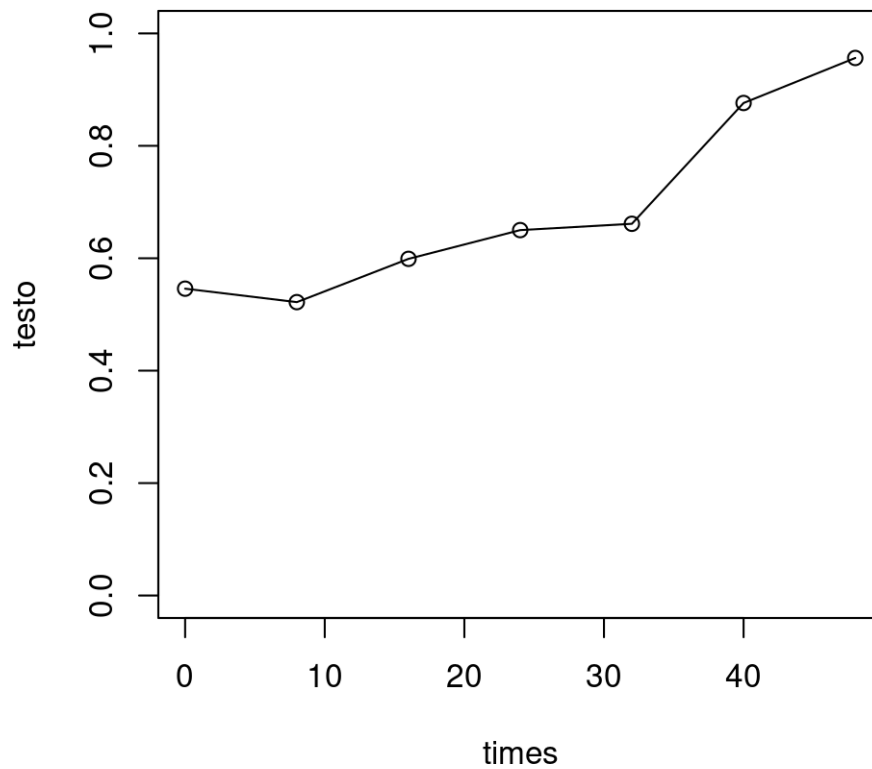
```
plot(times, testo, type = "o")
```



Try these out on your own.

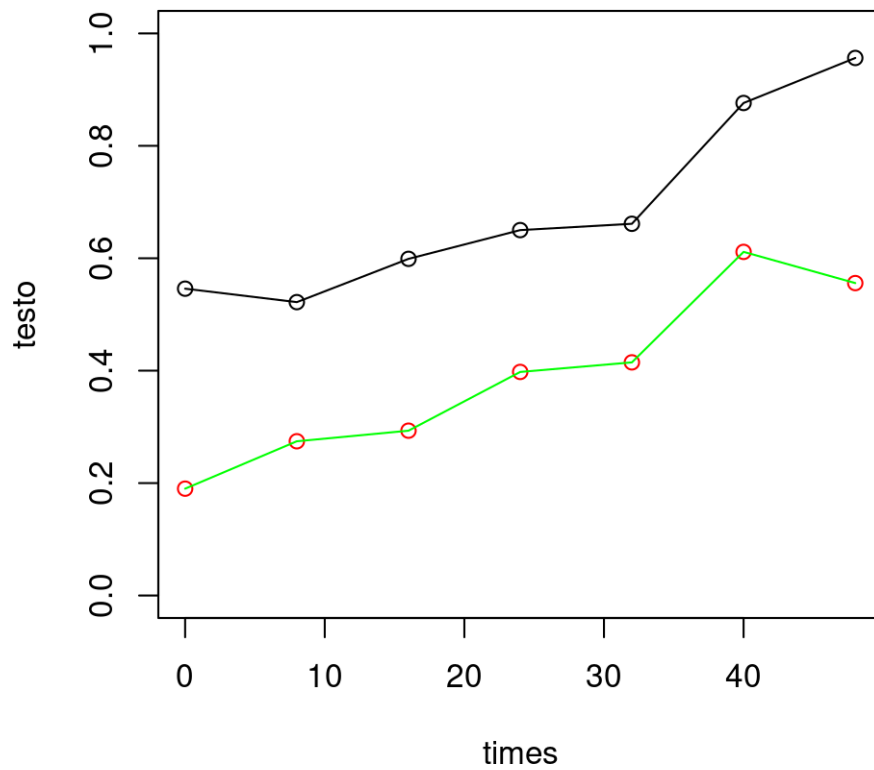
Next, to adjust the y and x axis, use `xlim` and `ylim`

```
plot(times, testo, type = "o", xlim = c(0,48), ylim = c(0,1) )
```



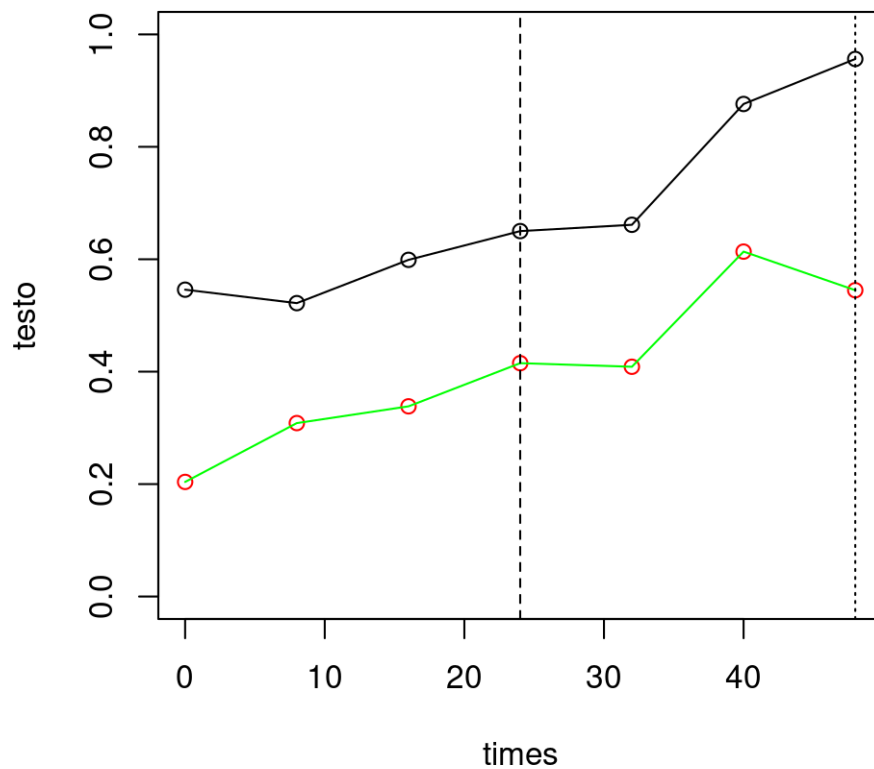
When a plot already exists (and is active), you can add other points or lines to the graph with the commands `points()` and `lines()`

```
testo2 <- seq(from = 0.2, to = 0.6, length.out = 7) + rnorm(7,0,0.05)
points(times, testo2, col = "red")
lines(times, testo2, col = "green")
```



You can add lines to your plot with `abline()` as we had done earlier, and there are 6 lines styles to adjust with the parameter `lty`.

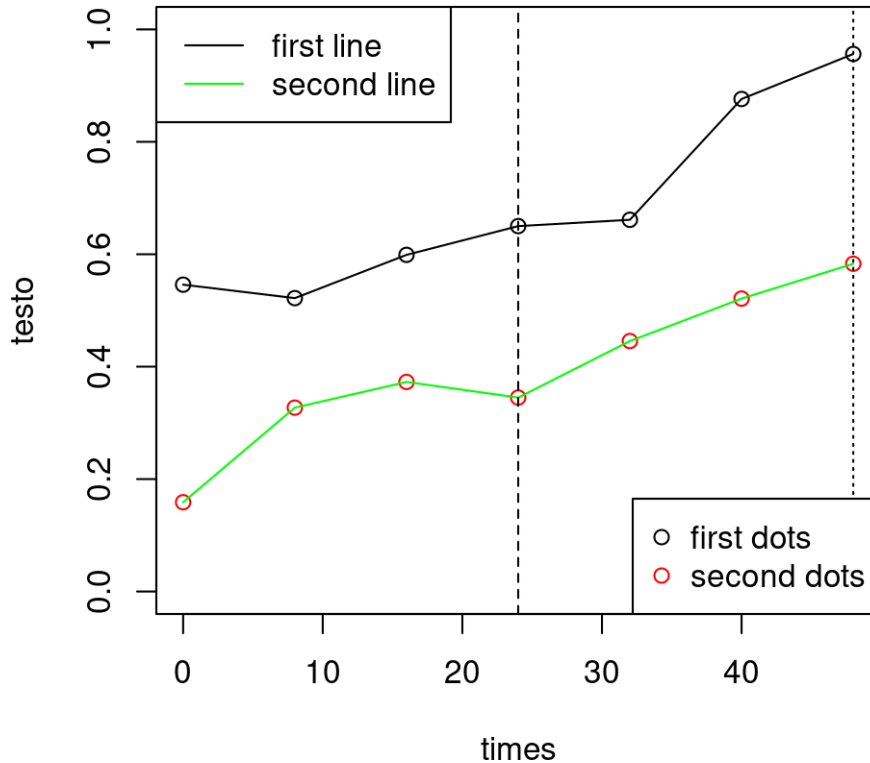
```
abline(v = 24, lty = 2) #vertical line at 24 hours, dashed
abline(v = 48, lty = 3) #vertical line at 48 hours, fine dashed
```



13.2 A legend from your own mind

You can add a legend to the plot very easily, and it is highly customizable also.

```
legend("topleft", c("first line", "second line"), lty = c(1,1), col = c("black","green"))
legend("bottomright", c("first dots", "second dots"), pch = c(1,1), col = c("black","red"))
```



13.3 Opening a new plot window

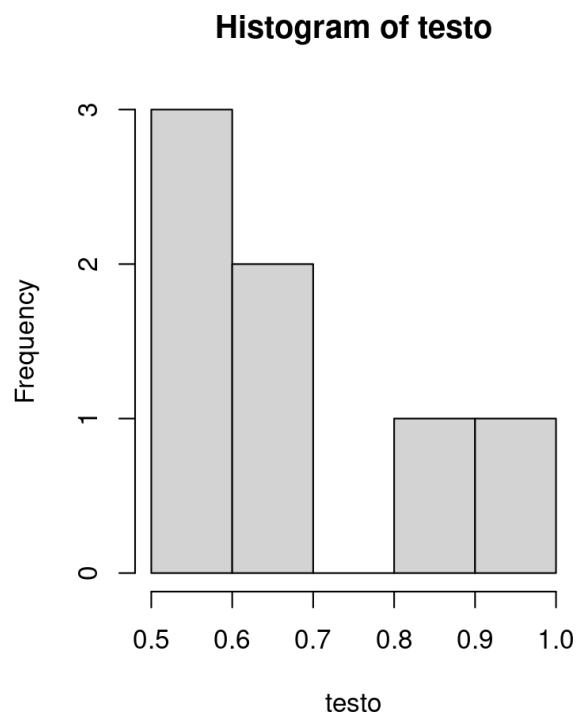
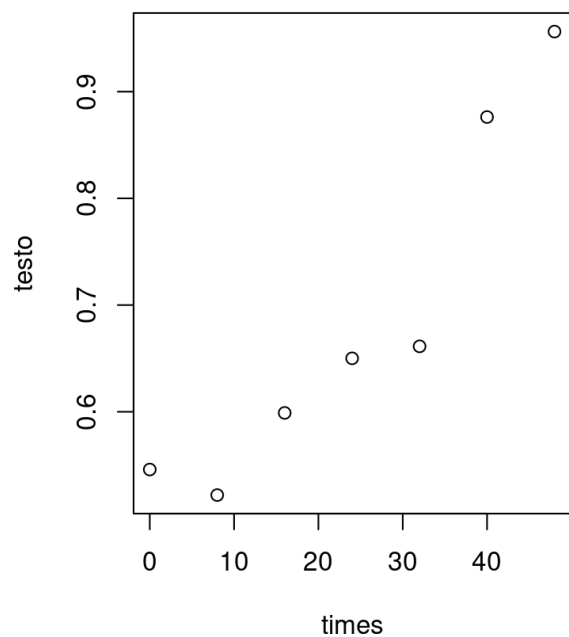
If you want to open a new plot window to work on, type `quartz()` on Macs and type `x11()` on Windows and Linux. X11 is the name of the Unix windowing system, which is where the command's name comes from. To close an active plot window, type `dev.off()`.

13.4 Putting more than one plot in a single figure.

To create multiple plotting environments in a single window, you can use the `par` command.

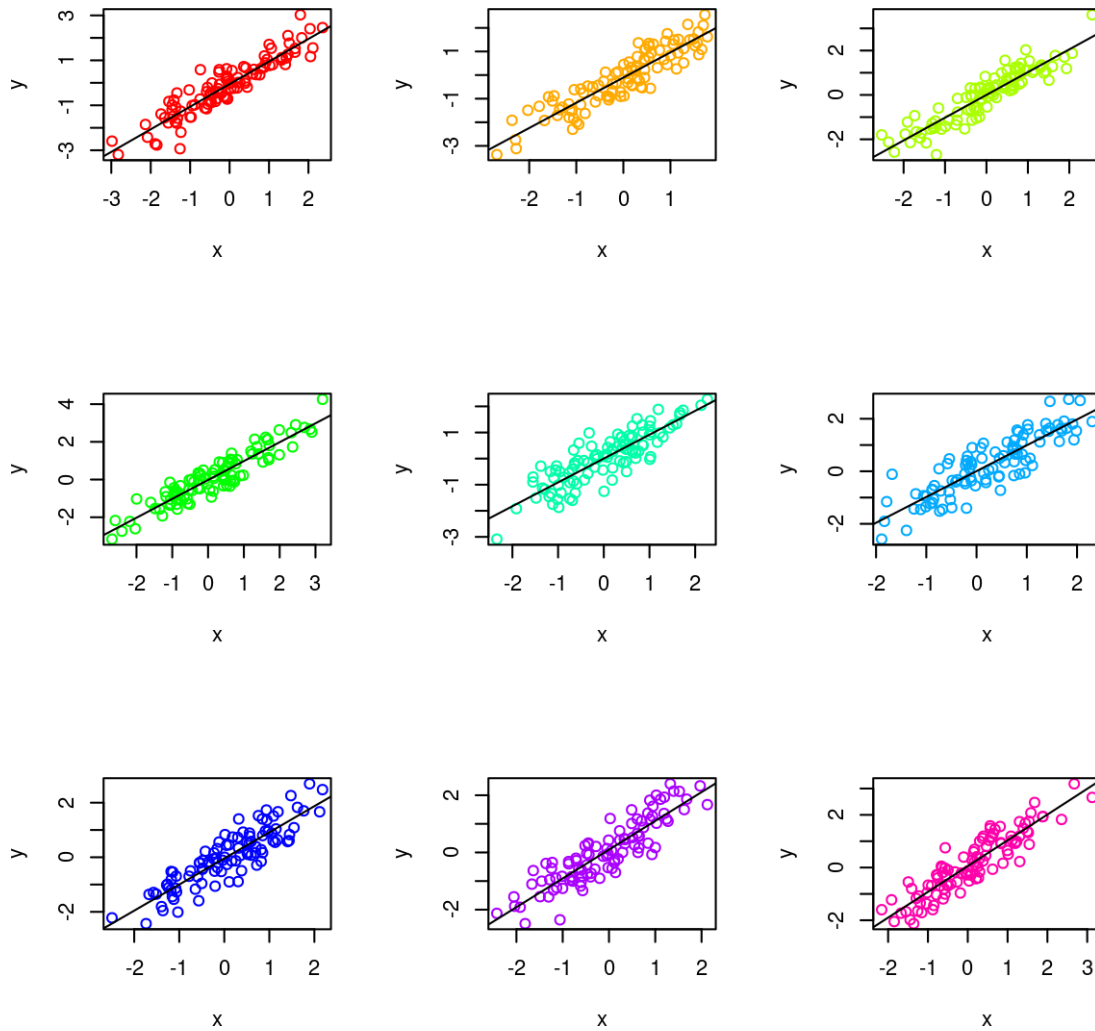
For example `par(mfrow = c(1,2))` creates a 1 row, 2 column plotting environment. Then if you make two consecutive plots, they will fill the two environments.

```
par(mfrow = c(1,2))
plot(times, testo)
hist(testo)
```



Use the mouse to adjust the plot window and see what will happen. You can put as many plots are you like.

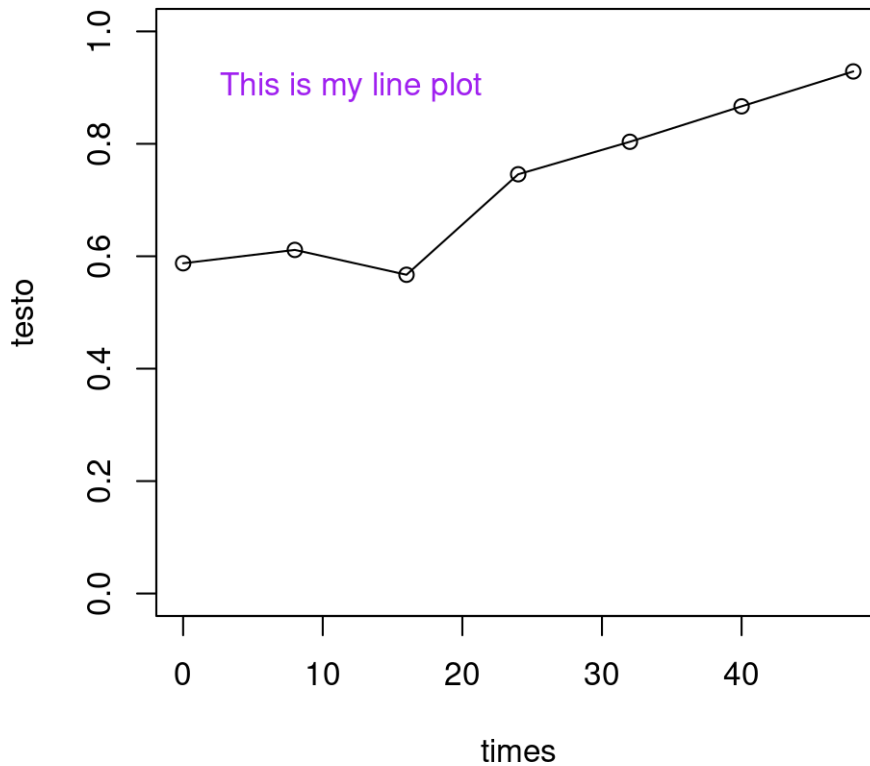
```
par(mfrow = c(3,3))
colors <- rainbow(9)
for (i in 1:9){
  x <- rnorm(100,0,1)
  y <- x + rnorm(100,0,0.5)
  plot(x,y,col = colors[i])
  abline(lm(y ~ x))
}
```



13.5 Adding text to a graph

You can easily add text to a graph with the `text()` command. The syntax is `text(x,y,"what you want to say")`. You can adjust whether the text is left or right justified to the starting point, its size, its color, its rotation and its font.

```
times <- seq(from = 0, to = 48, by = 8)
testo <- seq(from = 0.5, to = 0.9, length.out = 7) + rnorm(7,0,0.05)
plot(times,testo, xlim = c(0,48), ylim = c(0,1), type = "o")
text(12, 0.9, "This is my line plot", col = "purple", cex = 1)
```



13.5.1 Exercise

- `test_data.csv` is method comparison data between the Siemens Advia Centaur and Deborah French's LC-MS/MS testosterone method. Read the data in `test_data.csv`.
 - Examine the data with `head()`.
 - Produce a scatter plot and a difference plot side by side using the nmol/L data or the ng/dL data as you prefer.
 - Make the color of the points green and semitransparent in `pch = 16`.
 - Label the x and y-axes appropriately showing the units you have chosen using `xlab` and `ylab`.
 - Add an appropriate title using the `main` parameter.
 - Add a regression line—any type you like and the color of your choosing. Use `abline()` and `lm()` or `mcreg()`.
 - Add the line of identity in a dashed red line using `abline()` and the `lty` parameter.
 - Put a legend in the top left hand corner identifying the regression line and the line of identity specifying its color and type of dashing. Use `legend()`.
 - Below the regression equation put a statement about the correlation coefficient.
 - Write the regression relationship on the graph with the `text()` function.
 - Produce an appropriately proportioned difference plot with difference expressed as %.
 - Add the mean difference line.
 - Add the $\pm$ 2SD lines
 - Put the two figures on the same image with `par(mfrow = c(1,2))`.
 - Save them as a png, 300 dpi, 12 inches across, 6 inches deep.
 - Save also as a pdf.